

Computer Programming for Civil Engineering

Dr. Bilal Tayfur

May 4, 2026

This document is prepared for students of the Civil Engineering Department. The Python programming language is used in this course. However, the course is not about Python programming. It is about computer programming in general. In addition, some other programming languages are also used to demonstrate the some specific concepts. The main objective of this course is to introduce students to the basic concepts. Python programming language is selected because of its simplicity and ease of use. If you are familiar with any other programming language, you can use it to solve all exercises. The AI usage is allowed for students. However, in learning process, its not recommended.

With recent developments in computer science, everything changes rapidly. So, the things that will be mentioned here only valid for a short period of time. However, if you are focused on learning programming for engineering purposes, this course and Python language will be useful for you. Python makes some concepts (such as variables, arrays, data types etc.) easier to understand. But if you are interested in learning programming for "pure programming" purposes, maybe you should start with other programming languages. At some point, you can switch your focus to Python, anyway, the previous knowledge that you gain from more complex programming languages will be very useful for you.

Additional Resources

- <https://www.python.org/>
- <https://books.goalkicker.com/PythonBook/PythonNotesForProfessionals.pdf>
- <https://projecteuler.net/>
- Kutlu Darılmaz, Yapı Mühendisliği Örnekleriyle C++ Kullanımına Giriş, Birsen Yayınevi

Contents

1	Basic Concepts of Computer Programming	2
1.1	What is computer programming?	2
1.2	Why computer programming?	2
1.3	How to learn computer programming?	2
1.4	Why do we use programming languages in civil engineering?	3
1.5	How do programming languages work?	3
2	Introduction to Python	5
2.1	What is Python?	5
2.2	Why Python for Civil Engineering?	5
2.3	How to install Python?	6
2.3.1	Setting up VSCode (recommended editor)	6
2.4	How to run Python programs?	6
2.5	First Steps in Coding	7
2.6	The Interactive Shell (REPL)	8
2.7	Comments and Docstrings	9
	Summary	10
3	Introduction to Algorithms	11
3.1	What is an Algorithm?	11
3.2	Properties of a Good Algorithm	11
3.3	Flowcharts	11
3.3.1	Basic Flowchart Symbols	11
3.3.2	Tools for Flowcharts	12
4	Variables in Programming	14
4.1	Why we need variables?	14
4.2	Rules for Naming Variables	14
4.3	Variables and Memory Management	14
4.3.1	Volatile vs Non-Volatile Memory	14
4.3.2	How Variables are Stored (Bits and Bytes)	15
4.3.3	From Bits to Decimal: Variable Types Side by Side	15
4.4	Standard and Engineering Data Types	17
4.5	Efficiency and Data Types	18
4.5.1	Memory Footprint	18
4.5.2	Computational Speed	18
4.5.3	Precision vs Exactness	18
4.6	Type Conversion (Casting)	18
4.7	Checking Types: <code>type()</code> vs <code>isinstance()</code>	19
4.8	Introduction to Functions	20
	Summary	22
5	Operators and Expressions	23
5.1	Arithmetical operators	23
5.2	Assignment operators	24
5.3	Comparison operators	24
5.3.1	Chained comparisons	25
5.3.2	Identity vs Equality (<code>is</code> vs <code>==</code>)	25
5.4	Operator Precedence	25
5.5	Short-Circuit Evaluation	25
5.6	Logical operators	26

5.7	Bitwise operators	26
	Summary	29
6	Control Flow - Part 1 : If-Else Statements	30
6.1	Conditional Statements	30
6.1.1	If-Else Statements	30
6.1.2	Nested <code>if</code>	31
6.1.3	The Ternary (Conditional) Expression	31
6.1.4	Truthy and Falsy Values	31
6.1.5	Switch Statements (Match-Case)	31
	Summary	34
7	Control Flow - Part 2: Loops	35
7.1	For Loops	35
7.1.1	The <code>range()</code> function	35
7.1.2	<code>enumerate()</code> — when you need both the index and the value	36
7.1.3	<code>zip()</code> — iterate over two (or more) sequences in parallel	36
7.1.4	Nested Loops — iterating over 2D data	36
7.1.5	The Loop <code>else</code> Clause (Python-specific)	36
7.2	While Loops	36
7.3	Do-While Loops	37
7.4	Loop Control Statements	37
	Summary	39
8	Data Structures and Libraries	41
8.1	Lists	41
8.1.1	Indexing and Slicing	41
8.2	Dictionaries	42
8.3	List Methods at a Glance	44
8.4	Tuples	44
8.4.1	Tuple Unpacking and Multiple Return Values	44
8.5	Sets	45
8.6	List & Dict Comprehensions	45
8.7	Mutable vs Immutable: A Deep Trap	45
8.8	Introduction to Libraries	47
8.8.1	Why use libraries?	47
8.8.2	The Python Package Index (PyPI) and <code>pip</code>	48
8.8.3	Commonly Used Engineering Libraries	48
	Summary	50
9	Math, Arrays, and Dataframes	51
9.1	Math with Python	51
9.1.1	The <code>math</code> Module	51
9.2	Introduction to Arrays (NumPy)	53
9.2.1	Why Arrays?	53
9.2.2	Common NumPy Functions	53
9.2.3	Indexing, slicing, reshaping	54
9.2.4	Boolean indexing — the most useful trick	54
9.2.5	Broadcasting — arithmetic on different shapes	54
9.2.6	Matrix multiplication	55
9.3	Introduction to DataFrames (Pandas)	56
9.3.1	Power of Pandas	56

9.3.2	<code>loc</code> vs <code>iloc</code>	57
9.3.3	<code>groupby</code> and <code>apply</code>	57
Summary		58
10	Strings, Files, and Regular Expressions	60
10.1	String Manipulation	60
10.1.1	Quote styles, escapes, and raw strings	60
10.1.2	Key Methods	60
10.1.3	Formatting Reports (f-strings)	60
10.2	File Input/Output (I/O)	62
10.2.1	The <code>with open(...)</code> Pattern	62
10.3	Regular Expressions (Regex)	63
10.3.1	Common Patterns	63
Summary		67
11	Functions and Error Handling	68
11.1	What is a Function?	68
11.1.1	The Factory Analogy	68
11.1.2	Why use functions?	68
11.2	Anatomy of a Function	68
11.2.1	Return vs Print	68
11.3	Variable Scope (Local vs Global)	69
11.4	Advanced Function Arguments	70
11.4.1	Default Arguments	70
11.4.2	Flexible Arguments (<code>*args</code>)	71
11.4.3	Keyword Arguments (<code>**kwargs</code>)	72
11.4.4	The Default-Mutable-Argument Trap	73
11.4.5	Type Hints	73
11.5	Error Handling (Try / Except)	74
11.5.1	The Analogy: Runaway Truck Ramp	74
11.5.2	The <code>finally</code> Block	75
11.5.3	Raising Errors Yourself (<code>raise</code>)	75
11.6	Reading a Traceback	76
11.7	The <code>if __name__ == "__main__":</code> Idiom	77
Summary		78
12	File Handling and CSVs	79
12.1	Basic File Operations	79
12.2	The CSV Format	79
12.2.1	Why not just use Excel (.xlsx)?	79
12.3	Reading CSV Files	79
12.3.1	1. The <code>csv.reader</code>	80
12.3.2	2. The <code>csv.DictReader</code> (Pro Move)	81
12.4	Application: Statistical Analysis	81
12.5	Pandas vs the <code>csv</code> module	82
12.6	File Paths the Right Way: <code>pathlib</code>	82
12.7	JSON — the API/Web Data Format	82
12.8	Excel Files (.xlsx)	84
Summary		84

13 Data Visualization	86
13.1 Libraries	86
13.2 Two Plotting Styles: Pyplot vs Object-Oriented	86
13.3 Customisation Cheatsheet	86
13.4 Distributions (Histograms)	87
13.5 Correlations (Scatter Plots)	87
13.6 Comparing Groups (Box Plots)	87
13.7 The Engineering Dashboard (Subplots)	87
13.8 Two Civil-Engineering Classics	91
13.8.1 Force–Displacement Curve (Pushover-Style)	91
13.8.2 Twin-Axis Plot (Force and Displacement vs Time)	91
Summary	93
14 Importing Data from Online Sources	94
14.1 HTTP in 60 seconds	94
14.2 The <code>requests</code> library	94
14.3 From JSON to Pandas	94
14.4 Robust API Calls	94
14.5 Application: USGS Earthquake Feed	95
Summary	96
14.6 Practice (Project Euler)	97
15 Structural Analysis with Python	98
16 Advanced Programming Concepts	102
16.1 Memory and Pointers	102
16.1.1 Memory as a Grid of Addresses	102
16.1.2 What is a Pointer?	103
16.1.3 Why are Pointers Important?	103
16.1.4 Pointers in Python?	103
16.2 Object-Oriented Programming (OOP)	103
16.2.1 Core Concept: Class and Object	103
16.3 The Four Pillars of OOP	104
16.3.1 1. Encapsulation	104
16.3.2 2. Inheritance	104
16.3.3 3. Polymorphism	104
16.3.4 4. Abstraction	105
16.4 Recursion	105
16.5 Algorithm Complexity (Big O Notation)	105
16.6 Parallelism and Multithreading	106
16.7 Version Control (Git)	106
16.8 APIs (Application Programming Interfaces)	106
16.9 Debugging and Testing	107
16.9.1 Common Debugging Techniques	107
16.9.2 Testing	107
Appendix — Quick Reference	108
A. String Methods	108
B. List, Tuple, Set, Dict Methods	109
C. f-string Format Mini-Language	110
D. NumPy / Pandas Quick Reference	111
E. Common Python Errors	112

F. Programming Glossary	114
-----------------------------------	-----

Basic rules

Decimal points are always ”.” Most of the programming languages use ”.” as the decimal point. This is strict rule and there is a reason for this. The comma is usually used as parameter separator in many programming languages. If we use the comma as decimal point, it will create a lot of confusion.

1 Basic Concepts of Computer Programming

1.1 What is computer programming?

Computer programming (or coding) is the process of designing, writing, testing, and maintaining an executable computer program to accomplish a specific computing result or perform a predefined task. It involves writing instructions in a formal language that computers—or, more accurately, interpreters or compilers—can understand and execute.

Think of it as writing a very precise recipe: you start with raw ingredients (input data), follow a step-by-step logical sequence of instructions (algorithm), and eventually cook a meal (the solution or output). Unlike human cooks who can use their intuition, computers require exact, unambiguous instructions to function correctly.

Moreover, programming is not just about writing lines of code; it is fundamentally about problem-solving. It requires translating a real-world problem into a logical and mathematical abstraction that a machine can process.

1.2 Why computer programming?

Programming empowers us to transition from merely using software to actually creating our own tools and solutions. For an engineer, it provides the ability to:

1. **Automate Repetitive Tasks:** Perform tedious calculations instantly. Instead of manually calculating the bearing capacities of 1000 different foundations or extracting forces from hundreds of structural elements, a few lines of code can do it in seconds.
2. **Solve Complex and Scalable Problems:** Handle large-scale numerical computations that are impossible to do manually, such as Finite Element Analysis (FEA), Computational Fluid Dynamics (CFD), or complex optimization problems in structural design.
3. **Process and Analyze Data:** Extract valuable insights from vast amounts of data. Civil engineering today involves big data: traffic flow analyses, sensor readings from Structural Health Monitoring (SHM), LiDAR point clouds, and GIS datasets.
4. **Parametric Design:** Generate and evaluate hundreds of design alternatives rapidly by simply changing a few input variables, allowing engineers to find the most economical and safe designs.

1.3 How to learn computer programming?

Learning to program is very similar to learning a new spoken language or a musical instrument. It requires developing a new way of thinking, often called "computational thinking". Key principles include:

- **Practice Over Theory:** Writing code is the only way to learn code. Watching tutorials or reading books without hands-on practice will not build the necessary muscle memory.
- **Algorithmic Problem Solving:** The most crucial skill is breaking down a massive, overwhelming problem into smaller, logically ordered, and manageable steps.
- **Resilience and Patience:** Debugging (finding and fixing errors) is a major, inevitable part of the process. Even experienced developers spend more time debugging than writing new code. Do not get frustrated by errors; they are your feedback mechanism.
- **Consistency:** Practicing 30 minutes every day is much more effective than cramming for 5 hours once a week.

- **Not yielding to frustration:** Getting stuck is an inevitable part of the learning process. Do not give up and ask for help when you need it.
- **Learn with an aim:** You need a project idea to work on. Videos, tutorials, and books follow a certain structure and line. However the things you read from them will fly away immediately. But if you have a project to work on, you will need tools and process. In my experience, it is the best way to learn programming.

Note

Reading and understanding other people's code is as important as writing your own. Try to trace the logic and understand what every single line of an example does before modifying or running it.

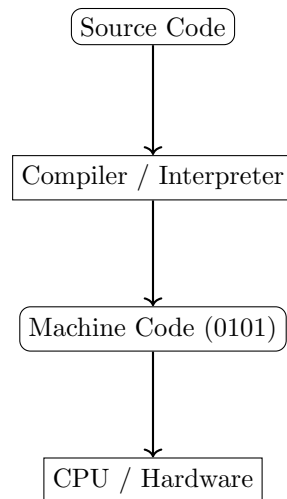
1.4 Why do we use programming languages in civil engineering?

While commercial engineering software products (like SAP2000, ETABS, AutoCAD, Plaxis) are incredibly powerful, they are closed ecosystems designed for generic use-cases. Programming bridges the gaps:

- **Customization and Interoperability:** Using APIs (Application Programming Interfaces) to automate commercial software, or to make two different software platforms "talk" to each other (e.g., extracting geometry from Rhino/Grasshopper and sending it to SAP2000).
- **Developing Proprietary In-House Tools:** Converting vulnerable and hard-to-maintain spreadsheet calculations into robust, version-controlled desktop or web applications tailored to a company's specific workflow.
- **Advanced Research:** Developing new constitutive material models, simulating complex soil-structure interactions, applying Machine Learning algorithms for concrete compressive strength prediction, or creating novel seismic analysis algorithms.
- **Reproducibility:** A script acts as a perfect record of your methodology. Unlike clicking around in a graphical user interface (GUI), a code script can be reviewed, verified, and rerun by anyone with the exact same results.

1.5 How do programming languages work?

At their core, computer processors (CPUs) only understand "Machine Code" (binary instructions consisting of 0s and 1s). Since writing in binary is practically impossible for humans, programming languages were invented to allow us to write logic in a more human-readable format, known as "Source Code". A translation layer then converts this source code into machine instructions.



Programming languages generally fall into two translation categories:

- **Compiled Languages (e.g., C++, Fortran, Rust):** The source code is translated entirely into machine code upfront before the program runs. This results in highly optimized and blazing fast execution speeds, but requires recompiling every time a change is made.
- **Interpreted Languages (e.g., Python, MATLAB):** The source code is read and executed line-by-line while the program is running. This makes testing and debugging very fast, but execution speed is generally slower than compiled languages.

The performance differences between programming languages are directly related to this translation process. Usually, the "lower" the level of the language (closer to hardware), the faster it executes. Therefore, native Python is not the strongest choice for pure, raw computational speed. However, with modern hardware advancements and the fact that most mathematically intensive Python libraries (like NumPy) are actually written in highly optimized C or Fortran under the hood, this performance gap is rapidly diminishing. The trade-off is often "Developer Time vs. Execution Time": Python saves you hours in writing the code, even if it takes a few seconds longer to run it.

2 Introduction to Python

2.1 What is Python?

Python is a high-level, interpreted, general-purpose programming language renowned for its elegant syntax, remarkable readability, and an incredibly vast ecosystem of third-party libraries. Created by Guido van Rossum and first released in 1991 (it is older than you might think!), it has fundamentally shaped modern scientific and software development.

One of Python's most defining characteristics is its use of **significant indentation**. Unlike languages like C++ or Java that use curly brackets `{ }` to define blocks of code, Python relies on whitespace (indentation). This essentially forces developers to write cleanly formatted code. While this strict spacing rule can be confusing for a beginner initially, it quickly becomes natural and makes Python code exceptionally easy to read and maintain. Every programming language has its unique syntax quirks, and mastering Python's learning curve requires hands-on practice. Python is widely considered one of the easiest languages for beginners, yet it has the depth required to build professional, enterprise-grade systems.

Python is often referred to as a "scripting language". A script is generally a small automation program designed for a specific environment. However, Python has evolved far beyond simple scripts; it is a full-fledged Object-Oriented programming language used to build massive web applications, artificial intelligence models, and complex engineering simulations.

Note

The concept of "level" in programming languages is sometimes misunderstood by beginners. It does not dictate the "difficulty" of learning the language, but rather its level of abstraction from the computer hardware. If a language syntax closely resembles human language and math, it is "high-level" (Python, MATLAB). If it closely relates to memory management and CPU registers, it is "low-level" (Assembly, C).

2.2 Why Python for Civil Engineering?

In the modern engineering landscape, Python has emerged as the dominant programming tool, largely replacing older paradigms. The reasons are numerous:

1. **Unmatched Readability:** Python code reads almost like a mathematical pseudo-code or plain English. This is incredibly valuable for engineers who are not full-time software developers; you can look at a 6-month-old Python script and still understand your underlying logical decisions.
2. **The Powerful Ecosystem (Libraries):** Python's true strength lies in its libraries—bundles of code developed by the community that you can import and use instantly. Some critical ones for engineering include:
 - **NumPy / SciPy:** The foundation for advanced numerical computations, linear algebra, matrix operations, and optimization.
 - **Pandas:** The ultimate tool for importing, cleaning, and manipulating large datasets (such as messy Excel spreadsheets or sensor logs).
 - **Matplotlib / Plotly:** For generating publication-quality static 2D/3D graphs or interactive dashboards.
 - **OpenSeesPy:** A Python interface for the OpenSees framework, widely used for advanced earthquake engineering and structural behavior simulations.
 - **Scikit-Learn / PyTorch:** For integrating Machine Learning and AI models into civil engineering predictions.

3. **Community and Open-Source Support:** Python is completely free and open-source. Because of its massive global user base, if you encounter an error or need a specific algorithm (like finding a specific numerical root), someone else has almost certainly solved it and shared the solution online. This "stack-overflow" culture shouldn't be underestimated.

2.3 How to install Python?

There are multiple ways to set up a Python environment. The most comprehensive method for scientific computing has historically been installing a comprehensive distribution like **Anaconda**, which comes pre-packaged with Python and hundreds of data science libraries out of the box. However, its immense size and complexity can be slightly overwhelming and bloated for absolute beginners.

A leaner, modern approach is to install standard Python directly and pair it with a powerful text editor equipped for code, known as an Integrated Development Environment (IDE), or an AI-assisted editor like **VSCode**, **Cursor**, or **Antigravity**. These tools provide features like syntax highlighting, automatic error checking, and even AI code generation, making the learning curve much smoother.

For Windows users (using Windows Package Manager):

```
winget install python
```

For Linux users (Ubuntu/Debian-based):

```
sudo apt update
sudo apt install python3
```

For Mac users (Homebrew is required):

```
brew install python
```

2.3.1 Setting up VSCode (recommended editor)

For this course you need a single working setup. The recommendation is Visual Studio Code, which is free and works identically on Windows, Mac, and Linux.

1. Install VSCode from code.visualstudio.com.
2. Open VSCode, click the *Extensions* icon on the sidebar (or press **Ctrl+Shift+X**).
3. Search for **Python** (publisher: Microsoft) and click *Install*. This single extension brings syntax highlighting, run-buttons, and the debugger.
4. Open a folder for your course (**File** → **Open Folder**), create a file `hello.py`, type `print("Hello, civil engineering!")`, and press the green run-arrow at the top right. The output appears in the integrated terminal.

If the run-arrow is missing, VSCode hasn't found Python yet. Open the command palette (**Ctrl+Shift+P**) → *Python: Select Interpreter* and pick the version you installed.

2.4 How to run Python programs?

Python code can be executed in several distinct environments depending on your current objective:

1. **Interactive Mode (REPL):** Typing Python commands one by one directly into a terminal or command prompt, getting immediate output line-by-line. Great for quick calculations.
2. **Script Mode (.py files):** Writing complete blocks of code in a simple text file with a `.py` extension and executing the entire file at once. This is the standard software development method.

3. **Jupyter Notebooks (.ipynb files):** An interactive, web-based environment where you can mix executable code blocks, styled markdown text, mathematical equations, and inline plots in a single cohesive document. This format is heavily favored by engineers, researchers, and data scientists because it behaves exactly like an interactive calculation report.

To run a basic Python script from your terminal:

```
python script.py
```

or

```
python3 script.py
```

Note

Depending on your Operating System and environment variables, the command executing python might be `python` or `python3`. This relates to your system's PATH configuration, which communicates to your terminal where exactly the Python executable physically resides on your hard drive. If the command is not recognized, it can be resolved by correctly configuring the environment path.

Managing Python Versions and Environments: As of today, the modern era of Python is strongly situated in the 3.12+ releases. Python libraries evolve rapidly, and they often depend heavily on the specific Python version you are using. It is a good practice to verify your installation. To check your installed Python version, execute:

```
python --version
```

Occasionally, a specific engineering library might not support the absolute newest Python release immediately. For beginners, encountering version conflicts is a common hurdle. Instead of forcefully trying to install or downgrade Python through pure pip commands—which often leads to a corrupted base python installation—engineers use **Virtual Environments** (via tools like `venv` or `conda`). A virtual environment acts as an isolated, independent Python workspace crafted for a specific project. This means Project A can run securely on Python 3.9 with older stable libraries, while Project B uses Python 3.12 with bleeding-edge tools, all on the same computer without ever conflicting. We will explore creating virtual environments deeply later when we deal with setting up OpenSeesPy.

2.5 First Steps in Coding

Let's analyze the traditional "Hello World" example, conventionally the very first program anyone writes when learning a new language.

Example (Hello World).



This concise program commands the computer to print a text message directly to the screen.

```
print("Hello , Civil Engineering Students!")  
print("Welcome to Computer Programming course.")
```

Now let's examine a vastly more practical, albeit simple, engineering calculation.



Example (Foundation Area).

Calculates the required area of a rectangular footing based on geometric inputs.

```
# Calculate the area of a foundation footing
width = 2.5 # meters
length = 3.0 # meters

area = width * length

print("Footing Width:", width, "m")
print("Footing Length:", length, "m")
print("Footing Area:", area, "m^2")

# The difference between a calculator and a programming
# script lies in here. Please notice we didnt use dimensions
# in the calculation. We used variables instead. In advance,
# we can code this script like following:

#read inputs from user
width = float(input("Enter footing width (m): "))
length = float(input("Enter footing length (m): "))

#calculate area
area = width * length

#print results
print("Footing Width:", width, "m")
print("Footing Length:", length, "m")
print("Footing Area:", area, "m^2")
```

As you can observe in the provided script, we generally avoid hard-coding absolute numbers into our core variables unless necessary. We use variables essentially as empty logical containers built to store and process dynamic values provided programmatically or entered by the user. This showcases the true power of automation: writing generalized logic once, enabling you to solve an infinite spectrum of input scenarios without rewriting code.

Note

Notice the **#** symbol in the code. This character initiates a **comment**. The computer compiler simply ignores everything appearing after the **#** on that respective line. Comments are intentionally drafted exclusively for humans reading the code! Professional developers write comments not to narrate *what* the code is obviously doing, but to clarify *why* specific structural decisions were made.

2.6 The Interactive Shell (REPL)

Before writing scripts, you can open the Python REPL ("Read-Evaluate-Print Loop") to try expressions one at a time:

```
$ python
Python 3.12.x ...
>>> 2 + 3
5
>>> beam_length = 6.0
>>> moment = 50 * beam_length / 8
>>> moment
37.5
>>> exit()
```

Use the REPL whenever you want to verify the syntax of a single line, check what a function returns,

or do quick arithmetic. The same window in VSCode is opened by typing *Python: Start REPL* from the command palette.

2.7 Comments and Docstrings

Three styles of in-code documentation appear from now on:

```
# Single-line comment: a quick remark about the next line.
```

```
# Multi-line "comment block": just stack #-lines.
```

```
# Used for short paragraphs of explanation.
```

```
"""Module-level docstring.
```

```
A triple-quoted string at the top of a file or function
that documents what it does. Tools like help() read this.
"""
```

Default to writing *no comment* unless the WHY is non-obvious. Code with too many trivial comments is harder to maintain than code with none.

Question 1

1. What does the following snippet print?

```
print(2 + 3)
print("2 + 3")
```

Line 1: _____, line 2: _____.

2. Compiled vs interpreted: which group does Python belong to, and which trade-off does that imply?
3. After installing Python with `winget install python`, the command `python --version` fails in PowerShell. What is the most likely cause, and how do you fix it without re-installing?

Solution:-

1. 5 on the first line; the literal text `2 + 3` on the second — Python evaluates the expression but prints the string verbatim.
2. Interpreted (line-by-line at run time). Trade-off: faster iteration and shorter scripts vs. slower raw execution compared to compiled languages.
3. Python is installed but not on the system `PATH`. Either restart the terminal so `PATH` refreshes, or add the install directory to `PATH` via Windows environment variables.

Common Errors

```
# 1) python: command not found (terminal error)
# Cause: Python not on PATH.
# Fix: reopen the terminal, or use 'py' on Windows.

# 2) SyntaxError: EOL while scanning string literal
print("Hello)          # missing closing quote
# Fix: add the missing "
```

Summary

1. Programming is precise problem-solving: translate a real problem into instructions a computer can follow without ambiguity.
2. Engineers use Python to automate repetitive calculations, integrate commercial software, and analyse data at scale.
3. Compiled vs interpreted languages trade execution speed for development speed. Python is interpreted but uses C-backed libraries (NumPy) under the hood for speed.
4. Three ways to run Python: the REPL, a `.py` script, or a Jupyter notebook. Each has its place.
5. Comments document *why*, not *what*; default to writing none unless a future reader would be confused.

3 Introduction to Algorithms

3.1 What is an Algorithm?

Before we write a single line of code, we must strictly define the problem we are trying to solve. An **algorithm** is a well-defined, step-by-step procedure or a set of rules to be followed in calculations or other problem-solving operations.

Think of an algorithm like a **cooking recipe** or a **construction method statement**. If you are pouring concrete for a column, you don't just "pour concrete". You follow a specific sequence:

1. Build the formwork.
2. Place the reinforcement bars (rebars).
3. Mix the concrete.
4. Pour the concrete into the formwork.
5. Vibrate the concrete to remove air bubbles.
6. Wait for curing.
7. Remove the formwork.

If you change the order (e.g., pour concrete before placing rebars), the result is a disaster. Programming is exactly the same. The computer is a diligent worker who follows your instructions blindly. If your algorithm (instructions) is wrong, the result will be wrong (Garbage In, Garbage Out).

3.2 Properties of a Good Algorithm

A robust algorithm must satisfy the following criteria:

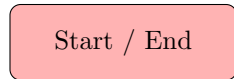
- **Input:** It should accept zero or more inputs.
- **Output:** It should produce at least one output.
- **Definiteness (Unambiguous):** Each step must be clear and lead to only one meaning. "Mix concrete until it looks good" is bad. "Mix concrete for 5 minutes" is good.
- **Finiteness:** The algorithm must terminate after a finite number of steps. It cannot run forever.
- **Effectiveness:** Every step must be basic enough to be carried out.

3.3 Flowcharts

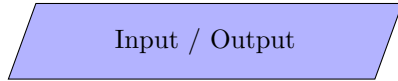
A flowchart is a visual representation of an algorithm. It uses standardized symbols to illustrate the logical flow of the program. Using flowcharts is critical because it allows engineers to discuss the logic without worrying about the specific syntax of a programming language (like Python or C++).

3.3.1 Basic Flowchart Symbols

We use the ISO 5807 standard for flowchart symbols.



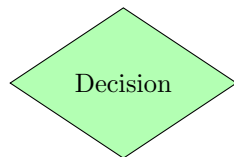
Oval (Terminal): Indicates the beginning or the end of a program flow.



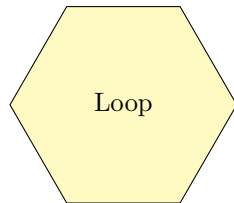
Parallelogram (Input/Output): Denotes asking the user for information (Input) or displaying results to the screen (Output). Example: "Read Beam Length", "Print Area".



Rectangle (Process): Represents a calculation, arithmetic operation, or data assignment. Example: $Area = Width \times Height$.



Diamond (Decision): A branching point. It checks a condition (True/-False or Yes/No) and splits the flow. Example: "Is $Factor_of_Safety > 1.5$?".



Hexagon (Preparation): Represents an initialization of a loop. Example: "For $i = 1$ to 10".

3.3.2 Tools for Flowcharts

While we sketch on paper during brainstorming, professional documentation requires digital tools:

- **Draw.io (diagrams.net):** A free, open-source, web-based tool. Highly recommended.
- **Microsoft Visio:** Industry standard for corporations.
- **Lucidchart:** A popular web-based alternative.

Example (Algorithm: Concrete Strength Check). Let's design a simple algorithm to check if a concrete sample passes a strength test. **Goal:** Check if measured strength (f_{ck}) is greater than required strength (f_{req}).

Flowchart Logic:

1. **Start** the program.
2. **Input** the measured strength and required strength.
3. **Decision:** Is Measured $>$ Required?

4. **Process (Yes):** Set status to "PASSED".
5. **Process (No):** Set status to "FAILED".
6. **Output** the status.
7. **End.**

4 Variables in Programming

4.1 Why we need variables?

Imagine a solver for a structural system. You have thousands of beams, each with a length, modulus of elasticity (E), and moment of inertia (I). If you write code like:

```
stiffness = 3 * 210000 * 0.00045 / (4000 * 4000 * 4000)
```

It is impossible to understand what these numbers mean. Is 210000 the modulus E ? Or is it something else? Also, if you need to change E to 200000, you have to find and replace every instance of 210000 manually. This is error-prone.

A **variable** is a symbolic name associated with a value and whose associated value may be changed. It is a container (or a labelled box) in the computer's memory.

4.2 Rules for Naming Variables

Naming is harder than it looks. In Python, we follow these rules and conventions:

1. **Must start with a letter** or underscore (`_`). Cannot start with a number. (e.g., `1beam` is wrong, `beam1` is correct).
2. **Case Sensitive**: `Area` and `area` are different variables.
3. **No Spaces**: Use underscores (`snake_case`) to separate words. (e.g., `beam_length`, not `beam length`).
4. **Meaningful Names**: Use `elastic_modulus` instead of `e` or `x`.
5. **Reserved Keywords**: Cannot use Python keywords as variable names. (e.g., `if`, `else`, `for`, `while`, `def`, `class`, etc.). In addition to these, stay away from using most of the standard words as variable names. If you need to use a word that can be used commonly, you can use underscores or any other characters to make it unique. For example, you can use `area_1` instead of `area` even there wont be an `area2`.

Note

Assignment Operator (=): In mathematics, $=$ means logical equality ($x = y$ means x is equal to y). In programming, $=$ is an **action**. `x = 5` means "Take the value 5 and put it into the box named `x`". `x = x + 1` is mathematically impossible, but in programming, it means "Take the current value of `x`, add 1, and save the result back into `x`".

4.3 Variables and Memory Management

To understand variables deeply, we must briefly look inside the computer hardware. When you write `x = 5`, what actually happens inside the machine?

4.3.1 Volatile vs Non-Volatile Memory

A computer has two main types of data storage:

1. **Hard Drive / SSD (Non-Volatile)**: This is your "Bookshelf". It stores your files, photos, and code scripts permanently. Data remains even when power is off.
2. **RAM (Random Access Memory) (Volatile)**: This is your "Desk". When you run a program, the computer loads the data from the hard drive to the RAM. Variables live here.

Note

Volatility: If your computer acts up and you restart it, all variables in the RAM are lost. This is why you must "save" files (write them from RAM to Disk) to keep them.

4.3.2 How Variables are Stored (Bits and Bytes)

Computers run on electricity. A switch is either ON or OFF. We represent this as 1 or 0.

- **Bit:** The smallest unit (0 or 1).
- **Byte:** A group of 8 bits (e.g., 01000001).

Everything is a number to a computer. The same value 5 takes a completely different shape depending on which *type* you store it in. Understanding this is the bridge between writing Python and understanding what your machine actually does.

4.3.3 From Bits to Decimal: Variable Types Side by Side

Below are the four primitive types you saw above, all storing a representation of "5" (or its closest equivalent), unpacked at the bit level.

Type	Value	Bits (typical)	Size
int	5	0000 0101	1 byte (8-bit storage)
float	5.0	0 10000001 01000...	4 bytes (IEEE 754 sf32)
str	"5"	0011 0101 (ASCII 53)	1 byte per char + overhead
bool	True	0000 0001	1 byte in CPython

Notice that the *same* symbol "5" is stored as 0000 0101 when interpreted as an integer but as 0011 0101 when interpreted as the character '5'. Adding 1 to an integer 5 gives 6; adding 1 to the character 5 (if you bypassed Python's safety) would give the character 6 because '5'+1 = '6' in ASCII. This is exactly why Python raises `TypeError` when you try "5" + 5.

Integer (unsigned int8) — bit pattern → base 10: Read the bits from *right to left* as powers of 2: position i is worth 2^i .

$$0101\,1010_2 = 0 \cdot 128 + 1 \cdot 64 + 0 \cdot 32 + 1 \cdot 16 + 1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 0 \cdot 1 = 90_{10}$$

Integer (signed int8, two's complement) — bit pattern → base 10: For signed numbers, the leading bit is the *sign*: 0 means positive, 1 means negative. To decode a negative value, invert all bits and add 1, then negate. Example: 1111 0011.

- Leading bit is 1 → negative.
- Invert: 0000 1100; add 1: 0000 1101 = 13_{10} .
- Therefore $1111\,0011_2 = -13_{10}$.

Decimal → signed int8: Encode -25 in two's complement.

- Take $|-25| = 25_{10} = 0001\,1001$.
- Invert: 1110 0110; add 1: 1110 0111.
- So $-25_{10} = 1110\,0111_2$.

Float (IEEE 754, conceptual sketch): A 32-bit float is split into 1 sign bit, 8 exponent bits, and 23 mantissa bits. The value is $(-1)^s \cdot 1.m \cdot 2^{e-127}$. We will not encode floats by hand, but you must internalise the consequence: *not every decimal can be represented exactly*. The classic example is 0.1, which is an infinite repeating fraction in binary, just like 1/3 is an infinite repeating decimal.

String (ASCII): A character is just an integer code. 'A' is 65, 'a' is 97, the digit-character '0' is 48 (so '5' is 53). When Python stores "5" it stores the byte 0011 0101, not 0000 0101. Modern Python uses UTF-8 by default which is ASCII-compatible for the first 128 characters and extends to the rest of the world's alphabets with multi-byte encodings.

Boolean: Logically a bool needs only 1 bit, but CPython stores it as a full integer (in fact `True == 1` and `False == 0`; you can do arithmetic with them: `sum([True, False, True]) == 2`).

Example (Same value, different bytes).



```
# Same value, different bytes
# -----
# Demonstrates how Python stores the "same" value (5) in
# different types, and how many bytes each one consumes.

import sys

# 1. Four versions of "five"
# -----
five_int    = 5
five_float  = 5.0
five_str    = "5"
five_bool   = True          # True == 1; not literally five but useful here

# 2. Show the bit pattern (where it makes sense)
# -----
# bin() returns the binary representation of an integer.
# zfill(8) pads to 8 bits so the byte boundary is visible.
print(f"int    5 -> {bin(five_int)[2:].zfill(8)}")
print(f"str    '5' -> char code {ord(five_str)} -> "
      f"{bin(ord(five_str))[2:].zfill(8)}")
print(f"bool   True -> {bin(five_bool)[2:].zfill(8)}")

# float bits require a separate trick (struct module)
import struct
float_bits = struct.pack('!f', five_float)
print(f"float  5.0 -> {float_bits.hex()} (4 bytes, IEEE 754 sf32)")

# 3. Memory footprint with sys.getsizeof
# -----
print("-" * 40)
print(f"sys.getsizeof(5)      = {sys.getsizeof(five_int)} bytes")
print(f"sys.getsizeof(5.0)    = {sys.getsizeof(five_float)} bytes")
print(f"sys.getsizeof('5')    = {sys.getsizeof(five_str)} bytes")
print(f"sys.getsizeof(True)   = {sys.getsizeof(five_bool)} bytes")

# 4. The classic float surprise
# -----
print("-" * 40)
print(f"0.1 + 0.2 == 0.3 ? {0.1 + 0.2 == 0.3}")
print(f"actual value      : {0.1 + 0.2!r}")
```


Note

Why does this matter for engineering? When you read sensor data or binary structural-analysis output (e.g. .out files from FEA software), the same bit stream can be interpreted as integers or floats depending on the format specification. Mis-reading the type silently corrupts every downstream calculation.

4.4 Standard and Engineering Data Types

We have covered the basics, but Engineering requires more.

1. **Integer (int)**: Exact whole numbers.
2. **Floating Point (float)**: Approximations of real numbers.
3. **Complex Numbers (complex)**: Numbers with real and imaginary parts ($z = a + bi$).
 - Engineering Application: Essential for **Structural Dynamics**, **Earthquake Engineering** (Frequency Domain analysis), and Electrical circuits.
 - Python Syntax: Use j instead of i . Example: $z = 3 + 4j$.
4. **String (str)**: Text data.
5. **Boolean (bool)**: Logic state (True/False). Efficient storage (can theoretically be 1 bit, though usually stored as a byte).
6. **NoneType (None)**: Represents "No Value". Useful for initializing variables that will be calculated later.

Example (Variables including Complex Numbers).



This script demonstrates extended data types including complex numbers.

```
# Variables and Data Types Example

# Integer (Whole number)
number_of_floors = 5
print("Floors:", number_of_floors, " | Type:", type(number_of_floors))

# Float (Decimal number)
beam_length = 4.5
print("Beam Length:", beam_length, "m", " | Type:", type(beam_length))

# Complex (Real + Imaginary)
# Useful for dynamics/vibrations applications
impedance = 3 + 4j
print("Impedance:", impedance, " | Type:", type(impedance))
print("Real part:", impedance.real, "Imaginary part:", impedance.imag)

# String (Text)
material_name = "Concrete (C30)"
print("Material:", material_name, " | Type:", type(material_name))

# Boolean (True/False)
is_safe = True
print("Is Structure Safe?", is_safe, " | Type:", type(is_safe))

# NoneType (Empty/Null)
# Used to represent 'no value' or initialization
```

```
result = None
print("Current result:", result, "| Type:", type(result))
```

4.5 Efficiency and Data Types

Choosing the right data type is critical for **Performance** and **Accuracy**.

4.5.1 Memory Footprint

Different types consume different amounts of RAM.

- Integers are generally compact.
- Floats require more detail (mantissa + exponent) to store the position of the decimal point.
- Strings are "expensive". Storing the number 123 as an integer takes a few bytes. Storing "123" as a string takes significantly more space because each character ('1', '2', '3') has its own code, plus overhead data.

4.5.2 Computational Speed

CPUs can process integers extremely fast. Floating-point operations take more cycles. **Optimization Tip:** If you are counting items (e.g., iterations, objects), always use Integers. Use Floats only when you need decimal precision.

4.5.3 Precision vs Exactness

- **Integers are Exact:** $1 + 1 = 2$. Always.
- **Floats are Approximate:** Computers cannot represent infinite precision (like $1/3 = 0.3333\dots$). Sometimes, $0.1 + 0.2$ might result in 0.30000000000000004 due to binary floating-point arithmetic.
- **Engineering Rule:** Never compare floats for exact equality. Do not check if `stress == 20.0`. Instead, check if it is close enough: `abs(stress - 20.0) < 0.0001`.

4.6 Type Conversion (Casting)

Since variables have types, we sometimes need to change them. This is called **casting**. The most common scenario in engineering scripting is reading input. The `input()` function always returns a **string**, even if the user types a number.

```
age = input("Enter age: ") # User types 20
# age is now "20" (String)
# next_year = age + 1 <— ERROR! String + Number
```

We must cast it:

```
age_num = int(age) # Converts "20" to 20 (Integer)
next_year = age_num + 1 # Works!
```

Example (Type Conversion).



```
# Type Conversion (Casting) Example
```

```

# String to Float
displacement_str = "12.5"
displacement_val = float(displacement_str)
print("Displacement (float):", displacement_val)

# Float to Integer
# Note: This truncates the decimal part!
load = 15.8
load_int = int(load)
print("Load (original):", load)
print("Load (int):", load_int)

# Number to String
# Useful for combining with other text
force = 100
message = "The applied force is " + str(force) + " kN"
print(message)

```

4.7 Checking Types: type() vs isinstance()

Two built-ins let you ask "what type is this?":

```

x = 5.0
print(type(x))           # <class 'float'>
print(type(x) == float)  # True
print(isinstance(x, float)) # True
print(isinstance(x, (int, float))) # True - matches either

```

Prefer `isinstance()` in real code: it accepts a tuple of acceptable types and works correctly with inheritance. `type()` is fine for printing during debugging.

Note

Walrus operator (`:=`, Python 3.8+). Sometimes you want to assign and use a value in the same expression:

```

if (n := len(loads)) > 10:
    print(f"Many loads: {n}")

```

Without the walrus you would call `len()` twice, or write a separate line. Use sparingly — clarity beats cleverness.

Question 2

Predict the output of each line. Write **Error** if it raises an exception, and name the exception type.

1. `int("2.5")` → _____
2. `int(float("2.5"))` → _____
3. `float("2.5e3")` → _____
4. `int("ten")` → _____
5. `bool("False")` → _____ (careful!)

Solution:-

1. `ValueError` — `int()` cannot parse a decimal string directly.
2. 2 — two-step cast: `string` → `float` → truncated `int`.
3. 2500.0 — scientific notation is allowed by `float()`.
4. `ValueError` — not a numeric string.
5. `True` — *any non-empty string* is truthy, including `"False"`.

Question 3

Fill in the blanks. Show your work for partial credit.

1. $0011\ 1100_2$ as an unsigned `int8` → _____₁₀
2. $1000\ 0000_2$ as a signed `int8` (two's complement) → _____₁₀
3. -25_{10} as a signed `int8` → _____₂
4. Which byte stores the character '7' in ASCII? _____
5. In Python, what does `0.1 + 0.2 == 0.3` evaluate to, and why?

Solution:-

1. $32 + 16 + 8 + 4 = 60_{10}$.
2. 1000 0000 is the most negative `int8`: invert → $0111\ 1111 = 127$, add 1 = 128, negate → -128_{10} .
3. $25 = 0001\ 1001$; invert → $1110\ 0110$; add 1 → $1110\ 0111$.
4. ASCII '7' = 55 = $0011\ 0111$.
5. **False.** Neither 0.1 nor 0.2 has an exact binary representation; their stored values plus rounding give 0.30000000000000004, not 0.3. Always compare floats with `math.isclose()`.

Exercise 4.1. Create a flowchart for an engineering problem that you are familiar with.

4.8 Introduction to Functions

The function concept and notation will be handled in future weeks, however, in learning phase we'll use some functions already. So this section is just a brief introduction to functions. As engineers, we love efficiency. We don't want to rewrite the same formula for "Area of a Circle" or "Moment of Inertia" fifty times in our code. A **Function** is a block of code which only runs when it is called. You can think of it as a "mini-program" inside your main program.

- You **define** it once.
- You **call** (use) it as many times as you like.

We interpret functions as a "Black Box":

- **Input** (Arguments): Raw materials go in.
- **Processing**: The function does some work.
- **Output** (Return): The finished product comes out.

Example (A Simple Function).



```
# Introduction to Functions
# -----
# A function is a reusable block of code.
# We define it once, and 'call' it many times.

# 1. Defining a simple function
def calculate_area(width, height):
    """Calculates area of a rectangle"""
    area = width * height
    return area

# 2. Using (Calling) the function
b1 = 3.0
h1 = 4.0
area1 = calculate_area(b1, h1)
print(f"Beam 1 (3x4) Area: {area1}")

b2 = 5.0
h2 = 0.5
area2 = calculate_area(b2, h2)
print(f"Beam 2 (5x0.5) Area: {area2}")

# 3. Built-in Functions we already use
# print(), type(), input(), int(), float() are all functions provided by Python.
print("Length of string 'Concrete':", len("Concrete"))
```

Note

We will examine Functions in detail. For now, just understand that commands like `print()`, `type()`, and `input()` are actually built-in functions provided by Python.

Common Errors

Two errors you will hit constantly in Week 2:

```
# 1) NameError: name 'beam_length' is not defined
beam_lenght = 5.0          # typo
print(beam_length)         # caught at runtime, not "compile" time

# 2) TypeError: can only concatenate str (not "int") to str
age = input("Age: ")       # "20"
next_year = age + 1        # str + int -> TypeError
# Fix: next_year = int(age) + 1
```

Read the *last line* of the traceback first: it tells you the error type and message. The line above pinpoints exactly which line of your file misfired.

Summary

1. An algorithm is an unambiguous, finite, step-by-step procedure — the recipe behind every program.
2. Flowcharts use standardised shapes (oval, parallelogram, rectangle, diamond, hexagon) for terminal, I/O, process, decision, and loop steps.
3. Variables are labelled boxes in RAM; pick names that survive a 6-month break (`elastic_modulus`, not `e`).
4. Engineering-relevant types: `int`, `float`, `str`, `bool`, `complex`, `None`.
5. The same value occupies very different bit patterns depending on type — `5`, `5.0`, `"5"`, and `True` are not bit-equivalent.
6. Two's complement (invert + 1) is how computers store negative integers.
7. `int()`, `float()`, `str()`, `bool()` cast values; `input()` always returns a string, so cast it before doing arithmetic.

5 Operators and Expressions

In the previous week, we learned about **variables** (the nouns of programming). Now, we will learn about **operators** (the verbs). Operators allow us to manipulate variables to perform data processing, calculations, and logic making.

An **Expression** is a combination of variables and operators that yields a result. For example, in the expression `a + b`:

- `a` and `b` are **operands**.
- `+` is the **operator**.

5.1 Arithmetical operators

These are the most familiar operators, used for mathematical calculations. In Civil Engineering, these are the bread and butter of your design scripts (e.g., calculating areas, moments, stiffness matrices).

Symbol	Name	Example	Description
<code>+</code>	Addition	<code>x + y</code>	Adds two numbers
<code>-</code>	Subtraction	<code>x - y</code>	Subtracts right from left
<code>*</code>	Multiplication	<code>x * y</code>	Multiplies two numbers
<code>/</code>	Division	<code>x / y</code>	Divides (Always returns float)
<code>%</code>	Modulus	<code>x % y</code>	Returns the remainder
<code>**</code>	Exponentiation	<code>x ** y</code>	x^y (Power)
<code>//</code>	Floor Division	<code>x // y</code>	Division result truncated to integer

Table 1: Python Arithmetic Operators

Note

Syntax Detail: In some older languages or low-level languages (like C), dividing two integers (`1/2`) results in `0` (integer). In Python 3 (High-level), `1/2` results in `0.5` (float), which is more intuitive for engineers. However, strict type handling is crucial in software like SAP2000 APIs.

Example (Calculations).

We calculate momentum, and demonstrate modulus usage for batching.

```
# Arithmetic Operators
# -----
L = 5.0    # Beam Length (m)
w = 12.0   # Distributed Load (kN/m)

# Exponentiation (**): M = w * L^2 / 8
moment = (w * L**2) / 8
print("Maximum Moment:", moment, "kNm")

# Modulus (%): Useful for cyclic patterns or checking remainders
# Example: We have 52 rebars, want to bundle them in groups of 10.
```



```

total_rebars = 52
group_size = 10
remaining = total_rebars % group_size
print("Rebars left over:", remaining)

# Floor Division (/): Integer result of division
filled_groups = total_rebars // group_size
print("Full groups created:", filled_groups)

# Assignment Operators
# -----
# Instead of writing x = x + 1, we write x += 1
concrete_volume = 100 # m3
print("Initial Volume:", concrete_volume)

# Pour 20 m3 more
concrete_volume += 20
print("After Pour:", concrete_volume)

# A mixer truck takes 8 m3, remove it
concrete_volume -= 8
print("After Truck Load:", concrete_volume)

```

5.2 Assignment operators

As discussed last week, `=` is an assignment, not equality. We often want to modify a variable's existing value. Instead of writing `x = x + 5`, we can use the shorthand `x += 5`. This helps usually in performance in low-level languages (not much in Python) but mostly in readability.

- `+=` : Add and assign (`a += b` $\rightarrow$ `a = a + b`)
- `-=` : Subtract and assign
- `*=` : Multiply and assign

5.3 Comparison operators

These operators compare two values and evaluate to a **Boolean** value (**True** or **False**). This is the heart of engineering decision making. "Is *Demand* < *Capacity*?" "Is *Stress* > *YieldStrength*?"

Symbol	Name	Engineering Context
<code>==</code>	Equal	Is span length exactly 5.0m?
<code>!=</code>	Not Equal	Is material different from Steel?
<code>></code>	Greater Than	Is Load > Capacity? (Failure?)
<code><</code>	Less Than	Is Deflection < Limit? (Safe?)
<code>>=</code>	Greater/Equal	Is Factor of Safety $\geq$ 1.5?
<code><=</code>	Less/Equal	Is cost $\leq$ budget?

Note

Common Mistake: Do not confuse `=` (Assignment) with `==` (Comparison). `if x = 5` is a syntax error in Python. `if x == 5` is a question.

5.3.1 Chained comparisons

Python lets you chain inequalities just like in maths. $0 < x < 100$ is the same as $(0 < x)$ and $(x < 100)$ but reads exactly like the formula.

```
strain = 0.0025
if 0.001 < strain < 0.005:      # in the design range
    print("ok")
```

5.3.2 Identity vs Equality (is vs ==)

`==` compares values, `is` compares whether two names point to the same object in memory. Use `is` *only* with the singletons `None`, `True`, `False`.

```
x = None
if x is None:      # idiomatic
    ...
```

5.4 Operator Precedence

When multiple operators meet in one expression, Python applies them in a fixed order. The most common pitfalls: `**` is right-associative and binds tighter than unary `-`; `not` binds tighter than `and`, which binds tighter than `or`.

Group (high → low)	Operators
Exponent (right-associ.)	<code>**</code>
Unary	<code>+x</code> , <code>-x</code> , <code>~x</code>
Multiplicative	<code>*</code> <code>/</code> <code>//</code> <code>%</code>
Additive	<code>+</code> <code>-</code>
Bit shifts	<code><<</code> <code>>></code>
Bitwise AND / XOR / OR	<code>&</code> <code>^</code> <code> </code>
Comparison	<code><</code> <code><=</code> <code>></code> <code>>=</code> <code>==</code> <code>!=</code>
Membership / identity	<code>in</code> , <code>is</code>
Logical NOT / AND / OR	<code>not</code> , <code>and</code> , <code>or</code>

Right-associativity of `**`: $2 ** 3 ** 2$ means $2 ** (3 ** 2) = 2 ** 9 = 512$, *not* $(2 ** 3) ** 2 = 64$. When in doubt, use parentheses.

5.5 Short-Circuit Evaluation

Python evaluates `and/or` *left-to-right* and stops as soon as the result is determined.

```
data = None
if data is not None and len(data) > 0:  # safe: len() never runs if data is None
    ...
```

This pattern is everywhere: it lets you guard against `None` or missing keys without nested `ifs`. The order matters: writing the safety check on the right would crash.

5.6 Logical operators

In real projects, decisions rely on multiple conditions. "The design is acceptable IF (Safety Check is OK) **AND** (Cost is Low)."

- **and**: Returns True if **both** operands are true.
- **or**: Returns True if **at least one** operand is true.
- **not**: Reverses the result (True becomes False).

Example (Design Checks).



Combining stress limits and deflection limits.

```
# Comparison Operators
# -----
# Comparing Demand vs Capacity is fundamental in Civil Engineering
demand_load = 150.0 # kN
capacity = 200.0    # kN

is_safe = capacity > demand
print("Is the structure safe?", is_safe)
print("Is critical?", capacity == demand)

# Logical Operators (and, or, not)
# -----
# Complex conditions often require multiple checks.

# Example: A generic design check
# 1. Stress must be below limit
# 2. Deflection must be below limit

stress = 150 # MPa
deflection = 12 # mm

stress_limit = 250 # MPa
deflection_limit = 20 # mm

# AND: Both must be True
design_check = (stress < stress_limit) and (deflection < deflection_limit)
print("Design passes checks:", design_check)

# OR: At least one is True
# Example: Warning if ANY limit is approached (say 80% usage)
warning_needed = (stress > 0.8 * stress_limit) or (deflection > 0.8 *
    deflection_limit)
print("Warning light on:", warning_needed)

# NOT: Reverses the boolean
print("Design failed:", not design_check)
```

5.7 Bitwise operators

Bitwise operators treat numbers as binary strings (bits) rather than decimal numbers. They operate bit by bit.

- & (AND), | (OR), ^ (XOR), ~ (NOT), << (Left Shift), >> (Right Shift).

While rarely used in high-level structural analysis scripts, they are critical in:

1. **Permission/option flags:** Many tools encode multiple yes/no settings as bits in one integer (Unix file permissions, FEA selection sets).
2. **Data Compression:** Storing massive point cloud data (LiDAR).
3. **Low-level Hardware:** Reading data from sensors (Arduino/Raspberry Pi) for Structural Health Monitoring.

Permission-flag example. Imagine three boolean settings packed into one integer: READ=1, WRITE=2, EXEC=4.

```

READ, WRITE, EXEC = 1, 2, 4
perms = READ | WRITE          # 0b011 = 3 (read + write)

if perms & WRITE:              # non-zero -> WRITE is on
    print("can write")

perms = perms | EXEC          # add EXEC -> 0b111 = 7

```

Question 4

If $a = 10$ (Binary: 1010) and $b = 4$ (Binary: 0100), what is $a \& b$? *Hint: Compare bits in each position. 1 AND 0 is 0.*

Exercise 5.1 (Beam Safety Verification). **Objective:** Write a Python script to verify the safety of a simply supported steel beam under a distributed load.

Given Parameters:

- Span Length (L): 6.0 m
- Distributed Load (w): 15.0 kN/m
- Elastic Modulus (E): 210×10^6 kPa (kN/m^2)
- Moment of Inertia (I): 0.0002 m^4
- Section Modulus (Z): 0.001 m^3
- Yield Strength (F_y): 250×10^3 kPa

Calculations:

1. Maximum Moment: $M_{max} = \frac{wL^2}{8}$
2. Bending Stress: $\sigma = \frac{M_{max}}{Z}$
3. Max Deflection: $\delta_{max} = \frac{5wL^4}{384EI}$
4. Allowable Deflection: $\delta_{allow} = \frac{L}{300}$

Pass Condition: The design is SAFE if **both** conditions are met:

1. $\sigma \leq F_y$

$$2. \delta_{max} \leq \delta_{allow}$$

Task: Implement variables, arithmetic operators for formulas, comparison operators for checks, and logical operators for the final decision. Print the final boolean result.



```
# Week 3 Exercise: Beam Safety Verification

# Input Data
# -----
L = 6.0          # Beam Length (m)
w = 15.0         # Distributed Load (kN/m)
E = 210000000.0  # Elastic Modulus (kPa or kN/m^2) [Steel]
I = 0.0002       # Moment of Inertia (m^4)
Z = 0.001        # Section Modulus (m^3)
Fy = 250000.0    # Yield Strength (kPa)

# 1. Calculate Maximum Moment (kNm)
# Formula: M = w * L^2 / 8
M_max = (w * L**2) / 8
print("Maximum Moment:", M_max, "kNm")

# 2. Calculate Bending Stress (kPa)
# Formula: Stress = M / Z
sigma = M_max / Z
print("Bending Stress:", sigma, "kPa")

# 3. Calculate Maximum Deflection (m)
# Formula: delta = 5 * w * L^4 / (384 * E * I)
delta_max = (5 * w * L**4) / (384 * E * I)
print("Max Deflection:", delta_max * 1000, "mm")

# 4. Define Limits
allowable_deflection = L / 300
print("Allowable Deflection:", allowable_deflection * 1000, "mm")

# 5. Perform Safety Checks
# Check 1: Is Stress within Elastic Range? (Stress <= Yield)
check_stress = sigma <= Fy

# Check 2: Is Deflection acceptable? (Delta <= Limit)
check_deflection = delta_max <= allowable_deflection

# Final Decision: BOTH must be True for the design to be Safe
is_design_safe = check_stress and check_deflection

print("-" * 30)
print("Stress Check:", check_stress)
print("Deflection Check:", check_deflection)
print("IS DESIGN SAFE?", is_design_safe)
print("-" * 30)
```

Question 5

Predict each value:

1. $2 ** 3 ** 2 \rightarrow$ _____

2. True and False or True $\rightarrow$ _____

3. Are `5 == 5.0` and `5 is 5.0` the same? _____
4. `0b1010 & 0b1100` → _____ (give result in binary)
5. Apply De Morgan's law to rewrite `not (a > 5 and a < 10)` without the outer `not`.

Solution:-

1. `512` — `**` is right-associative, so $2^{(3^2)} = 2^9$.
2. `True` — `and` binds tighter than `or`, so it is `(True and False) or True`.
3. Different. `5 == 5.0` is `True` (equal values); `5 is 5.0` is `False` (different objects).
4. `0b1000` (decimal 8). Bitwise AND keeps only the bits set in *both*.
5. `a <= 5 or a >= 10` — distribute the negation and flip each comparison.

Common Errors

```
# 1) SyntaxError: invalid syntax
if x = 5:                                # = is assignment, not comparison
    print("five")
# Fix: if x == 5:

# 2) TypeError: unsupported operand type(s) for +: 'int' and 'str'
total = 5 + "10"
# Fix: cast first -> total = 5 + int("10")
```

Summary

1. Operators are the verbs of programming. An expression is operands+operators that produces a value.
2. Arithmetic: `/` returns float, `//` truncates, `%` is the remainder.
3. Comparison operators (`==`, `!=`, `<`, `<=`, `>`, `>=`) return `bool`; chained inequalities (`0 < x < 100`) read like maths.
4. Logical `and/or/not` short-circuit left-to-right; use this to guard against `None` or empty data.
5. Operator precedence: `**` (right-assoc) `>` unary `>` `*///` `>` `+/-` `>` comparison `>` `not` `>` `and` `>` `or`. Use parentheses when in doubt.
6. `is` compares identity, `==` compares value. Use `is` only with `None`, `True`, `False`.
7. Bitwise operators encode multiple flags into one integer; not common in design scripts, but vital when reading binary data.

6 Control Flow - Part 1 : If-Else Statements

Up until now, our programs have been linear. The computer executes line 1, then line 2, then line 3, and so on. However, real life is not linear. We make decisions based on conditions.

- "If it rains, take an umbrella."
- "If $\sigma > F_y$, the beam fails."
- "If concrete strength is C30, use $E = 33$ GPa."

Control Flow allows us to change the order of execution based on logic. The most fundamental tool for this is the **Conditional Statement**.

6.1 Conditional Statements

Conditional statements allow the program to execute a block of code **only if** a specific condition is true.

6.1.1 If-Else Statements

In Python, the syntax relies heavily on **Indentation** (whitespace). Unlike C++ or Java which use curly braces {}, Python uses the colon : and indentation to define blocks.

Structure:

```
if condition:
    # Code to run if True
    # (Indented block)
elif another_condition:
    # Code to run if first is False AND this is True
else:
    # Code to run if ALL above are False
```

Example (Concrete Strength Classification).



Classification of concrete based on compressive strength.

```
# Conditional Statements (if-elif-else)
# -----
# Example: Determines the strength class of a concrete sample
# based on its characteristic cylinder strength (f_ck).

f_ck = float(input("Enter the characteristic cylinder strength (f_ck) in MPa: "))

# Logic:
# - Below 20 MPa: Low Strength (Non-structural usage mostly)
# - 20 to 50 MPa: Normal Strength (Standard buildings)
# - 50 to 90 MPa: High Strength (High-rise, Bridges)
# - Above 90 MPa: Ultra-High Strength

if f_ck < 20:
    category = "Low Strength Concrete"
    application = "Pavements, blinding concrete"
elif f_ck < 50:
    category = "Normal Strength Concrete"
    application = "Beams, columns, slabs in typical buildings"
elif f_ck < 90:
    category = "High Strength Concrete"
```

```
        application = "High-rise buildings , long-span bridges"
    else:
        # If none of the above are true
        category = "Ultra-High Strength Concrete"
        application = "Specialized structural elements , nuclear containment"

    print("-" * 30)
    print("Classification:", category)
    print("Typical Use:", application)
    print("-" * 30)
```

6.1.2 Nested if

You can place an **if** inside another **if**. Each inner block is checked only when the outer condition is true.

```
if material == "steel":
    if stress > 250:
        print("yielded")
    else:
        print("elastic")
else:
    print("not steel")
```

Two levels of nesting are usually fine; three or more is a smell — consider rewriting with **and/or**, early **returns**, or a function.

6.1.3 The Ternary (Conditional) Expression

For short either-or assignments Python offers a one-line form:

```
status = "PASS" if stress < 250 else "FAIL"
```

This is exactly equivalent to a four-line **if/else** that assigns **status**. Use it only when both branches are short and the meaning stays obvious.

6.1.4 Truthy and Falsy Values

Anywhere Python expects a boolean (an **if** condition, a **while** loop), it actually accepts any value and converts it on the fly. The following are *falsy*:

- False, None
- 0, 0.0, 0j
- Empty containers: "", [], (), {}, set()

Everything else is *truthy*, including non-empty strings (even "False") and any non-zero number. This lets you write the idiomatic

```
if my_list:                # equivalent to: if len(my_list) > 0
    process(my_list)
```

6.1.5 Switch Statements (Match-Case)

Historically, Python did not have a **switch** statement like C or Java. Programmers used long chains of **if-elif-else** blocks. However, Python 3.10 introduced the **match-case** statement, which is a powerful structural pattern matching tool.

It is best used when checking a single variable against specific distinct values (e.g., Menu options, Material codes, Load cases).

Example (Load Factors).

Selecting Partial Safety Factors based on Load Type codes.

```
# Switch Statements (match-case)
# -----
# Introduced in Python 3.10, 'match-case' is the powerful equivalent
# of 'switch' statements in C/C++/Java.
# It is used when comparing a variable against a list of specific values.

# Example: Get the Partial Safety Factor (gamma) for different load types
# based on Eurocode / Local Standards.

load_type = input("Enter the load type (D, L, W, E): ")

gamma = 0.0
description = ""

match load_type:
    case "D":
        description = "Dead Load"
        gamma = 1.4 # or 1.35 per EC
    case "L":
        description = "Live Load"
        gamma = 1.6 # or 1.5 per EC
    case "W":
        description = "Wind Load"
        gamma = 1.0 # factor varies by combination, simplified here
    case "E":
        description = "Earthquake Load"
        gamma = 1.0
    case _:
        # The underscore (-) acts as a default/catch-all case
        description = "Unknown Load Type"
        gamma = 1.0

print("-" * 30)
print(f"Load: {description}")
print(f"Partial Safety Factor (gamma): {gamma}")
print("-" * 30)
```

Beyond simple values: match-case also supports

- **Or-patterns:** case "DL" | "LL": matches either string.
- **Guards:** case x if x > 0: adds an extra condition.
- **Capture:** case (x, y): unpacks a tuple into named variables.
- **Wildcard:** case -: matches anything (use as the default branch).

```
def category(load_type, magnitude):
    match (load_type, magnitude):
        case ("DL" | "SDL", _):
            return "dead"
        case ("LL", m) if m > 5:
            return "heavy live"
        case ("LL", _):
            return "light live"
        case _:
            return "unknown"
```


Note

Indentation Matters: In Python, indentation is not just for style; it is syntax.

```
if x > 5:
print("Big") # ERROR! IndentationError
```

You must indent (typically 4 spaces) the code inside the block.

Exercise 6.1 (Fluid Flow Regimes). **Objective:** Determine the flow regime of a fluid in a pipe based on the Reynolds Number (Re).

Theory: The Reynolds number is a dimensionless quantity used to predict flow patterns.

1. **Laminar Flow:** $Re < 2000$. Smooth, constant flow.
2. **Transitional Flow:** $2000 \leq Re < 4000$. Unstable flow.
3. **Turbulent Flow:** $Re \geq 4000$. Chaotic flow.

Task: Write a Python script that:

1. Defines a variable Re (e.g., 3500).
2. Uses `if-elif-else` logic to determine the regime.
3. Prints the result in a sentence: "For $Re = 3500$, the flow is Transitional."

Question 6

1. Rewrite as one line using a ternary:

```
if stress < 250: status = "PASS"; else: status = "FAIL"
```

 Answer: _____
2. Will the body of `if []:` execute? Why?
3. Given `x = 0`, what does `if x or "default":` evaluate to, and what is the value of `x` or `"default"`?
4. In a `match` block, what is the difference between `case 1 | 2:` and `case (1, 2):`?

Solution:-

1. `status = "PASS" if stress < 250 else "FAIL".`
2. No — `[]` is falsy (empty container), so the `if` skips the body.
3. The body executes; `x or "default"` evaluates to the *string* `"default"` because 0 is falsy and `or` short-circuits to the second operand.
4. `1 | 2` is an or-pattern: matches if the value is 1 *or* 2. `(1, 2)` is a tuple pattern: matches the specific 2-tuple `(1, 2)`.

Common Errors

```
# 1) IndentationError: expected an indented block
if x > 5:
```

```
print("big")           # missing 4-space indent
# 2) SyntaxError: 'else' without 'if'
else:                  # an 'else' clause must follow an 'if' or 'for'/'while'
    pass
```

Summary

1. `if/elif/else` runs at most one branch; indentation is part of the syntax.
2. Use chained comparisons (`a < x < b`) instead of `(a < x) and (x < b)` for readability.
3. Ternary expression `a if cond else b` is for short either-or assignments only.
4. Truthy/falsy: empty containers and zeros are falsy; everything else is truthy. `if my_list:` is idiomatic.
5. `match-case` (Python 3.10+) supports literals, or-patterns, guards, captures, and a wildcard `_`.

7 Control Flow - Part 2: Loops

In engineering, we rarely do a calculation just once. We typically want to:

- Analyze **all** beams in a floor.
- Find the root of an equation by **iterating** until error is small.
- Integrate a complex function by summing **many** small areas.

Loops allows us to repeat a block of code multiple times.

7.1 For Loops

The **for** loop is used when we know the number of iterations in advance, or when we want to iterate over a sequence (like a list of items).

Syntax:

```
for variable in sequence:
    # Code to repeat
```

7.1.1 The range() function

Ideally suited for generating sequences of numbers.

- **range(5)**: Generates 0, 1, 2, 3, 4.
- **range(2, 6)**: Generates 2, 3, 4, 5.
- **range(0, 10, 2)**: Generates 0, 2, 4, 6, 8 (Step size 2).

Example (Iterating over Lists and Ranges).



Summing point loads and calculating cantilever moments.

```
# For Loops
# -----
# Iterate over a known sequence or range.

# Example 1: Summing loads acting on a beam
point_loads = [10.5, 20.0, 5.0, 15.0] # kN
total_load = 0.0

print("Calculating Total Load...")
for load in point_loads:
    print(f"Adding load: {load} kN")
    total_load += load

print(f"Total Point Load: {total_load} kN")
print("-" * 30)

# Example 2: Iterating with an index using range()
# Calculating moment at specific intervals
# Moment M(x) = w * x^2 / 2 (Cantilever under dist. load)
w = 10.0 # kN/m
length = 5.0 # m
steps = 6 # 0, 1, 2, 3, 4, 5

print("Moment Distribution along Cantilever:")
```

```
for x in range(steps):
    moment = (w * x**2) / 2
    print(f"Distance x={x}m -> Moment={moment} kNm")
```

7.1.2 enumerate() — when you need both the index and the value

A common beginner pattern is `for i in range(len(loads)): ... loads[i]`, which has to look up the value separately. `enumerate` hands you both at once:

```
loads = [10, 25, 5, 15]
for i, load in enumerate(loads):
    print(f"Bay {i+1}: {load} kN")
```

7.1.3 zip() — iterate over two (or more) sequences in parallel

When two lists share the same indexing (e.g. beam IDs and their lengths), `zip` pairs them up:

```
ids = ["B1", "B2", "B3"]
lengths = [5.0, 6.0, 4.5]
for beam_id, L in zip(ids, lengths):
    print(f"{beam_id}: {L} m")
```

If the iterables have different lengths, `zip` stops at the shortest. Pass `strict=True` (Python 3.10+) to raise an error on mismatch.

7.1.4 Nested Loops — iterating over 2D data

You can put a loop inside another to walk over a matrix or every (row, column) pair:

```
strain = [[0.001, 0.002, 0.0015],
          [0.003, 0.004, 0.0020]]
for row in strain:
    for value in row:
        print(value, end=" ")
    print()           # newline after each row
```

7.1.5 The Loop else Clause (Python-specific)

Both `for` and `while` can have an `else` clause that runs *only if the loop completed without hitting a `break`*. Useful for "search" patterns:

```
for x in candidates:
    if is_valid(x):
        print("found", x)
        break
else:
    print("none of the candidates was valid")
```

7.2 While Loops

The `while` loop is used when we do not know how many times we need to loop. It continues **as long as** a condition remains **True**.

Syntax:

```
while condition:
    # Code to repeat
```

Note

Infinite Loops: If the condition never becomes **False**, the program will run forever (or crash). You must ensure that something inside the loop changes the condition eventually.

Example (Iterative Design).

Finding the minimum required concrete grade by iteratively increasing it until capacity meets demand.

```
# While Loops
# -----
# Repeat code WHILE a condition is true.
# Useful when we don't know how many iterations we need beforehand.

# Example: Designing a column section (Iterative Approach)
# We strictly increase the concrete grade until capacity > demand.

demand = 2500 # kN
capacity = 0 # Initial
grade = 20 # Start with C20 concrete
area = 0.4 * 0.4 # 400x400 mm column

print(f"Required Capacity: {demand} kN")

while capacity < demand:
    # Calculate capacity (Simplified: 0.85 * fck * Area)
    # We ignore steel for this simple logic example
    fck = grade * 1000 # kPa
    capacity = 0.85 * fck * area

    print(f"Trying C{grade}: Capacity = {capacity:.1f} kN")

    if capacity < demand:
        grade += 5 # Increase grade by 5 MPa (C20 -> C25 -> C30...)

print("-" * 30)
print(f"Design Selected: C{grade} with Capacity {capacity:.1f} kN")
```

7.3 Do-While Loops

Many languages (C, C++, Java) have a **do-while** loop, which guarantees that the code runs **at least once** before checking the condition. Python **does not** have a built-in **do-while** statement. However, we can emulate it:

```
while True:
    # Code runs here
    if condition_to_stop:
        break # Exit the loop manually
```

7.4 Loop Control Statements

- **break:** Terminates the loop immediately.
- **continue:** Skips the rest of the current iteration and jumps to the next one.

Exercise 7.1 (Numerical Integration). Objective: Calculate the total force acting on a beam under a distributed load defined by a function $w(x)$, using numerical integration.

Theory: The total force F is the area under the load curve $w(x)$.

$$F = \int_0^L w(x) dx$$

Computers cannot do symbolic calculus easily, but they can do very fast addition. We can approximate the integral by dividing the area into N small rectangles of width dx .

$$F \approx \sum_{i=0}^{N-1} w(x_i) \cdot dx$$

Task: 1. Define a load function for a trapezoidal load: $w(x) = 2x + 5$ (kN/m). 2. Input the beam length L and number of steps N . 3. Write a loop to sum the areas $w(x_i) \cdot dx$. 4. Compare with the exact mathematical solution.



```
# Exercise: Numerical Integration (Riemann Sum)
#
# Calculate the integral of a function f(x) from a to b.
# Engineering Context: Total force from a varying distributed load.
# Load Function: w(x) = 2*x + 5 (Trapezoidal load distribution)
# Integration Range: x = 0 to x = L

def w(x):
    """Distributed load function w(x) = 2x + 5"""
    return 2.0 * x + 5.0

# Integration Parameters
print("Calculating Total Force from Distributed Load w(x) = 2x + 5")

L = float(input("Enter beam length L (m): ")) # Integration upper limit (b)
N = int(input("Enter number of segments N: ")) # Number of steps

# Step size
dx = L / N
total_force = 0.0

# Numerical Integration using Midpoint Rule (or simple Left/Right sum)
# Integral ~ Sum( w(x_i) * dx )
print(f"integrating from 0 to {L} with {N} steps (dx={dx}) ...")

for i in range(N):
    # Calculate x at the midpoint of the current slice for better accuracy
    x_i = i * dx + (dx / 2)

    # Calculate area of this slice
    force_slice = w(x_i) * dx

    # Add to total
    total_force += force_slice

print("\n" * 30)
print(f"Total Force (Approx): {total_force:.4f} kN")

# Exact Analytic Solution: Integral(2x + 5) = x^2 + 5x
```

```
# F_exact = (L^2 + 5*L) - (0)
exact_force = (L**2) + 5*L
print(f"Total Force (Exact) : {exact_force:.4f} kN")
print(f"Error : {abs(total_force - exact_force):.6f} kN")
print("-" * 30)
```

Question 7

1. For `for i, v in enumerate(["a", "b", "c"]):`, list every (i, v) pair produced.
2. How many iterations does `for x, y in zip([1,2,3], [10,20]):` run?
3. What does the following print?

```
for i in range(5):
    if i == 3:
        break
    else:
        print("done")
```

4. Rewrite using `enumerate`: `for i in range(len(loads)): print(i, loads[i]).`

Solution:-

1. (0,'a'), (1,'b'), (2,'c').
2. 2 — `zip` stops at the shorter sequence.
3. Nothing is printed — the `break` skips the loop's `else`.
4. `for i, load in enumerate(loads): print(i, load).`

Common Errors

```
# 1) Infinite while loop -- forgot to update the condition
i = 0
while i < 5:
    print(i)
    # i += 1  <-- missing! loop never terminates

# 2) ValueError: range() arg 3 must not be zero
for x in range(0, 10, 0):
    pass
# Fix: step must be non-zero.
```

Summary

1. Use `for` when the iteration count or sequence is known in advance; use `while` when looping until a condition.
2. `range(start, stop, step)` produces an iterable of integers; `stop` is exclusive.
3. `enumerate(seq)` yields (index, value) pairs — prefer it over `range(len(seq))`.
4. `zip(a, b)` walks over two iterables in parallel and stops at the shortest.

5. **break** exits a loop early; **continue** skips to the next iteration; the loop **else** runs only when no **break** fired.
6. Numerical integration via summing rectangles is one of many engineering use-cases for **for**.

8 Data Structures and Libraries

Engineering problems typically involve huge sets of data, not just single variables.

- A bridge has hundreds of beams. We cannot create `beam1`, `beam2`, ..., `beam100`.
- A material has many properties (E, ν, ρ, f_y). We need to group them.

One of the most powerful features of Python is its ability to handle these kinds of datasets. These are called **Data Structures**. For different purposes, there are different data structures. The most common ones are lists, tuples, dictionaries, and sets. Different libraries also bring their own (NumPy uses arrays, Pandas uses dataframes). Being fluent in moving data *between* structures is an essential skill.

8.1 Lists

A **List** is an ordered, changeable collection of items. It is the most versatile data type in Python. You can store numbers, strings, or even other lists inside a list.

Syntax:

```
my_list = [item1, item2, item3]
```

8.1.1 Indexing and Slicing

We access list items using their index. Remember, Python (and most programming languages) starts counting at **0**. This stems from how arrays are laid out in memory: the first element sits at offset 0 from the array's base address.

- `loads[0]`: The first item.
- `loads[-1]`: The last item — this is the *Pythonic* way. Negative indices count from the end (-2 is second-to-last, etc.). Avoid `loads[len(loads)-1]`; it is verbose and error-prone.
- `loads[0:3]`: **Slicing**. Gets items from index 0 up to (but not including) index 3.
- `loads[::2]`: **Stride**. Every second item. `loads[::-1]` reverses the list.
- `loads[2:8:2]`: from index 2 to 8, step 2.

```
nodes = ["A", "B", "C", "D", "E", "F"]
print(nodes[1:4])      # ['B', 'C', 'D']
print(nodes[::-1])     # ['F', 'E', 'D', 'C', 'B', 'A'] (reversed)
print(nodes[::2])      # ['A', 'C', 'E'] (every 2nd)
```

Example (List Operations).



Managing structural loads using lists: Indexing, Modifying, and Calculating Totals.

```
# Python Lists
# -----
# Lists are ordered collections of items.
# Engineering Use: Storing multiple load values, beam lengths, or sensor readings.

# 1. Defining a List
# -----
# Loads in kN acting on a structure
```

```
loads = [10.5, 25.0, 5.0, 15.0, 0.0]
print(f" All Loads: {loads}")

# 2. Accessing Elements (Indexing)
# -----
# Python is 0-indexed.
first_load = loads[0]
last_load = loads[-1] # Negative index counts from the end
print(f" First Load: {first_load} kN")
print(f" Last Load: {last_load} kN")

# 3. Slicing (Subsets)
# -----
# We want to analyze only the first 3 loads
# Syntax: list[start:end] (end is exclusive)
subset = loads[0:3]
print(f" First 3 loads: {subset}")

# 4. Modifying Lists (Mutable)
# -----
# Adding a new load (e.g. from a new user input)
loads.append(30.0)
print(f" After Append: {loads}")

# Correcting a value (e.g. data correction)
loads[2] = 7.5
print(f" After Correction: {loads}")

# 5. Engineering Operation
# -----
# Calculate Total Load and Max Load
total_load = sum(loads)
max_load = max(loads)

print("-" * 30)
print(f" Total Design Load: {total_load} kN")
print(f" Critical (Max) Load: {max_load} kN")
print("-" * 30)

# 6. Converting Data Structures
# -----
# Converting a List to a Tuple (to make it immutable/read-only)
# This is useful when you want to ensure data cannot be changed accidentally
loads_tuple = tuple(loads)
print(f" Loads as Tuple: {loads_tuple}")

# Converting a Tuple back to a List (to make edits)
# If we need to modify data again, we convert it back to a list
loads_list_again = list(loads_tuple)
loads_list_again.append(50.0)
print(f" Loads back as List (with new item): {loads_list_again}")
```

8.2 Dictionaries

A **Dictionary** is a collection which is unordered, changeable, and indexed. In Python dictionaries are written with curly brackets {}, and they have **keys** and **values**.

This is perfect for Engineering Properties. Instead of remembering that "index 0 is Elastic Modulus", we can access it using `material["E"]`.

```
beam_props = {
    "length": 5.0,
    "material": "Steel",
```

```

    "I": 0.00045
}

```

Example (Material Database).



Defining material properties (Concrete, Steel) using Dictionaries and calculating member weight.

```

# Python Dictionaries
# -----
# Dictionaries store data in Key-Value pairs.
# Engineering Use: Storing properties of defined materials or sections.
# Unlike lists, data is accessed by "Name" (Key), not index.

# 1. Defining a Material Database
# -----
concrete_C30 = {
    "type": "Concrete",
    "grade": "C30",
    "fck": 30.0,          # MPa
    "E": 33000.0,        # MPa (Elastic Modulus)
    "gamma": 25.0,       # kN/m3 (Unit Weight)
    "is-ductile": False
}

steel_S420 = {
    "type": "Steel",
    "grade": "S420",
    "fy": 420.0,         # MPa
    "E": 200000.0,       # MPa
    "gamma": 78.5,       # kN/m3
    "is-ductile": True
}

# 2. Accessing Data
# -----
print(f"Analyzing {concrete_C30['type']} {concrete_C30['grade']}...")
print(f"Characteristic Strength: {concrete_C30['fck']} MPa")

# 3. Modifying Data
# -----
# Updating a property (e.g. after lab test)
concrete_C30["fck"] = 32.5
print(f"Updated fck: {concrete_C30['fck']} MPa")

# Adding a new property
concrete_C30["cost_per_m3"] = 1500.0 # TL
print("Added Cost info:", concrete_C30)

# 4. Application: Calculating Weight of a Column
# -----
# Column: 300x300 mm, Height: 3 m
b = 0.3 # m
h = 0.3 # m
L = 3.0 # m
volume = b * h * L

# Weight = Volume * Unit Weight (gamma)
weight = volume * concrete_C30["gamma"]

```

```
print("-" * 30)
print(f"Column Volume: {volume:.3f} m3")
print(f"Column Weight: {weight:.3f} kN using {concrete_C30['grade']} concrete.")
print("-" * 30)
```

8.3 List Methods at a Glance

The full reference is in the Appendix; the daily-driver subset is here.

Call	Effect	Engineering use
<code>loads.append(x)</code>	Add x at end	Add a new sensor reading
<code>loads.extend(...)</code>	Add many at end	Merge two reading batches
<code>loads.insert(i,x)</code>	Insert at position i	Inject a calibration row
<code>loads.pop()</code>	Remove and return last	FILO stack of pending tasks
<code>loads.pop(0)</code>	Remove and return first	FIFO queue
<code>loads.remove(x)</code>	Remove first match of x	Discard a known-bad value
<code>loads.sort()</code>	In-place ascending	Order by load magnitude
<code>loads.sort(reverse=True)</code>	In-place descending	Largest first
<code>loads.reverse()</code>	In-place reverse	End-to-start traversal
<code>loads.count(x)</code>	How many times x appears	Count zero-readings
<code>loads.index(x)</code>	First position of x	Find when peak occurred
<code>sorted(loads)</code>	New sorted list (original untouched)	Build a ranking without mutating

Note

`sort()` mutates the list in place and returns `None`. `sorted(loads)` is a *function* that returns a new list. Chaining `loads.sort().reverse()` fails because `sort()` returns `None`, on which `reverse()` doesn't exist.

8.4 Tuples

A **Tuple** is a collection which is ordered and **unchangeable** (immutable). We use tuples for data that should not change during the program, like coordinates of a fixed node.

```
node_01 = (0.0, 5.0, 3.5) # x, y, z coordinates
# node_01[0] = 2.0 <-- ERROR! Cannot change a tuple.
```

8.4.1 Tuple Unpacking and Multiple Return Values

Because tuples are lightweight and immutable, Python uses them as the natural mechanism for "returning more than one value":

```
def beam_endpoints(L):
    return (0.0, L)          # returning a tuple

start, end = beam_endpoints(6.0) # unpacking on assignment
print(start, end)             # 0.0 6.0
```

8.5 Sets

A **Set** is an unordered collection of *unique* items. Two superpowers:

- Removing duplicates: `set([1, 2, 2, 3]) → {1, 2, 3}`.
- Constant-time membership check (`x in s` is $O(1)$ for a set, $O(n)$ for a list).

```
beams_floor1 = {"B1", "B2", "B3"}
beams_floor2 = {"B3", "B4", "B5"}

print(beams_floor1 | beams_floor2)    # union:      {B1..B5}
print(beams_floor1 & beams_floor2)    # intersection: {B3}
print(beams_floor1 - beams_floor2)    # difference:  {B1, B2}
print("B1" in beams_floor1)           # True (O(1))
```

Engineering use: deduplicate node IDs, find which load cases are common to two analyses, validate that every "support" is also in the "node list".

8.6 List & Dict Comprehensions

A **comprehension** is a one-line way to build a list, dictionary, or set from another iterable, optionally filtering. It replaces the `for+append` pattern.

Traditional vs. comprehension — exactly the same result:

```
# Traditional
squared = []
for x in range(5):
    squared.append(x**2)

# Comprehension (one line)
squared = [x**2 for x in range(5)]
```

Adding a filter clause:

```
loads = [10, -5, 25, 0, 15, -2]
positive = [x for x in loads if x > 0]    # [10, 25, 15]

# Dict comprehension: build a stress map
sections = ["A", "B", "C"]
forces = [120, 95, 180]
stress = {s: f / 0.01 for s, f in zip(sections, forces)}
```

Comprehensions are *idiomatic Python*: they are faster, more readable once you are used to them, and the form most professional code uses. You will meet them on every page of any real Python project, so practice them now.

8.7 Mutable vs Immutable: A Deep Trap

This is one of the highest-impact concepts in Python. Some types can be modified after creation (**mutable**: list, dict, set); others cannot (**immutable**: int, float, str, tuple, frozenset).

When you write `b = a`, Python does *not* copy the data — both names point to the same underlying object. With immutable types this never bites you. With mutable types, it can:

```
a = [1, 2, 3]
b = a                # b and a point to the SAME list
b.append(99)
print(a)              # [1, 2, 3, 99]  <— a changed too!
print(a is b)         # True          <— identity test
```

If you actually want a separate copy, ask for one explicitly:

```

b = a.copy()           # shallow copy
# or:                  b = list(a)
# nested data?        from copy import deepcopy; b = deepcopy(a)

```

Note

`==` compares *values*; `is` compares *identity* (same object in memory). `[1,2] == [1,2]` is True, but `[1,2] is [1,2]` is False. Use `is` only with None, True, False.

Example (Comprehensions and Set Operations).



Filtering structural elements with comprehensions, plus set operations on node IDs.

```

# Comprehensions and Sets in Engineering Context
# -----
# Filtering structural elements, deduplicating node lists,
# and building stress maps in one line each.

# 1. List of beam dictionaries
# -----
beams = [
    {"id": "B1", "length": 5.0, "stress": 180},
    {"id": "B2", "length": 6.0, "stress": 250}, # at yield
    {"id": "B3", "length": 4.5, "stress": 120},
    {"id": "B4", "length": 7.0, "stress": 310}, # over yield
    {"id": "B5", "length": 5.5, "stress": 90},
]

YIELD_STRESS = 250 # MPa (steel)

# 2. List comprehension: extract failing IDs
# -----
failing = [b["id"] for b in beams if b["stress"] > YIELD_STRESS]
print(f"Failing beams: {failing}") # ['B4']

# 3. Dict comprehension: id -> length squared
# -----
length_sq = {b["id"]: b["length"]**2 for b in beams}
print(f"Length-squared map: {length_sq}")

# 4. Set comprehension: unique stress levels (rounded)
# -----
unique_stress = {round(b["stress"], -1) for b in beams}
print(f"Distinct stress bins: {sorted(unique_stress)}")

# 5. Set operations on node IDs from two analyses
# -----
analysis_A = {"N1", "N2", "N3", "N4"}
analysis_B = {"N3", "N4", "N5", "N6"}
print(f"Common nodes (& both): {analysis_A & analysis_B}")
print(f"Only in A (- diff): {analysis_A - analysis_B}")
print(f"All nodes (|): {sorted(analysis_A | analysis_B)}")

```

Question 8

Predict each output:

1. `[x**2 for x in range(5) if x % 2 == 0]` → _____
2. `a = [1,2,3]; b = a; b[0] = 99; print(a)` → _____
3. `"abcdef"[::-2]` → _____
4. `set([1,2,2,3]) | {3,4}` → _____
5. Write a comprehension producing a dict that maps each beam ID in `["B1", "B2", "B3"]` to its length squared, given lengths `[5, 6, 4]`.

Solution:-

1. `[0, 4, 16]` — only even `x` contribute.
2. `[99, 2, 3]` — `b` aliases `a`; mutation visible through both names.
3. `'fdb'` — start at the end (`-1`), step `-2`.
4. `{1, 2, 3, 4}` — duplicates collapse, then union.
5. `{b: L**2 for b, L in zip(["B1", "B2", "B3"], [5, 6, 4])}`.

8.8 Introduction to Libraries

Python by itself is simple. Its true power comes from the ecosystem of **Libraries** (or **Packages**). A library is a collection of pre-written functions and classes that you can **import** and use. The open-source community has developed a vast ecosystem of libraries for Python, making it a versatile language for many applications. For almost every specific purpose, there is a library for it, and they are getting much smoother to use day by day.

Note

Until this point, we didn't mention about the classes properly. The class is an important concept in programming. As can be understood from the name, it is a general classification structure (Actually, the word of structure is a bit misleading. Because "structure" is also a different concept). Classes are very helpful to organize the functions, variables, and other data structures in a logical way. In the following weeks, we will discuss about the classes a little bit more.

8.8.1 Why use libraries?

Unless you cannot find a problem that has not been solved yet, you do not need to write your own library (unless you are creating something completely new). You can think of libraries as a toolbox. In that toolbox, you can find different tools (functions, classes, etc.) for different purposes. The only thing you need to know is which tool to use for which purpose (you cannot use a hammer to drive a screw — not properly, at least).

When you get familiar with programming, there will be an instinct to write your own functions, classes, or data structures. However, this instinct should be controlled. Especially in big projects that are developed by teams, it is very important to use existing solutions. Writing code is one thing, but writing it efficiently is another. Most commonly used libraries are developed by experts in the field and optimized for performance. You should spend your time understanding the content of libraries. This is one of the most efficient ways to improve yourself as a programmer.

- **Efficiency:** Don't reinvent the wheel. Someone has already written a highly optimized matrix solver (NumPy).
- **Capabilities:** Python doesn't built-in support for plotting graphs. We need a library (Matplotlib) for that.

8.8.2 The Python Package Index (PyPI) and pip

PyPI (pypi.org) is the repository for Python software. `pip` is the package installer for Python.

```
# Installing a library via terminal
pip install numpy
pip install matplotlib
```

8.8.3 Commonly Used Engineering Libraries

The Python ecosystem is vast, containing thousands of libraries. For engineers, these libraries serve as the primary toolkit for solving complex problems ranging from numerical analysis to data visualization and machine learning. Here are some of the most essential ones:

NumPy (Numerical Python)

The cornerstone of scientific computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays efficiently.

- **Capabilities:** High-performance multidimensional array object, tools for integrating C/C++ and Fortran code, linear algebra, Fourier transform, and random number capabilities.
- **Key Functions:** `np.array()`, `np.linspace()`, `np.linalg.solve()`, `np.fft.fft()`, `np.random.rand()`.

Pandas

A fast, powerful, flexible, and easy-to-use open-source data analysis and manipulation tool. It is built on top of NumPy and is essential for handling structured data.

- **Capabilities:** Data manipulation and analysis, DataFrame object for data manipulation with integrated indexing, reading and writing data between different formats (CSV, Excel, SQL, JSON).
- **Key Functions:** `pd.read_csv()`, `pd.DataFrame()`, `df.groupby()`, `df.merge()`, `df.plot()`.

Matplotlib

The standard plotting library. It provides an object-oriented API for embedding plots into applications and creates publication-quality figures.

- **Capabilities:** Publication quality figures in a variety of hardcopy formats, comprehensive library for creating static, animated, and interactive visualizations.
- **Key Functions:** `plt.plot()`, `plt.scatter()`, `plt.bar()`, `plt.contourf()`, `plt.savefig()`.

SciPy (Scientific Python)

A collection of mathematical algorithms and convenience functions built on the NumPy extension.

- **Capabilities:** Algorithms for optimization, integration, interpolation, eigenvalue problems, algebraic equations, differential equations, and statistics.
- **Key Functions:** `optimize.minimize()`, `integrate.odeint()`, `interpolate.griddata()`, `signal.butter()`.

SymPy

A Python library for symbolic mathematics. It aims to become a full-featured Computer Algebra System (CAS).

- **Capabilities:** Symbolic mathematics, algebraic simplification, calculus (derivatives involved), solving equations, discrete math, matrices.
- **Key Functions:** `sp.symbols()`, `sp.diff()`, `sp.integrate()`, `sp.solve()`, `sp.simplify()`.

Scikit-learn

The industry standard for Machine Learning in Python. It provides simple and efficient tools for predictive data analysis.

- **Capabilities:** Classification, regression, clustering, dimensionality reduction, model selection, and preprocessing.
- **Key Functions:** `LinearRegression()`, `KMeans()`, `train_test_split()`, `metrics.mean_squared_error()`.

Seaborn

A statistical data visualization library based on Matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics.

- **Capabilities:** Dataset-oriented API for examining relationships between multiple variables, specialized support for categorical variables, visualizing univariate and bivariate distributions.
- **Key Functions:** `sns.heatmap()`, `sns.pairplot()`, `sns.boxplot()`, `sns.violinplot()`.

OpenPyXL / XlsxWriter

Libraries for reading and writing Excel 2010 xlsx/xlsm files.

- **Capabilities:** Reading and writing Excel files, programmatic data formatting, cell manipulation, and chart creation within Excel files.
- **Key Functions:** `load_workbook()`, `sheet.cell()`, `workbook.save()`.

Statsmodels

A library that complements SciPy for statistical modeling. It offers classes and functions for the estimation of many different statistical models.

- **Capabilities:** Estimation of statistical models, conducting statistical tests, statistical data exploration, linear regression, time series analysis.
- **Key Functions:** `OLS()`, `tsa.ARIMA()`, `stats.ttest_ind()`, `qqplot()`.

Plotly

A graphing library for making interactive, publication-quality graphs online.

- **Capabilities:** Interactive graphing, creating complex interactive dashboards, 3D charts, statistical graphs, maps.
- **Key Functions:** `px.scatter_3d()`, `go.Figure()`, `px.line()`, `fig.write_html()`.

NetworkX

A tool for the creation, manipulation, and study of the structure, dynamics, and functions of complex networks.

- **Capabilities:** Data structures for graphs, digraphs, and multigraphs, many standard graph algorithms, network structure and analysis measures.
- **Key Functions:** `nx.shortest_path()`, `nx.draw()`, `nx.degree_centrality()`, `nx.minimum_spanning_tree()`.

ObsPy

A comprehensive framework for processing seismological data.

- **Capabilities:** Read and write support for many file formats (MSEED, SAC, etc.), signal processing routines, easy access to data centers, seismological signal analysis.
- **Key Functions:** `read()`, `st.filter()`, `st.plot()`, `signal.invsim()`.

TensorFlow / PyTorch (Advanced)

The two dominant open-source libraries for Deep Learning.

- **Capabilities:** End-to-end open source platform for machine learning, flexible ecosystem of tools, deep learning models, neural networks.
- **Key Functions:** `tf.keras.layers.Dense()`, `torch.nn.Linear()`, `model.fit()`, `backward()`.

Common Errors

```
# 1) IndexError: list index out of range
loads = [10, 20, 30]
print(loads[5])      # only 3 items -- max valid index is 2

# 2) KeyError: 'modulus'
material = {"E": 210000}
print(material["modulus"]) # key not in dict
# Fix: material.get("modulus", default_value)

# 3) TypeError: 'tuple' object does not support item assignment
node = (0.0, 5.0)
node[0] = 1.0        # tuples are immutable
# Fix: convert to list first -> node = list(node); node[0] = 1.0
```

Summary

1. **List:** ordered, mutable, allows duplicates. Master `append`, `pop`, slicing, and stride.
2. **Tuple:** ordered, immutable. Used for fixed records and multiple return values.
3. **Dictionary:** key→value mapping. Use `d.get(k, default)` to avoid `KeyError`.
4. **Set:** unordered, unique items, $O(1)$ membership check. Perfect for deduplication and union/intersection.
5. **Comprehensions** (`[expr for x in seq if cond]`) replace the manual `for+append` pattern.
6. **Mutability matters:** `b = a` is aliasing, not copying. Use `a.copy()` or `deepcopy()` when separation is needed.
7. Negative indexing (`[-1]`) and stride slicing (`[::-1]`, `[::2]`) are idiomatic Python.
8. `pip install <pkg>` brings in third-party libraries from PyPI; never reinvent NumPy or Pandas.

9 Math, Arrays, and Dataframes

In Week 7, we will dive deeper into the core tools that make Python an Engineering Powerhouse. We will move beyond simple lists and basic arithmetic to specialized Libraries designed for heavy mathematical lifting.

9.1 Math with Python

Python has a built-in module called **math** that provides standard mathematical functions. While simple operators (+, -, *, /) are useful, they cannot calculate sine, cosine, or square roots.

9.1.1 The math Module

To use it, you must first import it:

```
import math
```

1. **Constants:** High-precision values for π (`math.pi`) and e (`math.e`).
2. **Rounding Tricks:**
 - `round(x)`: Standard rounding (to nearest integer).
 - `math.floor(x)`: Rounds **down** to the nearest integer. (e.g., $3.9 \rightarrow 3$).
 - `math.ceil(x)`: Rounds **up** to the nearest integer. (e.g., $3.1 \rightarrow 4$). Used when you need "at least" X items.
3. **Trigonometry: CRITICAL WARNING:** All computer trig functions (`sin`, `cos`, `tan`) expect the angle in **RADIANS**, not degrees.
 - To convert: `math.radians(degrees)`.
 - Example: `math.sin(math.radians(90))`.
4. **The "isclose" Trick:** Floating point numbers are not exact. $0.1 + 0.2$ is often 0.30000000000000004 . **NEVER** check if `a == b` for floats. **ALWAYS** use `math.isclose(a, b)`.

Example (Math Module Functions).



A comprehensive guide to using math functions, including the differences between floor/ceil and degrees/radians.

```
import math

# 1. Basic Constants
# -----
print(f"Pi: {math.pi}")
print(f"Euler's number (e): {math.e}")

# 2. Rounding Functions (The Tricks)
# -----
val = 3.6
print(f"\nValue: {val}")
print(f"round(3.6): {round(val)}")          # Standard rounding (nearest even for .5
    in Python 3!)
print(f"math.floor(3.6): {math.floor(val)}") # Always down
print(f"math.ceil(3.6): {math.ceil(val)}")  # Always up
```

```

print(f"math.trunc(3.6): {math.trunc(val)}") # Cuts decimal part

# Trick: Rounding to specific decimal places
# Round 3.14159 to 2 decimal places
pi_approx = round(math.pi, 2)
print(f"Round(pi, 2): {pi_approx}")

# 3. Powers and Roots
# -----
print("\nPowers and Roots:")
print(f"2^3 (pow): {math.pow(2, 3)}") # Returns float
print(f"Sqrt(16): {math.sqrt(16)}")

# 4. Logarithms
# -----
# Natural Log (ln) -> math.log(x)
# Base 10 Log -> math.log10(x)
val = 100
print(f"\nNatural Log (ln) of {val}: {math.log(val):.4f}")
print(f"Log10 of {val}: {math.log10(val)}")

# 5. Trigonometry (Crucial: RADIANS vs DEGREES)
# -----
angle_deg = 45
angle_rad = math.radians(angle_deg) # Convert deg to rad

print(f"\nAngle: {angle_deg} degrees")
print(f"Angle: {angle_rad:.4f} radians")
print(f"Sin(45 deg): {math.sin(angle_rad):.4f}") # sin expects radians!
print(f"Cos(45 deg): {math.cos(angle_rad):.4f}")

# Inverse Trig
val = 1.0
print(f"Arcsin(1.0): {math.degrees(math.asin(val))} degrees") # Result in radians,
    convert to deg

# 6. Floating Point Comparison (The 'isclose' Trick)
# -----
# NEVER check if floatA == floatB
a = 0.1 + 0.2
b = 0.3
print(f"\nIs 0.1 + 0.2 == 0.3? {a == b}") # False!
print(f"Using math.isclose: {math.isclose(a, b)}") # True

```

Note

Special Value: NaN (Not a Number)

In engineering calculations and data analysis, you will frequently encounter the value NaN. It stands for "Not a Number".

What is it? It is a special floating-point value defined by the IEEE 754 standard to represent undefined or unrepresentable values.

When do we see it?

1. High-School Math Errors:

- 0/0 (Indeterminate form).
- $\infty - \infty$.
- $\sqrt{-1}$ (Square root of a negative number in real-number domain).

2. Missing Data (Most Common):

In Civil Engineering, sensor data often has gaps. If a sensor fails to record a temperature at 12:00 PM, libraries like Pandas will fill that cell with

NaN to indicate "Missing Value".

Dangerous Properties:

- **Propagation:** Any math operation with NaN results in NaN.

$$5 + \text{NaN} = \text{NaN}$$

This is dangerous because one missing value can corrupt your entire calculation.

- **Inequality:** NaN is **not equal** to anything, including itself.

$$\text{NaN} == \text{NaN} \rightarrow \text{False}$$

You cannot check for it using `if x == NaN`. You **must** use special functions: `math.isnan(x)` or `np.isnan(x)`.

9.2 Introduction to Arrays (NumPy)

We learned about **Lists** last week. Lists are great for general data, but they are **slow** and **clumsy** for math. If you have two lists of forces **F1** and **F2**, you cannot just do $F_{total} = F1 + F2$. Python will just join the lists together, not add the numbers.

For Engineering, we use **NumPy Arrays**.

9.2.1 Why Arrays?

- **Vectorization:** You can perform math on the whole array at once (e.g., `stress = force / area`).
- **Speed:** Arrays are up to 50x faster than lists.
- **Memory:** They use less RAM.

9.2.2 Common NumPy Functions

- `np.array([...])`: Converts a list to an array.
- `np.linspace(start, stop, num)`: Generates 'num' points evenly spaced between start and stop. This is **essential** for plotting graphs (e.g., generating x-coordinates for a beam).
- `np.zeros((rows, cols))`: Creates a matrix full of zeros. useful for initializing stiffness matrices.

Example (NumPy Basics).



Demonstrating the difference between Lists and Arrays, and generating data with `linspace`.

```
import numpy as np

# 1. Why NumPy? Lists vs Arrays
# -----
# Calculating stress = Force / Area for multiple beams
forces = [1000, 2000, 1500] # List
area = 0.5                  # Scalar
```

```

# List approach (Error prone or needs loop)
# stresses = forces / area <-- TypeError: unsupported operand type(s) for /: '
#   list' and 'float'

# NumPy approach (Vectorization)
forces_arr = np.array([1000, 2000, 1500])
stresses = forces_arr / area # Element-wise operation!
print("Stresses (Array):", stresses)

# 2. Creating Arrays
# -----
# Manual
arr_1d = np.array([1, 2, 3])
arr_2d = np.array([[1, 2, 3], [4, 5, 6]]) # Matrix (2 rows, 3 cols)

print(f"\n1D Array shape: {arr_1d.shape}")
print(f"\n2D Array shape: {arr_2d.shape}")

# Generators (Very useful for graphs)
# linspace(start, stop, count) -> Generates 'count' numbers evenly spaced
x_values = np.linspace(0, 10, 5) # 0, 2.5, 5.0, 7.5, 10.0
print(f"\nLinspace (0 to 10): {x_values}")

# Zeros and Ones (Pre-allocating memory)
zeros_mat = np.zeros((3, 3)) # 3x3 matrix of zeros
print(f"\nZeros 3x3:\n{zeros_mat}")

# 3. Operations & Statistical Methods
# -----
data = np.array([10.5, 12.0, 9.8, 11.2])
print(f"\nData: {data}")
print(f"Mean: {data.mean()}")
print(f"Standard Deviation: {data.std()}")
print(f"Max Index (argmax): {data.argmax()}") # Index of the max value

```

9.2.3 Indexing, slicing, reshaping

NumPy arrays support all the slicing tricks of lists, plus 2D indexing with a comma:

```

M = np.zeros((3, 4))          # 3x4 zero matrix
M[0, 1] = 5                  # row 0, column 1
row0 = M[0, :]               # first row (1D array)
col1 = M[:, 1]               # second column
top_left = M[0:2, 0:2]       # 2x2 sub-block

flat = M.flatten()           # 1D copy
M2 = M.reshape((4, 3))       # same data, new shape (no copy)

```

9.2.4 Boolean indexing — the most useful trick

You can pass a boolean array of the same shape to filter elements:

```

forces = np.array([10, 250, 75, 320, 50, 280])
failing = forces[forces > 200] # array([250, 320, 280])
forces[forces < 0] = 0         # clip negatives to zero in place

```

This replaces a hand-written `for+if` loop and is dramatically faster on large data.

9.2.5 Broadcasting — arithmetic on different shapes

NumPy automatically stretches a smaller array along missing dimensions to match a larger one. Two simple cases:

```
# scalar broadcasts over every element:
stress = np.array([100, 200, 300]) / 0.005      # divide each by 0.005

# (3,1) + (1,4) -> (3,4) outer-style addition:
col = np.array([[1], [2], [3]])
row = np.array([[10, 20, 30, 40]])
print(col + row)      # 3x4 result
```

9.2.6 Matrix multiplication

Use `@` or `np.dot` (not `*`, which multiplies element-wise):

```
K = np.array([[2, -1], [-1, 2]])
F = np.array([10, 5])
u = np.linalg.solve(K, F)      # solves Ku = F
```

Example (NumPy Advanced).



Boolean filtering, broadcasting and a 2x2 stiffness solve.

```
# NumPy: boolean filtering, broadcasting, linear solve
# -----
import numpy as np

# 1. Boolean filtering -- find members exceeding capacity
# -----
forces = np.array([10, 250, 75, 320, 50, 280])
limit = 200
print(f"Forces over {limit} kN:", forces[forces > limit])
print(f"Indices of failures : ", np.where(forces > limit)[0])

# 2. Broadcasting -- scale and shift in one shot
# -----
# Stress in MPa given force (kN) and area (mm^2):
forces_kN = np.array([100, 150, 200])
area_mm2 = 500
stress_MPa = forces_kN * 1e3 / area_mm2      # broadcasts scalar
print("stress (MPa):", stress_MPa)

# (3,1) + (1,4) -> (3,4) outer-style addition
col = np.array([[1], [2], [3]])
row = np.array([[10, 20, 30, 40]])
print("\nbroadcast sum:\n", col + row)

# 3. Reshape and transpose
# -----
flat = np.arange(12)
M = flat.reshape((3, 4))
print("\nmatrix:\n", M)
print("transpose:\n", M.T)

# 4. 2x2 stiffness solve Ku = F
# -----
K = np.array([[2.0, -1.0],
              [-1.0, 2.0]])
F = np.array([10.0, 5.0])
u = np.linalg.solve(K, F)
print("\nu =", u)
```

```
print("verify K @ u =", K @ u)
```

9.3 Introduction to DataFrames (Pandas)

While NumPy is for **Numbers**, Pandas is for **Data**. Think of a Pandas **DataFrame** as a programmable Excel sheet.

- It has **Rows** and **Columns**.
- Columns have names (Headers).
- Methods to load data: `pd.read_csv("data.csv")`, `pd.read_excel("data.xlsx")`.

9.3.1 Power of Pandas

- **Filtering:** `df[df["Stress"] > 200]` instantly gives you all failing rows.
- **Statistics:** `df.describe()` gives count, mean, std, min, max instantly.
- **Access:** `df.iloc[0]` gives the first row. `df["Name"]` gives the Name column.

Example (Pandas Intro).



Creating a simple Material Database and filtering it based on density.

```
import pandas as pd

# 1. Creating a DataFrame (Table)
# -----
# We can create it from a Dictionary
data = {
    "Material": ["Concrete", "Steel", "Wood"],
    "Density_kg_m3": [2400, 7850, 600],
    "Elastic_Modulus_GPa": [30, 200, 11]
}

df = pd.DataFrame(data)
print("\n—— Material Table ——")
print(df)

# 2. Accessing Data
# -----
print("\n—— Column Access ——")
print(df["Density_kg_m3"])

print("\n—— Row Access (iloc) ——")
print("First Row (Concrete):")
print(df.iloc[0])

# 3. Simple Stats
# -----
print("\n—— Statistics ——")
print(df.describe())

# 4. Filtering (Conditionals)
# -----
# Find materials with Density > 1000
heavy_materials = df[df["Density_kg_m3"] > 1000]
```



```
print("\n—— Materials Heavy > 1000 kg/m3 ——")
print(heavy_materials)
```

9.3.2 loc vs iloc

`iloc[i, j]` uses integer positions (like NumPy). `loc[label, "col"]` uses the row label and column name.

```
df = pd.DataFrame({"E": [210e3, 33e3], "rho": [7850, 2400]},
                  index=["steel", "concrete"])

df.iloc[0, 0]          # 210000.0 (positional)
df.loc["steel", "E"]    # 210000.0 (label-based)
df.loc[df["rho"] > 5000] # rows where density > 5000 -> only steel
```

9.3.3 groupby and apply

`groupby` clusters rows by a column, after which any aggregate (`mean`, `max`, `count`) acts within each group.

```
# Average strength per cement type:
df.groupby("cement_type")["strength"].mean()

# Apply a custom function column-wise:
df["fck_MPa"] = df["fck_kPa"].apply(lambda x: x / 1000)
```

Example (Pandas Advanced).



`loc/iloc`, boolean filter, and `groupby` on the concrete dataset.

```
# Pandas: loc/iloc, boolean filter, groupby
# -----
import pandas as pd

# 1. Build a small dataset
# -----
data = {
    "id":      ["S1", "S2", "S3", "S4", "S5", "S6"],
    "cement_type": ["CEM I", "CEM I", "CEM II", "CEM II", "CEM III", "CEM III"],
    "cure_days":  [28, 28, 28, 7, 28, 7],
    "strength":   [42.5, 41.0, 36.0, 21.0, 50.0, 28.5], # MPa
}
df = pd.DataFrame(data).set_index("id")

# 2. iloc vs loc
# -----
print("iloc[0]      :", df.iloc[0].to_dict()) # first row positionally
print("loc['S3']     :", df.loc["S3"].to_dict()) # row labelled S3

# 3. Boolean filter
# -----
strong_28d = df[(df["cure_days"] == 28) & (df["strength"] > 35)]
print("\n28-day samples above 35 MPa:")
print(strong_28d)

# 4. Groupby aggregation
# -----
```

```

mean_by_type = df.groupby("cement_type")["strength"].mean()
print("\nMean strength per cement type:")
print(mean_by_type)

# 5. Apply a custom function column-wise
#
df["category"] = df["strength"].apply(
    lambda s: "high" if s >= 40 else ("medium" if s >= 25 else "low")
)
print("\nWith category column:")
print(df)

```

Question 9

1. `np.array([1, 2, 3]) * 2` → _____
2. For a DataFrame `df` with default integer index, when do `df.iloc[0]` and `df.loc[0]` produce the same row, and when do they differ?
3. Broadcasting: adding shape (3,1) to shape (1,4) produces what shape?

4. `math.isclose(0.1 + 0.2, 0.3)` → _____

Solution:-

1. `array([2, 4, 6])` — scalar broadcasts to every element.
2. Identical when the index is exactly 0,1,2,... They differ once you sort, filter, or assign labels — `loc` follows the label, `iloc` keeps using integer position.
3. (3, 4) — the 1's stretch to fill the other axis.
4. True — the default tolerance is roughly 10^{-9} relative.

Common Errors

```

# 1) ValueError: shapes (3,) and (4,) not aligned
np.dot(np.array([1,2,3]), np.array([1,2,3,4]))

# 2) KeyError: 'Strength'      (typo in column name)
df["Strength"]                # actual column is "strength"

# 3) IndexError: index 5 is out of bounds for axis 0 with size 3
arr = np.array([1, 2, 3])
arr[5]

```

Summary

1. Always import the `math` module before using `sin`, `cos`, `sqrt`, etc., and pass angles in radians.
2. Compare floats with `math.isclose`, never `==`.
3. NumPy arrays are typed, contiguous, and orders of magnitude faster than lists for numerical work.

4. Boolean indexing (`a[a > x]`) and broadcasting are the two biggest productivity wins from NumPy.
5. `@` / `np.dot` multiply matrices; `*` multiplies element-wise.
6. Pandas DataFrames behave like a programmable Excel sheet; use `loc` for labels, `iloc` for positions.
7. `groupby(...).mean()` replaces dozens of lines of manual aggregation.

10 Strings, Files, and Regular Expressions

Engineering is not just about numbers. It is also about **Text**.

- Structural Analysis software (SAP2000, ETABS) exports results in huge text files (.out, .log).
- Sensors visualize data in CSV (Comma Separated Values) format.
- Reports need to be generated automatically.

In Week 8, we learn how to deal with the "Text" part of programming.

10.1 String Manipulation

A string is just a sequence of characters. In Python, there is no separate `char` type — a single character is just a length-1 string. Strings are also *immutable*: every method returns a *new* string and leaves the original untouched.

10.1.1 Quote styles, escapes, and raw strings

Python accepts 'single', "double", and '''triple''' quotes interchangeably. Inside a string, special characters are written with a backslash:

Escape	Meaning	Escape	Meaning
<code>\n</code>	newline	<code>\\</code>	a literal backslash
<code>\t</code>	tab	<code>\'</code>	literal single quote
<code>\r</code>	carriage return	<code>\"</code>	literal double quote

A **raw string** prefix `r"..."` disables escapes — essential for Windows paths and regex patterns:

```
path = r"C:\Users\bt\data.csv"    # backslashes stay literal
regex = r"\d+\.\d+"               # avoids escaping every backslash
```

10.1.2 Key Methods

- **Slice:** `s[0:4]` works exactly like lists. You can grab specific parts of a text.
- **Strip:** `s.strip()` removes invisible "garbage" characters from the beginning and end of a string (like spaces and newlines `n`). This is crucial when reading files.
- **Split:** `s.split(",")` chops a string into a list of smaller strings based on a delimiter. Perfect for CSV files.
- **Replace:** `s.replace("m", "")` replaces specific characters. Useful for removing units from numbers.

10.1.3 Formatting Reports (f-strings)

Instead of manually concatenating strings (`"Load: " + str(load) + " kN"`), which is ugly and slow, use **f-strings**.

After the variable name and a colon you can specify the format. The compact reference is below; the full table is in Appendix C.

Spec	Use	Example
:.2f	2 decimal places (fixed)	<code>f"{3.14159:.2f}"</code> → "3.14"
:.3e	3-digit scientific	<code>f"{210000:.3e}"</code> → "2.100e+05"
:,.2f	thousands separator	<code>f"{1234567.89:,.2f}"</code> → "1,234,567.89"
:.1%	percentage	<code>f"{0.075:.1%}"</code> → "7.5%"
:>10	right-align, width 10	<code>f"{'A':>10}"</code> → " A"
:<10	left-align, width 10	<code>f"{'A':<10}"</code> → "A "
:^10	centred, width 10	<code>f"{'A':^10}"</code>
:05d	integer with leading zeros	<code>f"{42:05d}"</code> → "00042"
:08.3f	padded float	<code>f"{3.14:08.3f}"</code> → "0003.140"

A working report layout:

```
print(f"{'Member':<6} {'Stress [MPa]':>12} {'SF':>6}")
for mid, s, sf in [("B1", 180.5, 1.39), ("B2", 244.7, 1.02)]:
    print(f"{mid:<6} {s:>12.2f} {sf:>6.2f}")
```

Example (String Operations).



Cleaning, splitting, and formatting engineering data strings.

```
# String Manipulation for Engineers
# -----
# In engineering, we often get data as messy text strings.
# We need to clean, split, and format them.

# 1. The "Slice" (Accessing parts of a string)
# -----
sample_code = "BEAM-2024-SECTION-A"
print(f"Original: {sample_code}")
print(f"First 4 chars: {sample_code[0:4]}") # "BEAM"
print(f"Last char: {sample_code[-1]}")     # "A"

# 2. Cleaning Data (.strip)
# -----
# Essential when reading files! ' 500 \n' -> '500'
messy_input = " 500.5 \n"
clean_input = messy_input.strip()
print(f"\nMessy: '{messy_input}'")
print(f"Clean: '{clean_input}'")

# 3. Splitting Data (.split)
# -----
# Essential for CSV files or log strings
data_line = "Concrete,C30,2400,0.2"
values = data_line.split(",") # Splits where it finds ","
print(f"\nSplit Data: {values}")
# Note: They are still Strings! We need to cast them.
density = float(values[2])
print(f"Density (float): {density}")

# 4. Replacement (.replace)
```

```
# -----
# Fixing units or typos
report = "Length is 5m, Width is 2m"
numeric_report = report.replace("m", "") # Remove 'm'
print(f"\nReport: {report}")
print(f"Numeric ready: {numeric_report}")

# 5. F-String Formatting (Reporting)
# -----
load = 123.456789
d = 2.0
# We want: "Load: 123.46 kN | Depth: 2 m"
# :.2f means "Float with 2 decimal places"
formatted = f"Load: {load:.2f} kN | Depth: {d:.0f} m"
print(f"\nFormatted Report: {formatted}")
```

10.2 File Input/Output (I/O)

Variables live in RAM. When you turn off the computer, they vanish. To save them, we write them to Files (Hard Disk).

10.2.1 The with open(...) Pattern

In older languages, you had to manually open and close files. If your code crashed before closing, the file could get corrupted. Python uses the `with` statement, which acts as a safety wrapper. It closes the file automatically, even if errors occur.

Modes:

- "r" (Read): Errors if file missing. Default mode.
- "w" (Write): **Danger!** Creates a new file. If file exists, **deletes everything** in it.
- "a" (Append): Adds to the end of an existing file.

Example (Reading and Writing Files).



Writing a log file and reading it back to detect high stress events.

```
# File I/O (Input/Output) Basics
# -----
# How to read and write text files safely.

filename = "site_log.txt"

# 1. Writing to a File ('w' mode)
# -----
# 'w' mode deletes the file content if it exists and starts fresh!
# The 'with' statement automatically closes the file (SAFE).
print("Writing data to file...")
with open(filename, "w") as f:
    f.write("Time, Temperature, Stress\n") # Header
    f.write("12:00, 25.5, 100\n")          # Data
    f.write("12:15, 26.0, 110\n")
    f.write("12:30, 26.2, 115\n")
print("Write complete.\n")

# 2. Appending to a File ('a' mode)
```

```
# -----  
# 'a' adds to the end without deleting existing content.  
with open(filename, "a") as f:  
    f.write("12:45,25.8,105\n")  
  
# 3. Reading a File ('r' mode)  
# -----  
print("Reading file line by line:")  
with open(filename, "r") as f:  
    # f.readlines() reads all lines into a LIST  
    all_lines = f.readlines()  
  
for line in all_lines:  
    # line looks like "12:00,25.5,100\n"  
    # 1. Strip the invisible newline '\n' at the end  
    clean_line = line.strip()  
  
    # 2. Split by comma  
    parts = clean_line.split(",")  
  
    print(f"Row: {parts}")  
  
    # Example: Check for high stress (ignoring Header)  
    if parts[0] != "Time":  
        stress = float(parts[2])  
        if stress > 110:  
            print(f" -> WARNING: High Stress detected at {parts[0]}!")  
  
# 4. Deleting the file (Cleanup)  
# -----  
import os  
# os.remove(filename) # Uncomment to delete the file after running
```

10.3 Regular Expressions (Regex)

Sometimes, `.split()` is not enough. Imagine searching for a pile diameter in a PDF report. It could be written as:

- "D=80cm"
- "Dia: 800 mm"
- "Diameter 0.8m"

Standard string methods fail here because the format varies. **Regex** allows you to define a **Pattern** instead of a specific text.

Think of it as "Ctrl+F" on steroids.

10.3.1 Common Patterns

We use the `re` library. The minimum vocabulary you need:

Token	Matches
<code>\d</code>	a single digit (0-9)
<code>\w</code>	a letter, digit or <code>_</code>
<code>\s</code>	any whitespace (space, tab, newline)
<code>\b</code>	a word boundary (zero-width)
<code>.</code>	any single character (except newline)
<code>[abc]</code>	one of a, b, c
<code>[^abc]</code>	any character except a, b, c
<code>(...)</code>	a capture group
<code>+</code>	one or more of the previous token
<code>*</code>	zero or more
<code>?</code>	zero or one (also makes <code>+/*</code> non-greedy)
<code>{n,m}</code>	between <i>n</i> and <i>m</i> repetitions
<code>^, \$</code>	start / end of string

Three different search calls.

- `re.match(pat, s)` — only succeeds at the *start* of `s`.
- `re.search(pat, s)` — finds the *first* match anywhere in `s`.
- `re.findall(pat, s)` — returns *all* non-overlapping matches as a list.

```
import re
text = "Section M30 has fck=30 MPa; M50 has fck=50 MPa"
re.findall(r"M\d+", text)           # ['M30 ', 'M50 ']
re.findall(r"fck=(\d+)", text)      # ['30 ', '50 '] — captured groups
```

Example (Regex for Extracting Data).



extracting error codes and geometrical dimensions from mixed text using `re.search` and `re.findall`.

```
import re # Import the Regex module

# Regular Expressions (Regex)
# -----
# The "Smart Find" tool.
# Used when .split() is too complicated because the text is inconsistent.

# Scenario: Extracting error codes from messy logs
log_entry = "Error happened at [Disk-1] with Code: #404 (Timeout)"
print(f"Log: {log_entry}")

# Goal: Find the number "404"
# Naive way (splitting by space) would fail because of "#" or "("

# 1. re.search (Find the first match)
```



```

# -----
# Pattern: #\d+
# #    -> Look for a hashtag
# \d    -> Look for a digit (0-9)
# +    -> Look for ONE or MORE digits
match = re.search(r"#\d+", log_entry)

if match:
    print(f"Found code string: {match.group()}") # Returns "#404"
    # To get just the number, we can slice:
    print(f"Code number: {match.group()[1:]}")    # Returns "404"
else:
    print("No code found.")

# 2. re.findall (Find ALL matches)
# -----
# Scenario: Extracting beam dimensions from a text
# "Beam A is 300x500 mm and Beam B is 400x600 mm"
text = "Section 1: 300x500mm, Section 2: 25.5x40.0cm"

# Pattern Explanation:
# \d+    -> Digits
# \.?    -> Optional dot (for decimals like 25.5)
# \d*    -> Optional decimals after dot
# x      -> The letter 'x'
pattern = r"\d+\.\?\d*x\d+\.\?\d*"

dimensions = re.findall(pattern, text)
print(f"\nText: {text}")
print(f"Found Dimensions: {dimensions}")

# 3. Simple Patterns Guide
# -----
# \d    : Any digit (0-9)
# \w    : Any letter or number (A-Z, 0-9)
# .     : Any character (dot means 'anything')
# +     : One or more
# *     : Zero or more

```

Example (Engineering Report Formatting).



Aligned columns, engineering precision, and percentage formatting — a tiny structural-check report.

```

# Engineering report formatting with f-strings
# -----
# Aligned columns, engineering precision, percentage,
# scientific notation, and zero-padding — all in one go.

YIELD = 250.0    # MPa

members = [
    ("B1", 180.5),
    ("B2", 244.7),
    ("B3", 305.0),    # over yield
    ("B4", 92.3),
]

```

```

# 1. Header with alignment
# -----
print(f"{'ID':<5} {'Stress [MPa]':>14} {'sigma/Fy':>10} {'Verdict':>10}")
print("-" * 45)

# 2. Rows with mixed precision and alignment
# -----
for mid, sigma in members:
    ratio = sigma / YIELD
    verdict = "OK" if sigma <= YIELD else "FAIL"
    print(f"{'mid':<5} {'sigma':>14.2f} {'ratio':>10.1%} {'verdict':>10}")

# 3. Scientific notation and thousands separator
# -----
E = 210_000_000_000 # Pa (Young's modulus of steel)
print(f"\nE = {E:.3e} Pa ({E:,} Pa with thousands separator)")

# 4. Zero padding for IDs
# -----
for i in range(1, 4):
    print(f"sample_{i:03d}.csv")

```

Question 10

1. `f"{3.14159:.2f}"` → _____
2. `f"{42:>5}"` → _____ (width 5, right-aligned)
3. `"a,b,,c".split(",")` → _____
4. `re.findall(r"\d+", "M30 C25/30")` → _____
5. Difference between `r"\n"` and `"\n"`? _____

Solution:-

1. "3.14".
2. " 42" — three leading spaces to reach width 5.
3. ['a', 'b', '', 'c'] — consecutive commas leave an empty string.
4. ['30', '25', '30'].
5. `r"\n"` is two characters (backslash and n). `"\n"` is one character (a newline).

Common Errors

```

# 1) TypeError: 'str' object does not support item assignment
s = "hello"
s[0] = "H" # strings are immutable
# Fix: s = "H" + s[1:]

# 2) re.error: bad escape \d at position 0
re.findall("\d+", text) # works in Py3 but a UserWarning is emitted
# Fix: always use raw strings -> r"\d+"

```

Summary

1. Strings are immutable sequences of characters; methods return new strings.
2. Use raw strings (`r"..."`) for Windows paths and regex patterns.
3. f-strings replace string concatenation; the format mini-language controls width, alignment, precision, and type.
4. Master `strip`, `split`, `join`, `replace`, `startswith` — they cover 80% of text munging.
5. File I/O uses `with open(path, mode) as f:`; modes are `r`, `w` (truncates!), `a`.
6. Regex finds patterns instead of literal text; `re.findall` returns all matches at once.

11 Functions and Error Handling

In the previous weeks, we wrote code sequentially. Line 1, Line 2, Line 3... But as Engineering projects get complex, writing everything in one long script turns into a nightmare. Week 9 introduces the most important concept in efficient programming: **Functions**.

11.1 What is a Function?

A function is a reusable block of code that performs a specific task. You have already used them: `print()`, `len()`, `math.sqrt()`. Now, you will build your own.

11.1.1 The Factory Analogy

Imagine a **Function** as a **Machine** in a factory (e.g., a Concrete Mixer).

- **Input (Arguments)**: It takes raw materials (Water, Cement, Sand).
- **Process (Body)**: Inside the machine, it mixes them. You don't need to know the internal mechanics, you just trust it works.
- **Output (Return Value)**: It ejects the product (Fresh Concrete).

11.1.2 Why use functions?

DRY Principle: Don't Repeat Yourself. If you need to calculate the Moment of Inertia of a T-beam in 10 different places in your code, do not copy-paste the formula 10 times. Write a function `calc_inertia_T()` and call it 10 times. If you find a bug in the formula, you fix it in **one place**, and it updates everywhere.

11.2 Anatomy of a Function

To define a function in Python, we use the `def` keyword.

```
def function_name(parameter1, parameter2):  
    # Body of the function  
    result = parameter1 + parameter2  
    return result
```

11.2.1 Return vs Print

This is the #1 confusion for beginners.

- **Print**: Shows the result on the screen for the **Human**. The computer forgets it immediately.
- **Return**: Gives the result back to the **Computer**. The program can save it in a variable and use it for the next calculation.

Analogy: **Print** is like a chef shouting "Soup is ready!". **Return** is the waiter actually bringing the soup to your table.

Example (Basic Function Structure).

Inputs, Outputs, Docstrings (Documentation), and Default Arguments.

```
# Anatomy of a Function
```



```
# -----
# Analogy: A Function is like a "Concrete Mixer Truck".
# Input (Arguments): Water, Cement, Aggregate.
# Process: Mixing/Rotating.
# Output (Return): Fresh Concrete.

def calculate_beam_weight(length, width, height, density=2400):
    """
    Calculates the weight of a beam.

    Parameters:
    length (float): Length in meters.
    width (float): Width in meters.
    height (float): Height in meters.
    density (float): Density in kg/m3 (Default is Concrete: 2400).

    Returns:
    float: The total weight in kg.
    """

    # 1. Processing (The Logic)
    volume = length * width * height
    weight = volume * density

    # 2. Output (Return)
    return weight

# Using the function (Calling the machine)
# -----

# Scenario 1: Standard Concrete Beam using Positional Arguments
wt_1 = calculate_beam_weight(5.0, 0.3, 0.5)
print(f"Beam 1 Weight (Concrete): {wt_1} kg")

# Scenario 2: Steel Beam (Overriding the default density) using Keyword Arguments
# Keyword arguments make it readable and order doesn't matter much.
wt_2 = calculate_beam_weight(length=4.0, width=0.1, height=0.2, density=7850)
print(f"Beam 2 Weight (Steel): {wt_2} kg")

# Scenario 3: What if we didn't use 'return'?
def useless_calculator(a, b):
    result = a + b
    print(f"Result is {result}")
    # No return statement implies "return None"

value = useless_calculator(10, 20)
print(f"Captured Value: {value}") # Output: None. We cannot use this result in
    other calculations!
```

11.3 Variable Scope (Local vs Global)

Where a variable "lives" matters.

- **Global Scope** (The Construction Site): Variables defined in the main body. Everyone can see them.
- **Local Scope** (The Site Office): Variables defined **inside** a function. They only exist while that function is running. Once the function finishes, they are destroyed.



Example (Scope and Shadowing).

Demonstrating why a function cannot access local variables of another function, and how local variables can "shadow" global ones.

```
# Variable Scope (Local vs Global)
# -----
# Analogy:
# Global Variable: The "Site Crane". Everyone on the site can see and use it.
# Local Variable: The "Site Engineer's Personal Pen". Only the engineer in that
#                 office can use it.

# 1. Global Scope
project_name = "Suspension Bridge" # This is Global

def print_project_info():
    # We can READ global variables inside a function
    print(f"Working on: {project_name}")

print_project_info()

# 2. Local Scope
def calculate_stress(force, area):
    # 'safety_factor' is created INSIDE the function.
    # It is LOCAL. It dies when the function finishes.
    safety_factor = 1.5
    design_stress = (force / area) * safety_factor
    return design_stress

stress = calculate_stress(1000, 10)
print(f"Calculated Stress: {stress}")

# Error Scenario
# print(safety_factor)
# NameError: name 'safety_factor' is not defined.
# The main program cannot reach inside the function to get the pen!

# 3. Shadowing (Tricky!)
# -----
limit = 100 # Global

def check_limit():
    limit = 50 # Local variable with SAME name (Shadows the global one)
    print(f"Inside function, limit is: {limit}")

check_limit()
print(f"Outside function, limit is: {limit}") # Still 100! The global one was
untouched.
```

11.4 Advanced Function Arguments

Sometimes, we need more flexibility in how we call our machines.

11.4.1 Default Arguments

Some parameters can have a "fallback" value. If the user doesn't specify them, the factory uses the default.

```
def mix_concrete(cement, water, admixture="None"):
    # If user forgets admixture, we assume "None"
    ...
```

11.4.2 Flexible Arguments (*args)

What if we don't know how many inputs we will get? Imagine a function that calculates the **Total Weight** of items in a truck. The truck might have 1 item, or 50 items. We cannot define `def weight(item1, item2, item3...)`.

We use `*args`. The asterisk `*` tells Python: "Accept any number of positional arguments and pack them into a Tuple named `args`".

Analogy: A "Universal Socket Wrench". It adapts to fit however many bolts you throw at it.

Example (Integral Calculator with *args).



A smart function that adapts its formula (Linear, Quadratic, Cubic) based on the number of coefficients provided by the user using `*args`.

```
def calculate_integral(x1, x2, *coeffs):
    """
    Calculates the definite integral of a polynomial function between x1 and x2.

    The function form depends on the number of coefficients provided:
    1 arg (a)          -> f(x) = ax
    2 args (a, b)       -> f(x) = ax + b
    3 args (a, b, c)    -> f(x) = ax^2 + bx + c
    4 args (a, b, c, d) -> f(x) = ax^3 + bx^2 + cx + d
    5 args (a..e)       -> f(x) = ax^4 + bx^3 + cx^2 + dx + e

    Parameters:
    x1 (float): Lower limit of integration.
    x2 (float): Upper limit of integration.
    *coeffs: Coefficients of the polynomial (a, b, c, ...).

    Returns:
    float: The value of the definite integral.
    None: If the number of coefficients is not supported.
    """

    # Helper function to calculate F(x) (Antiderivative value at x)
    def antiderivative(x, c):
        n = len(c)
        val = 0

        # Case 1: f(x) = ax (Special case per instructions)
        if n == 1:
            a = c[0]
            # Integral of ax is (a/2)x^2
            val = (a / 2) * x**2

        # Case 2: Linear f(x) = ax + b
        elif n == 2:
            a, b = c
            val = (a / 2) * x**2 + b * x

        # Case 3: Quadratic f(x) = ax^2 + bx + c
        elif n == 3:
```

```

        a, b, c_const = c
        # Integral: (a/3)x^3 + (b/2)x^2 + cx
        val = (a / 3) * x**3 + (b / 2) * x**2 + c_const * x

    # Case 4: Cubic f(x) = ax^3 + bx^2 + cx + d
    elif n == 4:
        a, b, c_const, d = c
        val = (a / 4) * x**4 + (b / 3) * x**3 + (c_const / 2) * x**2 + d * x

    # Case 5: Quartic f(x) = ax^4 + ... + e
    elif n == 5:
        a, b, c_const, d, e = c
        val = (a / 5) * x**5 + (b / 4) * x**4 + (c_const / 3) * x**3 + (d / 2)
            * x**2 + e * x

    else:
        raise ValueError(f"Unsupported number of coefficients: {n}. Please
            provide 1 to 5 coefficients.")

    return val

try:
    # Calculate F(x2) - F(x1)
    # We pass 'coeffs' tuple to our inner helper function
    result = antiderivative(x2, coeffs) - antiderivative(x1, coeffs)
    return result

except ValueError as e:
    print(f"Error: {e}")
    return None
except TypeError:
    print("Error: Inputs must be numbers.")
    return None

# Main Execution Block
# -----
if __name__ == "__main__":
    print("—— Integral Calculator ——")

    # 1. Linear Special: f(x) = 5x, from 0 to 10
    res1 = calculate_integral(0, 10, 5)
    print(f"Integral of 5x (0 to 10): {res1}")

    # 2. Linear Standard: f(x) = 2x + 3, from 0 to 5
    # Integral = x^2 + 3x = (25+15) - 0 = 40
    res_lin = calculate_integral(0, 5, 2, 3)
    print(f"Integral of 2x + 3 (0 to 5): {res_lin}")

    # 3. Quadratic: f(x) = 1x^2 + 0x + 0, from 0 to 3
    res2 = calculate_integral(0, 3, 1, 0, 0)
    print(f"Integral of x^2 (0 to 3): {res2}")

    # 4. Error Case (Unsupported length, e.g., 6 coefficients)
    print("\n—— Testing Error Handling ——")
    calculate_integral(0, 5, 1, 2, 3, 4, 5, 6) # Should print error

```

11.4.3 Keyword Arguments (**kwargs)

args** captures *positional* arguments into a tuple. The mirror image is *kwargs** (two asterisks), which captures *keyword* arguments into a dictionary. This is how Python functions accept open-ended option sets.

```
def design_check(stress, capacity, **opts):
```



```

    sf = opts.get("safety_factor", 1.5)
    material = opts.get("material", "steel")
    print(f"{material}: SF={sf}, demand/capacity = {stress/capacity:.2f}")

design_check(180, 250, safety_factor=1.7, material="C30")
# kwargs collects {'safety_factor': 1.7, 'material': 'C30'}

```

You can combine the four argument kinds in one signature, in this strict order: `def f(positional, default=val, *args, **kwargs):`

11.4.4 The Default-Mutable-Argument Trap

This is the single most famous Python beginner trap.

```

def add_load(value, history=[]):      # DANGEROUS default
    history.append(value)
    return history

print(add_load(10))    # [10]
print(add_load(20))    # [10, 20]  <-- WHY?!

```

The default `[]` is created *once*, when the function is defined, and re-used across every call. The fix is to use `None` as the sentinel and create a fresh list inside:

```

def add_load(value, history=None):
    if history is None:
        history = []
    history.append(value)
    return history

```

11.4.5 Type Hints

Modern Python encourages annotating types. They are *not* checked at runtime by Python itself, but they help readers, IDEs, and tools like `mypy` catch bugs early.

```

def stress(force: float, area: float) -> float:
    """Return axial stress (force / area)."""
    return force / area

```

The signature now reads almost like the maths textbook. Use type hints in any function that lives longer than five minutes.

Example (kwargs in a Beam Design Check).



```

# Beam design check with **kwargs and type hints
# -----
# Demonstrates: type hints, default arguments, **kwargs,
# raising domain-specific errors, and the __main__ guard.

class StressExceedsYieldError(Exception):
    """Raised when computed stress exceeds material yield strength."""

def beam_design_check(
    force: float,
    area: float,
    yield_strength: float = 250.0,

```

```
    **opts ,
) -> dict:
    """Verify axial stress against yield , with optional safety factor .

    Extra options accepted via **opts:
        safety_factor : float , default 1.5
        material      : str ,   default "steel"
    """
    sf      = opts.get("safety_factor", 1.5)
    material = opts.get("material", "steel")

    if area <= 0:
        raise ValueError("area must be positive")

    sigma = force / area
    allowable = yield_strength / sf

    if sigma > yield_strength:
        raise StressExceedsYieldError(
            f"sigma={sigma:.1f} > F_y={yield_strength}"
        )

    return {
        "material": material ,
        "stress": sigma ,
        "allowable": allowable ,
        "passes": sigma <= allowable ,
    }

if __name__ == "__main__":
    result = beam_design_check(150, 0.0008,
                               yield_strength=275,
                               safety_factor=1.7,
                               material="C30 concrete")

    print(result)
```

11.5 Error Handling (Try / Except)

In Civil Engineering, we design systems not just for normal loads, but for failures. We assume earthquakes *will* happen. Similarly, in Programming, we must assume **Errors will happen**. Users will enter text instead of numbers, files will be missing, or internet connections will drop.

Without error handling, a Python program "crashes" (stops immediately and displays red error text) when it hits an error. This is unacceptable for professional software.

11.5.1 The Analogy: Runaway Truck Ramp

Imagine your code is a **Heavy Truck** driving down a steep highway.

- **The Main Highway (try):** This is the normal path. The truck attempts to drive here. As long as the brakes work, it stays on this road.
- **Brake Failure (The Exception):** Suddenly, a critical problem occurs (e.g., Division by Zero, File Not Found). The truck cannot continue on the highway safely.
- **The Runaway Ramp (except):** This is a dedicated gravel path designed to catch the truck if things go wrong. Instead of crashing into other cars (crashing the Operating System or losing data), the truck steers into the ramp and stops safely. The trip is interrupted, but **no catastrophe occurs**.

11.5.2 The finally Block

Sometimes, we need to clean up regardless of what happened. *Analogy:* Whether the truck finished the trip or stopped in the ramp, the **Logbook** must be signed. The **finally** block runs **always**, ensuring files are closed or connections are disconnected.

Example (Catching Errors).



Handling `ZeroDivisionError` and `TypeError` without crashing the program, using the Try-Except safety mechanism.

```
# Error Handling (Try-Except)
# -----
# Analogy: "The Safety Net".
# If a trapeze artist falls (Error occurs), the net catches them (Except block),
# so the show doesn't stop (Program doesn't crash).

def safe_division(load, area):
    try:
        stress = load / area
        return stress
    except ZeroDivisionError:
        print("CRITICAL ERROR: Area cannot be zero! Infinite stress!")
        return None
    except TypeError:
        print("ERROR: Please input numbers, not text!")
        return None

# Test 1: Normal
print(f"Test 1: {safe_division(100, 10)}")

# Test 2: Zero Area (Would normally crash the program)
print(f"Test 2: {safe_division(100, 0)}")

# Test 3: Wrong Input
print(f"Test 3: {safe_division(100, 'ten')}")

print("\nProgram continued running successfully after errors!")
```

11.5.3 Raising Errors Yourself (raise)

Catching errors is half the picture. The other half is *signalling* an error from your own code when an invariant is violated. Use **raise**:

```
def stress(force, area):
    if area <= 0:
        raise ValueError("area must be positive")
    return force / area
```

You can also define your own exception class for domain-specific errors — this is the only piece of OOP you need today:

```
class StressExceedsYieldError(Exception):
    pass

if sigma > F_y:
    raise StressExceedsYieldError(f"sigma={sigma} > F_y={F_y}")
```

A custom class lets calling code catch *exactly* this kind of failure with `except StressExceedsYieldError:`.

11.6 Reading a Traceback

When Python crashes, it prints a **traceback** — a stack of where the program was when the error happened. Beginners panic at the red wall of text. Senior engineers read it from the *bottom up*.

```
Traceback (most recent call last):
  File "analysis.py", line 12, in <module>
    result = stress(100, 0)
  File "analysis.py", line 5, in stress
    return force / area
ZeroDivisionError: division by zero
```

Reading order:

1. **Last line first:** `ZeroDivisionError: division by zero`. That tells you *what* broke.
2. Now look at the lowest **File** entry: `File "analysis.py", line 5, in stress`. That's the line where the failure actually happened.
3. Walk upward through the entries to learn *how* you got there: `stress(100, 0)` was called from line 12.

Note

The deepest frame is rarely the bug — it is often a library function doing exactly what you asked. The *first frame inside your own code* is usually where the mistake is.

Example (Anatomy of a Traceback).



A script designed to crash. Run it, read the traceback, and identify the source of the bug.

```
# Traceback demo: this script is *meant* to crash.
# -----
# Run it ('python week09-traceback-demo.py'), read the
# traceback, and identify which line is the real bug.

def axial_stress(force, area):
    # Line 8: the actual bug --- 'zero' is never defined.
    return force / area / zero

def report(member_id, force, area):
    sigma = axial_stress(force, area) # Line 13
    print(f"{member_id}: stress = {sigma:.2f} MPa")

def main():
    members = [
        ("M1", 100, 0.005),
        ("M2", 150, 0.005),
    ]
    for mid, F, A in members:
        report(mid, F, A) # Line 23

if __name__ == "__main__":
    main() # Line 27
```

11.7 The if `__name__ == "__main__":` Idiom

You will see this exact line in almost every Python script. It separates two roles:

- Code that should run when the file is *executed directly* (`python script.py`).
- Code that should *not* run when the file is *imported* as a module by another script.

```
def stress(F, A):
    return F / A

if __name__ == "__main__":
    # Quick test or demo only when run directly
    print(stress(100, 5))
```

Without the guard, importing this file from another script would unintentionally run the test code. Get into the habit early.

Question 11

1. `def f(a, b=[]): b.append(a); return b`. After `f(1)` and then `f(2)`, what does the second call return?
2. For `def calc(*args, **kwargs):`, the call `calc(1, 2, x=3)` produces what `args` and `kwargs`?
3. Given the traceback below, which line of `my_script.py` contains the actual bug?

```
Traceback (most recent call last):
  File "my_script.py", line 18, in <module>
    main()
  File "my_script.py", line 14, in main
    print(area(0))
  File "my_script.py", line 9, in area
    return side * side / zero
NameError: name 'zero' is not defined
```

4. What is the difference between `raise ValueError("bad input")` and `print("Error: bad input")`?

Solution:-

1. `[1, 2]` — the default list is shared between calls (mutable-default trap).
2. `args = (1, 2)`, `kwargs = {'x': 3}`.
3. Line 9. The line above (14) just *called* `area`; the actual undefined name appears at the deepest frame in your code (line 9).
4. `raise` stops execution and propagates the exception up the call stack until something catches it (or the program crashes with a traceback). `print` writes a string to `stdout` and execution continues normally — callers have no way to detect the "error".

Common Errors

```
# 1) TypeError: stress() missing 1 required positional argument: 'area'
def stress(force, area): return force / area
stress(100)                # forgot the second argument
```

```
# 2) UnboundLocalError: cannot access local variable 'count'
count = 0
def add():
    count = count + 1    # assignment makes count local; reading it first fails
# Fix: declare 'global count' OR pass it as an argument and return.
```

Summary

1. Functions group reusable logic; favour many small functions over one giant script (DRY principle).
2. **return** sends data back to the caller; **print** only displays it. They are not interchangeable.
3. Local variables vanish when the function returns; global variables persist but should be used sparingly.
4. ***args** packs positional arguments into a tuple, ****kwargs** packs keyword arguments into a dict.
5. Never use a mutable default (**def f(x=[]):**); use **None** as the sentinel.
6. Type hints (**def f(x: float) -> float:**) document intent and catch bugs early; they do not change runtime behaviour.
7. **try/except/else/finally** catches errors; **raise** signals them. Define custom exception classes for domain rules.
8. Read tracebacks from the bottom: error type first, then your own deepest frame.
9. Wrap demo/test code in **if __name__ == "__main__":**.

12 File Handling and CSVs

Data comes in many shapes. As engineers, we rarely type data into the code manually. Instead, we **Read** it from files. Week 10 covers the standard methods to handle external data files, with a heavy focus on **CSV** (Comma Separated Values).

12.1 Basic File Operations

We discussed `open()` briefly in Week 8. Now we will use it in more detail.

- **Text Files (.txt):** Good for simple notes or unstructured logs (mostly logs. Developers always use plain text formats. However in practice, the extension of the file defines the purpose of the file. Of course there could be exceptions but .out, .log, .txt files can be opened by any text editor.).
- **Binary Files (.bin, .jpg):** Images or compiled programs. We won't touch these yet.
- **Structured Text (.csv, .json, .xml):** The standard for data exchange (the most easy and proper way to handle data or datasets).

12.2 The CSV Format

CSV is the universal language of Engineering Data. It is just a text file where columns are separated by commas.

```
Time, Force, Displacement
10.0, 500, 2.5
10.1, 510, 2.6
```

Note

In CSV files, columns are usually separated by commas. However, this is not a strict rule. You can easily see that semicolons are also used as separators. To open a CSV file with semicolons or any other separator, you can use the following parameters:

```
with open('data.csv', 'r', newline='') as file:
    reader = csv.reader(file, delimiter=';')
```

12.2.1 Why not just use Excel (.xlsx)?

Excel files are complex, binary "zip" archives full of formatting, fonts, and heavy metadata. CSV is **plain text**. It is lightweight, fast, and can be opened by **any** software (Python, MATLAB, AutoCAD, Excel, Notepad). The advantages of Excel is that graphing and formatting is easier. However, when you get familiar with Python, you will find it much easier to handle data with Python. In addition to that, you'll be able to generate graphs that all in a standard format which is so important for data presentation.

12.3 Reading CSV Files

Python provides a built-in library called `csv`.

12.3.1 1. The `csv.reader`

Reads the file row by row. Each row is returned as a ****List of Strings****.

1. You must open the file first.
2. You pass the file object to `csv.reader()`.
3. You iterate through it with a `for` loop.

Example (CSV Basics).



Demonstrating Manual Parsing vs. the `csv` Library.

```
import csv

# WEEK 10: CSV File Operations
# -----
# "CSV" stands for Comma Separated Values.
# It is the most common format for Engineering Data exchange.

filename = "code/week10-concrete-data.csv"

# 1. Reading CSV Manually (The Hard Way)
# -----
# Useful to understand what CSV actually is: Just text with commas!
print("—— Method 1: Manual Parsing ——")
with open(filename, "r") as f:
    # Read first 3 lines only
    for i in range(3):
        line = f.readline().strip()
        columns = line.split(",")
        print(f"Row {i}: {columns}")

# 2. Reading with 'csv' Library (The Standard Way)
# -----
# Handles edge cases (like commas inside text) automatically.
print("\n—— Method 2: csv.reader ——")
with open(filename, "r") as f:
    reader = csv.reader(f)

    # Getting the Header
    header = next(reader) # Skips the first line and saves it
    print(f"Header: {header}")

    # Read data
    row_count = 0
    for row in reader:
        row_count += 1
        # Let's print only the first 2 data rows
        if row_count <= 2:
            print(f>Data {row_count}: {row}")

# 3. Writing to CSV
# -----
output_file = "code/week10-report.csv"
data_to_write = [
    ["Test.ID", "Result", "Notes"],
    [1, "Pass", "Good surface"],
    [2, "Fail", "Cracked early"]
]
```



```
print(f"\n—— Writing to {output_file} ——")
with open(output_file, "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(data_to_write)
print("Write complete.")
```

12.3.2 2. The csv.DictReader (Pro Move)

Instead of remembering that "Strength" is Column 8 (e.g., `row[8]`), `DictReader` lets you use column names (e.g., `row["Strength"]`). This makes your code **readable** and **resistant to column changes**.

12.4 Application: Statistical Analysis

Let's analyze a real dataset of 300 Concrete Cylinder tests. **Scenario:** You are the Quality Control Engineer. You have a messy raw data file. You need to:

- Filter out the tests that are NOT 28-day cure time.
- Calculate the Mean and Standard Deviation of the strength.
- Decide if the batch Passes or Fails.

Example (Statistics on CSV Data).



A complete script to read, filter, parse, and statistically analyze engineering data.

```
import csv
import statistics

# WEEK 10: Parsing & Statistics Application
# -----
# Goal: Read 300 concrete tests, filter by 28-day strength,
# and calculate statistics.

filename = "code/week10-concrete-data.csv"

# Lists to hold our data columns
strengths_28_days = []
batch_codes = []

print(f"Reading {filename}...")

with open(filename, "r") as f:
    reader = csv.DictReader(f) # DictReader map header to values!
    # row is now like: {'Sample_ID': 'CYL-1001', 'Compressive_Strength_MPa':
    #                 '35.2', ...}

    for row in reader:
        try:
            # Parse Data
            days = int(row["Cure_Time_Days"])
            strength = float(row["Compressive_Strength_MPa"])
            batch = row["Batch_Code"]

            # Application Logic: We only care about standard 28-day tests
            if days == 28:
                strengths_28_days.append(strength)
```

```

        batch_codes.append(batch)

    except ValueError:
        print("Skipping invalid row...")

# Statistical Analysis
# -----
count = len(strengths_28_days)
avg = statistics.mean(strengths_28_days)
std_dev = statistics.stdev(strengths_28_days)
min_val = min(strengths_28_days)
max_val = max(strengths_28_days)

# Reporting
print("\n—— 28-Day Concrete Strength Analysis ——")
print(f"Total Samples Analyzed: {count}")
print(f"Mean Strength:          {avg:.2f} MPa")
print(f"Standard Dev:            {std_dev:.2f} MPa")
print(f"Range:                   {min_val} – {max_val} MPa")

# Acceptance Check (Example C30/37 Concrete)
# A simplified rule: Mean must be >= 38 MPa (fck + margin)
target = 38.0
if avg >= target:
    print(f"\nRESULT: BATCH PASSED (Mean > {target})")
else:
    print(f"\nRESULT: BATCH FAILED (Mean < {target})")

```

12.5 Pandas vs the csv module

For one-off scripts and embedded systems the built-in `csv` module is great. For real analysis, Pandas is shorter and faster:

```

import pandas as pd
df = pd.read_csv("data.csv")
print(df.describe())           # statistics in one line
strong = df[df["strength"] > 35] # filter rows

```

Use `pd.read_csv("...", delimiter=";")` when the file uses semicolons; `encoding="utf-8"` when it has Turkish characters.

12.6 File Paths the Right Way: pathlib

Hard-coding strings like `"C:\data\file.csv"` bites you the first time the code runs on Linux/Mac. Python's `pathlib` builds platform-agnostic paths:

```

from pathlib import Path

base = Path("data")
csv_path = base / "concrete" / "samples.csv"  # OS-correct separator
print(csv_path.exists())
print(csv_path.suffix)           # '.csv'
print(csv_path.with_suffix(".json")) # data/concrete/samples.json

```

Use `Path` objects everywhere: `open(csv_path)` and `pd.read_csv(csv_path)` both accept them.

12.7 JSON — the API/Web Data Format

JSON ("JavaScript Object Notation") is plain text that maps directly to Python dicts and lists. Most modern web APIs return JSON; sensors and structural-monitoring services often log it as well.

```

import json

# 1. Read JSON file -> Python dict
with open("readings.json") as f:
    data = json.load(f)

# 2. Or parse a string directly
record = json.loads('{ "id": "B1", "stress": 180.5 }')

# 3. Write a Python object out as JSON
with open("out.json", "w") as f:
    json.dump({"members": [record]}, f, indent=2)

```

Example (JSON sensor log).



Read a small JSON sensor log, filter high-deflection events, and write the result back.

```

# JSON sensor log: read, filter, write back
# -----
# Demonstrates json.load, list-comprehension filter,
# and json.dump with pretty-printing.

import json
from pathlib import Path

# 1. Sample data -- in real life this would be an existing file.
# -----
sample = {
    "site": "Bridge-A",
    "readings": [
        {"t": "08:00", "deflection_mm": 1.2, "temp_C": 18},
        {"t": "12:00", "deflection_mm": 4.8, "temp_C": 28},
        {"t": "16:00", "deflection_mm": 6.7, "temp_C": 31},
        {"t": "20:00", "deflection_mm": 2.1, "temp_C": 22},
    ],
}

src = Path("readings.json")
src.write_text(json.dumps(sample, indent=2))

# 2. Read JSON -> Python dict
# -----
with open(src) as f:
    log = json.load(f)

# 3. Filter high-deflection events
# -----
THRESHOLD_MM = 4.0
high_events = [r for r in log["readings"]
               if r["deflection_mm"] > THRESHOLD_MM]

print(f"{len(high_events)} reading(s) above {THRESHOLD_MM} mm:")
for ev in high_events:
    print(f"    {ev['t']} -> {ev['deflection_mm']} mm")

# 4. Write filtered subset back as JSON
# -----
out = Path("alarms.json")
with open(out, "w") as f:

```

```
json.dump({"site": log["site"], "alarms": high_events}, f, indent=2)
print(f"Wrote {out.resolve()}")
```

12.8 Excel Files (.xlsx)

For Excel you have two main options. Pandas reads sheets directly:

```
df = pd.read_excel("project.xlsx", sheet_name="Loads")
df.to_excel("results.xlsx", index=False)
```

Cell-level control (formatting, formulas, charts) requires `openpyxl`:

```
from openpyxl import load_workbook
wb = load_workbook("project.xlsx")
ws = wb["Loads"]
ws.cell(row=2, column=3).value = 50.0
wb.save("project.xlsx")
```

Question 12

1. Which mode opens a JSON file just to read it? _____
2. What does `Path("a") / "b" / "c.csv"` produce on Windows? On Linux?
3. `pd.read_csv("data.csv", delimiter=";")` differs from the default call how?
4. `json.dumps({1: "x"})` succeeds; `json.dumps({(1,2): "x"})` fails. Why?

Solution:-

1. "r" (read) — the default for `open`, so you may even omit it.
2. Windows: `a\b\c.csv`. Linux/Mac: `a/b/c.csv`. `pathlib` picks the correct separator automatically.
3. It splits each line on `;` instead of `,`. Common for European CSV exports.
4. JSON keys must be strings. Python int keys are auto-stringified, but tuple keys cannot be represented in JSON at all.

Common Errors

```
# 1) FileNotFoundError: [Errno 2] No such file or directory: 'data.csv'
# Cause: working directory != folder containing the file.
# Fix: print(Path.cwd()) and either cd or use an absolute Path.

# 2) json.JSONDecodeError: Expecting value: line 1 column 1
# Cause: the file is not valid JSON (often has a BOM or stray text).

# 3) UnicodeDecodeError: 'charmap' codec can't decode byte 0xE7
# Cause: file is UTF-8 (with Turkish characters) but Windows opened it as cp1252.
# Fix: open(path, encoding="utf-8")
```

Summary

1. with `open(path, mode)` as `f`: is the safe pattern; the file is auto-closed.

2. CSV is plain-text and universal; the `csv` module gives row-by-row access, Pandas gives whole-table operations.
3. `pathlib` replaces hand-built strings; `Path("a")/"b"` is OS-aware.
4. JSON maps directly to Python dicts/lists; use `json.load/dump` for files, `loads/dumps` for strings.
5. Excel via Pandas (whole sheets) or `openpyxl` (cell-level edits).
6. Always pass `encoding="utf-8"` on Windows when the file may contain non-ASCII characters.

13 Data Visualization

"A picture is worth a thousand numbers." In Engineering, we deal with massive datasets. Looking at a spreadsheet with 10,000 rows tells you nothing. Plotting that data reveals:

- **Trends:** Is the bridge moving over time?
- **Outliers:** Is one sensor giving garbage data?
- **Correlations:** Does adding more cement actually increase strength?

In Week 11, we will learn how to turn data into insight using Python.

13.1 Libraries

- **Matplotlib (plt):** The grandfather of Python plotting. Powerful but requires a lot of code for complex plots.
- **Seaborn (sns):** Built on top of Matplotlib. Designed for **Statistical Data Visualization**. It makes beautiful plots in one line of code.

13.2 Two Plotting Styles: Pyplot vs Object-Oriented

Matplotlib offers two interchangeable APIs.

Pyplot style (quick scripts):

```
import matplotlib.pyplot as plt
plt.plot(x, y)
plt.title("Force-Displacement")
plt.xlabel("u [mm]"); plt.ylabel("F [kN]")
plt.show()
```

Object-oriented style (recommended for anything more complex):

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(x, y, label="Specimen 1")
ax.set_title("Force-Displacement")
ax.set_xlabel("u [mm]"); ax.set_ylabel("F [kN]")
ax.legend(); ax.grid(True)
fig.savefig("force_disp.png", dpi=200)
```

The OO style scales to multi-panel figures (`fig, axes = plt.subplots(2, 2)`) without hidden global state.

13.3 Customisation Cheatsheet

Goal	Call
Title and axis labels	<code>ax.set_title, set_xlabel, set_ylabel</code>
Axis range	<code>ax.set_xlim(a, b), set_ylim</code>
Logarithmic axis	<code>ax.set_xscale("log"), set_yscale("log")</code>
Grid	<code>ax.grid(True, which="both")</code>
Legend	<code>ax.legend(loc="best")</code> (curves need <code>label=</code>)
Twin Y axis	<code>ax2 = ax.twinx()</code>
Save to file	<code>fig.savefig("fig.png", dpi=200)</code>

Style shorthand for plot. The third positional argument combines colour, line, and marker:

- Colour: `r`, `g`, `b`, `k` (black), `m`, `c`, `y`.
- Line: `-`, `--`, `:`, `-..`.
- Marker: `o`, `s`, `*`, `x`, `+`, `^`.

Example: `ax.plot(x, y, "r--o")` = red, dashed, circle markers.

13.4 Distributions (Histograms)

The first thing you should do with any new dataset is to check the **Distribution**. Is your data **Normal** (Bell curve)? Or is it **skewed**?

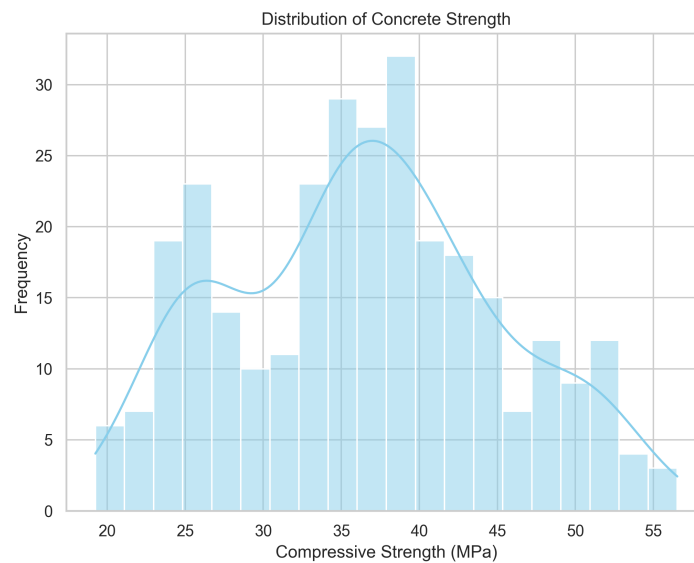


Figure 1: Distribution of Concrete Strength. Notice the Bell Curve shape.

13.5 Correlations (Scatter Plots)

To see if two variables are related, we use Scatter Plots. Below, we check if heavier samples carry more load. The color shows the Cure Time.

13.6 Comparing Groups (Box Plots)

How does Cement Type affect strength? A Box Plot shows the Median, Quartiles, and Outliers.

13.7 The Engineering Dashboard (Subplots)

Professional reports often combine multiple plots into one figure using `plt.subplots()`.

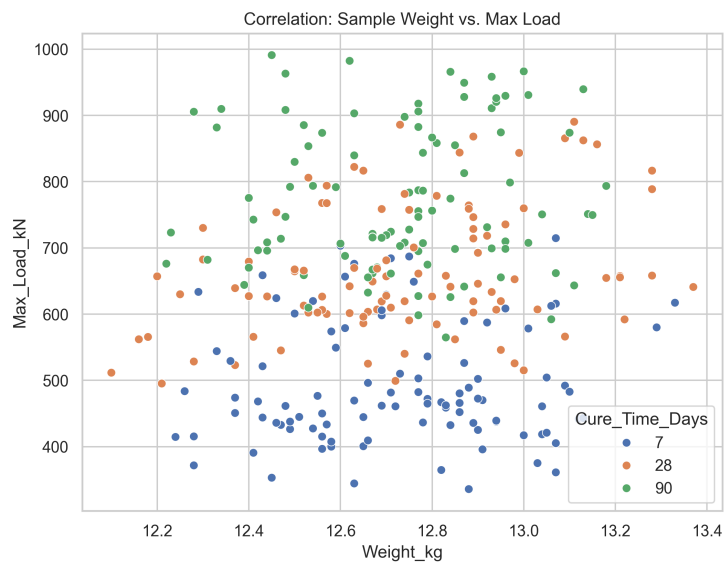


Figure 2: Scatter Plot: Weight vs Load. There is a clear positive correlation.

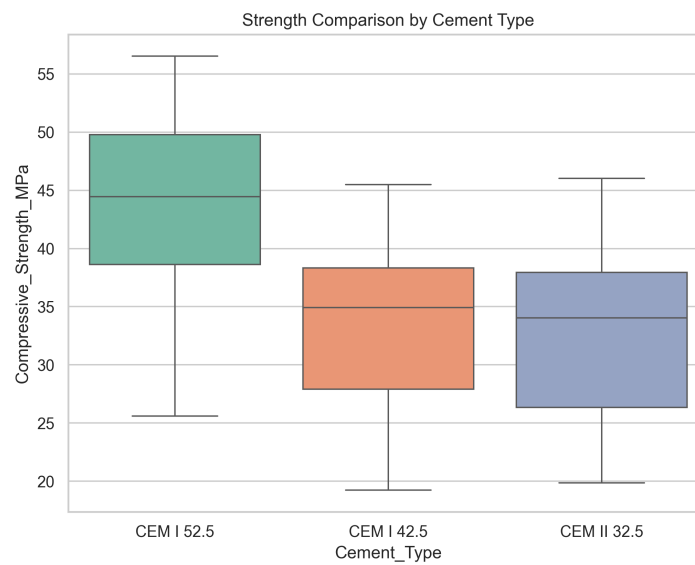


Figure 3: Box Plot: Strength by Cement Type. CEM I 52.5 is clearly stronger.

Example (Generating Plots).

The Python code used to generate all the figures above.

```
import pandas as pd
```



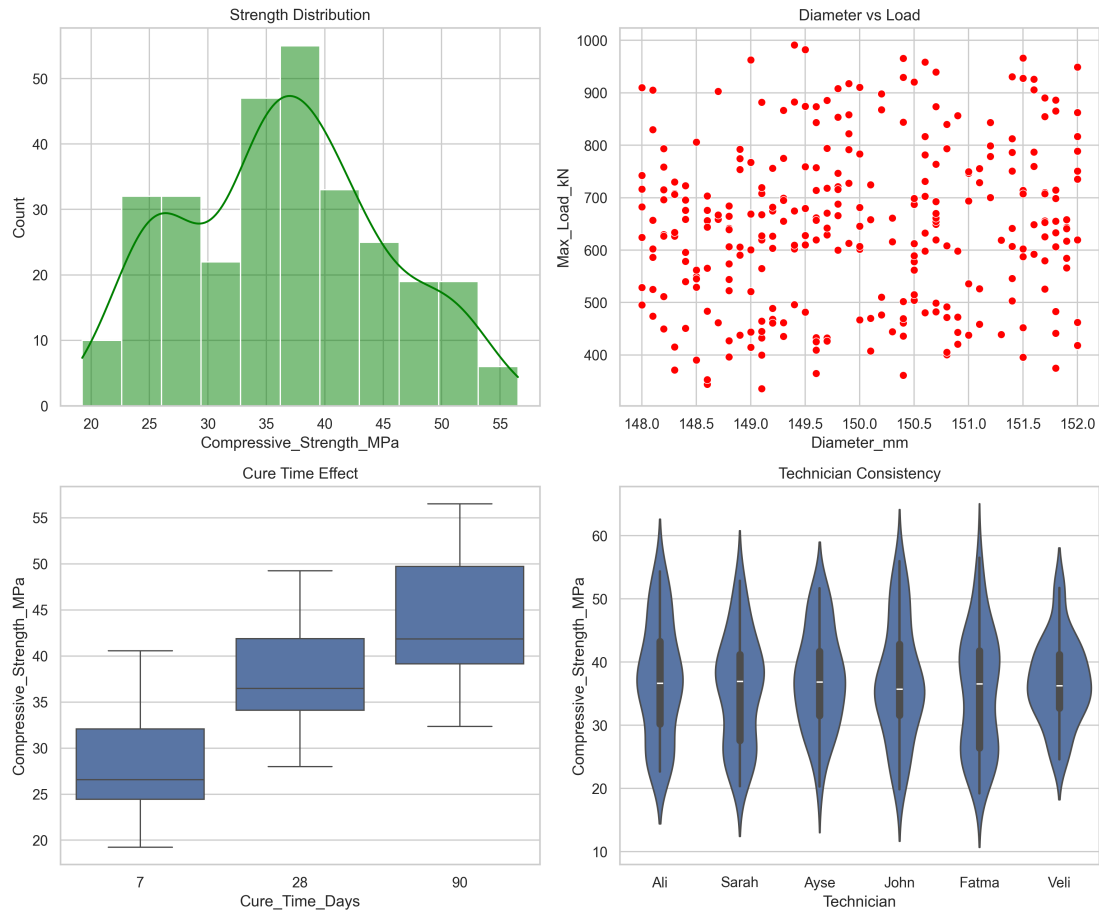


Figure 4: A complete Engineering Analysis Dashboard.

```
import matplotlib.pyplot as plt
import seaborn as sns
import os

# Ensure the output directory exists
output_dir = "../En-en/fig"
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

# 1. Load Data
# -----
df = pd.read_csv("week10-concrete_data.csv")

# Set a nice theme
sns.set_theme(style="whitegrid")

# 2. Histogram (Distribution of Strength)
# -----
plt.figure(figsize=(8, 6))
# Histogram with a Kernel Density Estimate (KDE) line
sns.histplot(data=df, x="Compressive_Strength_MPa", kde=True, bins=20, color="skyblue")
```

```

plt.title("Distribution of Concrete Strength")
plt.xlabel("Compressive Strength (MPa)")
plt.ylabel("Frequency")
plt.savefig(f"{output_dir}/week11_hist.png", dpi=300)
print("Saved: week11_hist.png")

# 3. Scatter Plot (Correlation: Weight vs Load)
# -----
plt.figure(figsize=(8, 6))
# Does heavier concrete carry more load?
sns.scatterplot(data=df, x="Weight_kg", y="Max_Load_kN", hue="Cure_Time_Days",
                palette="deep")
plt.title("Correlation: Sample Weight vs. Max Load")
plt.savefig(f"{output_dir}/week11_scatter.png", dpi=300)
print("Saved: week11_scatter.png")

# 4. Box Plot (Comparison by Category)
# -----
plt.figure(figsize=(8, 6))
# Compare strength across different cement types
sns.boxplot(data=df, x="Cement_Type", y="Compressive_Strength_MPa", palette="Set2")
plt.title("Strength Comparison by Cement Type")
plt.savefig(f"{output_dir}/week11_boxplot.png", dpi=300)
print("Saved: week11_boxplot.png")

# 5. Bar Plot (Average Strength by Cure Time)
# -----
plt.figure(figsize=(8, 6))
sns.barplot(data=df, x="Cure_Time_Days", y="Compressive_Strength_MPa", errorbar="sd",
            capsize=1)
plt.title("Strength Gain over Time (Mean + Std Dev)")
plt.savefig(f"{output_dir}/week11_barplot.png", dpi=300)
print("Saved: week11_barplot.png")

# 6. Subplots (Engineering Dashboard)
# -----
fig, axs = plt.subplots(2, 2, figsize=(12, 10))

# Plot A: Hist
sns.histplot(data=df, x="Compressive_Strength_MPa", kde=True, ax=axs[0, 0], color="green")
axs[0, 0].set_title("Strength Distribution")

# Plot B: Scatter
sns.scatterplot(data=df, x="Diameter_mm", y="Max_Load_kN", ax=axs[0, 1], color="red")
axs[0, 1].set_title("Diameter vs Load")

# Plot C: Box
sns.boxplot(data=df, x="Cure_Time_Days", y="Compressive_Strength_MPa", ax=axs[1, 0])
axs[1, 0].set_title("Cure Time Effect")

# Plot D: Violin (Advanced Boxplot)
sns.violinplot(data=df, x="Technician", y="Compressive_Strength_MPa", ax=axs[1, 1])
axs[1, 1].set_title("Technician Consistency")

plt.tight_layout()
plt.savefig(f"{output_dir}/week11_dashboard.png", dpi=300)
print("Saved: week11_dashboard.png")

```

13.8 Two Civil-Engineering Classics

13.8.1 Force–Displacement Curve (Pushover-Style)

Example (Force–Displacement).



A bilinear capacity curve plotted with the OO API, with axis labels, grid, legend, and saved to PNG.

```
# Bilinear force-displacement (pushover-style) capacity curve
# -----
# Demonstrates: object-oriented Matplotlib API, axis labels,
# grid, legend, annotation, and saving to PNG.

import numpy as np
import matplotlib.pyplot as plt

# 1. Synthetic capacity curve: elastic up to yield, then plastic plateau
# -----
u_yield = 12.0      # mm
F_yield = 220.0     # kN
u_max = 60.0
F_max = 250.0      # mild hardening

u_elastic = np.linspace(0, u_yield, 30)
F_elastic = (F_yield / u_yield) * u_elastic

u_plastic = np.linspace(u_yield, u_max, 60)
F_plastic = F_yield + (F_max - F_yield) * (u_plastic - u_yield) / (u_max - u_yield)

u = np.concatenate([u_elastic, u_plastic])
F = np.concatenate([F_elastic, F_plastic])

# 2. Plot with the OO API
# -----
fig, ax = plt.subplots(figsize=(6.5, 4.5))
ax.plot(u, F, "b-", linewidth=2, label="Capacity curve")
ax.axvline(u_yield, color="grey", linestyle="—", alpha=0.6)
ax.scatter([u_yield], [F_yield], color="red", zorder=5, label="Yield point")

ax.set_title("Force—Displacement (Pushover)")
ax.set_xlabel("Displacement u [mm]")
ax.set_ylabel("Base shear F [kN]")
ax.set_xlim(0, u_max)
ax.set_ylim(0, F_max * 1.1)
ax.grid(True, alpha=0.4)
ax.legend(loc="lower right")

fig.tight_layout()
fig.savefig("force_displacement.png", dpi=200)
plt.show()
```

13.8.2 Twin-Axis Plot (Force and Displacement vs Time)

**Example (OO Plotting with twinx).**

```
# Twin-axis plot: force and displacement vs time
# -----
# Two physical quantities with different units share the
# x-axis but get their own y-axis (one on each side).

import numpy as np
import matplotlib.pyplot as plt

# 1. Synthetic time history (e.g. cyclic loading)
# -----
t = np.linspace(0, 4 * np.pi, 400)
F = 100 * np.sin(t)          # kN
u = 3 * np.sin(t - 0.3)     # mm (slight phase lag)

# 2. OO twin-axis figure
# -----
fig, ax1 = plt.subplots(figsize=(7, 4))

color1 = "tab:blue"
ax1.plot(t, F, color=color1, label="Force F [kN]")
ax1.set_xlabel("time [s]")
ax1.set_ylabel("F [kN]", color=color1)
ax1.tick_params(axis="y", labelcolor=color1)
ax1.grid(True, alpha=0.3)

ax2 = ax1.twinx()           # second y-axis sharing the x-axis
color2 = "tab:red"
ax2.plot(t, u, color=color2, linestyle="--", label="Displ. u [mm]")
ax2.set_ylabel("u [mm]", color=color2)
ax2.tick_params(axis="y", labelcolor=color2)

# Combine legends from both axes
lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc="upper right")

fig.suptitle("Cyclic loading: force and displacement vs time")
fig.tight_layout()
fig.savefig("twin_axis.png", dpi=200)
plt.show()
```

Question 13

1. What does `ax.plot(x, y, "r--o")` draw, in words?
2. For `fig, ax = plt.subplots(2, 1)`, what is the shape and how do you reach the lower panel?
3. How do you switch the y-axis to a logarithmic scale on an existing `ax`?

4. Why is `plt.show()` usually needed at the end of a script run from the terminal?

Solution:-

1. A red, dashed line with circle markers connecting the data points.
2. `axes` is an array of length 2; bottom panel is `axes[1]`.
3. `ax.set_yscale("log")`.
4. Without `show()` the script ends and the figure window closes immediately. Inside Jupyter it is implicit.

**Common
Errors**

```
# 1) Plot window never appears
# Cause: plt.show() missing -- script exits and closes the figure.

# 2) UserWarning: This figure includes Axes ... but tight_layout cannot make
# Often because labels/titles overflow. Fix: fig.set_constrained_layout(True)
# or call fig.tight_layout() *after* labels are set.

# 3) Legend missing curves
# Cause: forgot label= in plot() -> ax.plot(x, y, label="curve1")
```

Summary

1. Always check the distribution first; histograms catch outliers and skew before any analysis.
2. Scatter plots reveal correlations; box plots compare groups; subplots assemble a dashboard.
3. Prefer the object-oriented API (`fig, ax = plt.subplots()`) for anything beyond a single quick plot.
4. The colour-line-marker shorthand ("`r--o`") is fast for exploratory work; named keyword arguments are clearer for final figures.
5. Save with `fig.savefig(name, dpi=200)` for crisp report-quality images.

14 Importing Data from Online Sources

Up to now we have read data from CSV/JSON files that already lived on disk. In real engineering work, the data is usually *remote*: a government API, a sensor cloud, a research dataset hosted online. Week 12 introduces the standard way to fetch that data.

14.1 HTTP in 60 seconds

Web data exchange is built on **HTTP**. You send a **request** (most often a **GET**) to a URL; the server responds with:

- A numeric **status code** (200 = OK, 404 = not found, 500 = server error, 429 = too many requests).
- A **body**, very often JSON-encoded.

14.2 The requests library

requests is the de-facto standard for HTTP in Python. Install it once with `pip install requests`.

```
import requests
url = "https://api.example.com/data"
resp = requests.get(url, timeout=10)
print(resp.status_code)          # 200, 404, ...
data = resp.json()               # parses the JSON body into a Python dict/list
```

Always pass `timeout=`: without it, a slow or dead server will hang your script forever.

14.3 From JSON to Pandas

APIs typically return a list of records (a list of dicts). Pandas turns that into a DataFrame in one line:

```
import pandas as pd
records = resp.json()["features"]  # depends on the API's schema
df = pd.DataFrame(records)
print(df.head())
```

14.4 Robust API Calls

Real-world HTTP is unreliable: connections drop, servers return 500, networks time out. Wrap your call in a try/except:

```
import requests
try:
    resp = requests.get(url, timeout=10)
    resp.raise_for_status()          # raises HTTPError for 4xx/5xx
    data = resp.json()
except requests.exceptions.Timeout:
    print("server too slow")
except requests.exceptions.ConnectionError:
    print("network unreachable")
except requests.exceptions.HTTPError as e:
    print("server replied with error:", e.response.status_code)
```

14.5 Application: USGS Earthquake Feed

The U.S. Geological Survey publishes open data on every recorded earthquake. The "all earthquakes, past day" feed lives at:

`https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/all_day.geojson`

The response is GeoJSON — a JSON dialect with a top-level **features** list. Each feature contains a **properties** object (magnitude, place, time) and a **geometry** object (longitude, latitude, depth).

Example (USGS Earthquake Import).



Download the daily feed, filter events with magnitude > 4, build a Pandas DataFrame, and print the top events.

```
# USGS earthquake feed import
# -----
# Fetches the "all earthquakes, past day" GeoJSON feed,
# converts the features list to a Pandas DataFrame,
# filters events with magnitude > 4, and prints a summary.

import requests
import pandas as pd

URL = ("https://earthquake.usgs.gov/earthquakes/feed/v1.0/"
      "summary/all_day.geojson")

# 1. Robust HTTP call with timeout and error handling
# -----
try:
    resp = requests.get(URL, timeout=15)
    resp.raise_for_status()
except requests.exceptions.Timeout:
    raise SystemExit("USGS server too slow (timeout)")
except requests.exceptions.ConnectionError:
    raise SystemExit("No network or USGS unreachable")
except requests.exceptions.HTTPError as e:
    raise SystemExit(f"HTTP error: {e.response.status_code}")

payload = resp.json()
features = payload["features"]
print(f"Retrieved {len(features)} events.")

# 2. Flatten the GeoJSON into a clean DataFrame
# -----
rows = []
for f in features:
    p = f["properties"]
    lon, lat, depth = f["geometry"]["coordinates"]
    rows.append({
        "time": pd.to_datetime(p["time"], unit="ms"),
        "place": p["place"],
        "mag": p["mag"],
        "depth.km": depth,
        "lon": lon,
        "lat": lat,
    })
df = pd.DataFrame(rows)

# 3. Filter and rank significant events
# -----
```

```

strong = df[df["mag"] > 4.0].sort_values("mag", ascending=False)
print(f"\n{len(strong)} events with M > 4.0 in the past day:")
print(strong.head(10)[["time", "mag", "depth_km", "place"]]
      .to_string(index=False))

# 4. Quick statistics
# -----
print(f"\nMean magnitude (all events) : {df['mag'].mean():.2f}")
print(f"Max magnitude                  : {df['mag'].max():.2f}")
print(f"Mean depth                      : {df['depth_km'].mean():.1f} km")

```

Question 14

1. `r = requests.get(url); r.status_code` returned 200. What does that signal?

2. Which function turns a JSON *string* into a Python dict? _____
3. `pd.DataFrame(list_of_dicts)` produces what?
4. Why should every `requests.get(...)` call include a `timeout=?`

Solution:-

1. Success: the server understood the request and returned the resource.
2. `json.loads()` (or `resp.json()` when you have a `requests` response object).
3. A DataFrame whose columns are the union of all dict keys; each dict becomes one row.
4. Without it, a hung server makes your script wait forever; with it, you get a `Timeout` you can catch.

Common Errors

```

# 1) requests.exceptions.ConnectionError: Failed to establish a new connection
#    Cause: no internet, firewall, or wrong URL.

# 2) json.JSONDecodeError: Expecting value: line 1 column 1
#    Cause: the response was not JSON (e.g. an HTML error page).
#    Fix: print resp.text first to see what came back.

# 3) KeyError: 'features'
#    Cause: the schema you assumed isn't there. Print resp.json().keys()
#    to inspect what the API actually returns.

```

Summary

1. Most engineering data lives behind a URL, not on your disk; `requests` is the bridge.
2. Always pass `timeout=` and call `raise_for_status()` or check `status_code` before parsing.
3. APIs almost always return JSON; `resp.json()` converts it directly to a Python dict/list.
4. A list of dicts → DataFrame in one line via `pd.DataFrame(records)`.
5. Wrap calls in `try/except` for `Timeout`, `ConnectionError`, and `HTTPError`.

6. Real data has surprises (missing keys, unexpected shapes); inspect with `print(resp.json())` before assuming structure.

14.6 Practice (Project Euler)

For those who want extra exercises, Project Euler (projecteuler.net/archives) is a perfect source. The interesting part is not whether your code works, but how efficient it is. Each practice group is assigned one problem; submissions are sorted by CPU time.

Question 15

Smallest Multiple 2520 is the smallest number that can be divided by each of the numbers from 1 to 10 without any remainder. What is the smallest positive number that is evenly divisible by all of the numbers from 1 to 20?

15 Structural Analysis with Python

In this lecture, the demonstration of the structural analysis with Python will be done. The OpenSeesPy is a Python interface to the OpenSees (Open System for Earthquake Engineering Simulation) finite element analysis software. Contrary to most of the commercial software, OpenSeesPy is an open-source and reliable software that is widely used in both academia and industry.

As today, OpenSeesPy works best with Python 3.12. However, the recent version of the Python is 3.13. So first of all, we need to get the Python 3.12. The first option is to setting previous version as default version of Python. The second option is to use virtual environments. In this example, we will use the second option to prevent any conflict with the other Python packages.

```
# Install Python 3.12
winget install Python.Python.3.12
# Check the Python installation
py -3.12 --version
# Create a virtual environment
py -3.12 -m venv myenv312
# Activate the virtual environment
myenv312\Scripts\activate
# Upgrade pip
python -m pip install --upgrade pip
# Install the required packages
pip install openseespy==3.7.1.2 opsvis numpy pandas matplotlib requests scikit-learn
```

After the installation, we can start to use OpenSeesPy. The following code is a simple example of how to use OpenSeesPy to analyze a simple two storey, three bay 2D frame.

```
import openseespy.opensees as ops
import opsvis as opsv

import matplotlib.pyplot as plt

ops.wipe()
ops.model('basic', '-ndm', 2, '-ndf', 3)

colL, girL = 4., 6.

Acol, Agir = 2.e-3, 6.e-3
IzCol, IzGir = 1.6e-5, 5.4e-5

E = 200.e9

Ep = {1: [E, Acol, IzCol],
      2: [E, Acol, IzCol],
      3: [E, Agir, IzGir]}

# Nodes
# Ground floor nodes (y=0)
ops.node(1, 0., 0.)           # Left bottom corner
ops.node(2, girL, 0.)         # Middle bottom
ops.node(3, 2*girL, 0.)       # Middle bottom 2
ops.node(4, 3*girL, 0.)       # Right bottom corner

# 1. floor nodes (y=colL)
ops.node(5, 0., colL)         # Left 1. floor
ops.node(6, girL, colL)       # Middle 1. floor
ops.node(7, 2*girL, colL)     # Middle 1. floor 2
ops.node(8, 3*girL, colL)     # Right 1. floor

# 2. floor nodes (y=2*colL)
ops.node(9, 0., 2*colL)       # Left 2. floor
ops.node(10, girL, 2*colL)    # Middle 2. floor
ops.node(11, 2*girL, 2*colL)  # Middle 2. floor 2
```

```
ops.node(12, 3*girL, 2*colL) # Right 2. floor

# Ground floor nodes fixed
ops.fix(1, 1, 1, 1)
ops.fix(2, 1, 1, 0)
ops.fix(3, 1, 1, 0)
ops.fix(4, 1, 1, 0)

opsv.plot_model()
plt.title('plot.model before defining elements')
plt.savefig('model_before_elements.png', dpi=300, bbox_inches='tight')
plt.clf()

ops.geomTransf('Linear', 1)

# Columns - Ground floor to 1. floor
ops.element('elasticBeamColumn', 1, 1, 5, Acol, E, IzCol, 1) # Left column
ops.element('elasticBeamColumn', 2, 2, 6, Acol, E, IzCol, 1) # Middle column 1
ops.element('elasticBeamColumn', 3, 3, 7, Acol, E, IzCol, 1) # Middle column 2
ops.element('elasticBeamColumn', 4, 4, 8, Acol, E, IzCol, 1) # Right column

# Columns - 1. floor to 2. floor
ops.element('elasticBeamColumn', 5, 5, 9, Acol, E, IzCol, 1) # Left column
ops.element('elasticBeamColumn', 6, 6, 10, Acol, E, IzCol, 1) # Middle column 1
ops.element('elasticBeamColumn', 7, 7, 11, Acol, E, IzCol, 1) # Middle column 2
ops.element('elasticBeamColumn', 8, 8, 12, Acol, E, IzCol, 1) # Right column

# Beams - 1. floor
ops.element('elasticBeamColumn', 9, 5, 6, Agir, E, IzGir, 1) # 1. bay
ops.element('elasticBeamColumn', 10, 6, 7, Agir, E, IzGir, 1) # 2. bay
ops.element('elasticBeamColumn', 11, 7, 8, Agir, E, IzGir, 1) # 3. bay

# Beams - 2. floor
ops.element('elasticBeamColumn', 12, 9, 10, Agir, E, IzGir, 1) # 1. bay
ops.element('elasticBeamColumn', 13, 10, 11, Agir, E, IzGir, 1) # 2. bay
ops.element('elasticBeamColumn', 14, 11, 12, Agir, E, IzGir, 1) # 3. bay

Px = 2.e+3
Wy = -10.e+3
Wx = 0.

# Apply distributed loads to all beams
Ew = {9: ['-beamUniform', Wy, Wx], # 1. floor 1. bay
      10: ['-beamUniform', Wy, Wx], # 1. floor 2. bay
      11: ['-beamUniform', Wy, Wx], # 1. floor 3. bay
      12: ['-beamUniform', Wy, Wx], # 2. floor 1. bay
      13: ['-beamUniform', Wy, Wx], # 2. floor 2. bay
      14: ['-beamUniform', Wy, Wx]} # 2. floor 3. bay

ops.timeSeries('Constant', 1)
ops.pattern('Plain', 1, 1)

# Apply point loads to each floor
ops.load(5, Px, 0., 0.) # Left 1. floor
ops.load(6, Px, 0., 0.) # Middle 1. floor
ops.load(7, Px, 0., 0.) # Middle 1. floor 2
ops.load(8, Px, 0., 0.) # Right 1. floor

ops.load(9, Px, 0., 0.) # Left 2. floor
ops.load(10, Px, 0., 0.) # Middle 2. floor
ops.load(11, Px, 0., 0.) # Middle 2. floor 2
ops.load(12, Px, 0., 0.) # Right 2. floor

for etag in Ew:
    ops.eleLoad('-ele', etag, '-type', Ew[etag][0], Ew[etag][1],
```

```
Ew[etag][2])

ops.constraints('Transformation')
ops.numberer('RCM')
ops.system('BandGeneral')
ops.test('NormDispIncr', 1.0e-6, 6, 2)
ops.algorithm('Linear')
ops.integrator('LoadControl', 1)
ops.analysis('Static')
ops.analyze(1)

ops.printModel()

opsv.plot_model()
plt.title('plot_model after defining elements')
plt.savefig('model_after_elements.png', dpi=300, bbox_inches='tight')
plt.clf()

opsv.plot_load()
plt.savefig('load_diagram.png', dpi=300, bbox_inches='tight')
plt.clf()

opsv.plot_reactions()
plt.savefig('reactions.png', dpi=300, bbox_inches='tight')
plt.clf()

# sfac = 80.

opsv.plot_defo()
plt.savefig('deformation.png', dpi=300, bbox_inches='tight')
plt.clf()
# opsv.plot_defo(sfac)
# fmt_interp = {'color': 'blue', 'linestyle': 'solid', 'linewidth': 1.2, 'marker':
#               '.', 'markersize': 6}
# opsv.plot_defo(sfac, fmt_interp=fmt_interp)

# 4. plot N, V, M forces diagrams

sfacN, sfacV, sfacM = 3.e-5, 3.e-5, 3.e-5

opsv.section_force_diagram_2d('N', sfacN)
plt.title('Axial force distribution')
plt.savefig('axial_force.png', dpi=300, bbox_inches='tight')
plt.clf()

opsv.section_force_diagram_2d('T', sfacV)
plt.title('Shear force distribution')
plt.savefig('shear_force.png', dpi=300, bbox_inches='tight')
plt.clf()

opsv.section_force_diagram_2d('M', sfacM)
plt.title('Bending moment distribution')
plt.savefig('bending_moment.png', dpi=300, bbox_inches='tight')
plt.clf()

print("All figures are saved as PNG:")
print("-- model_before_elements.png")
print("-- model_after_elements.png")
print("-- load_diagram.png")
print("-- reactions.png")
print("-- deformation.png")
print("-- axial_force.png")
print("-- shear_force.png")
print("-- bending_moment.png")
```

```
exit ()
```

Some of the outputs of the analysis can be seen in the following figures.

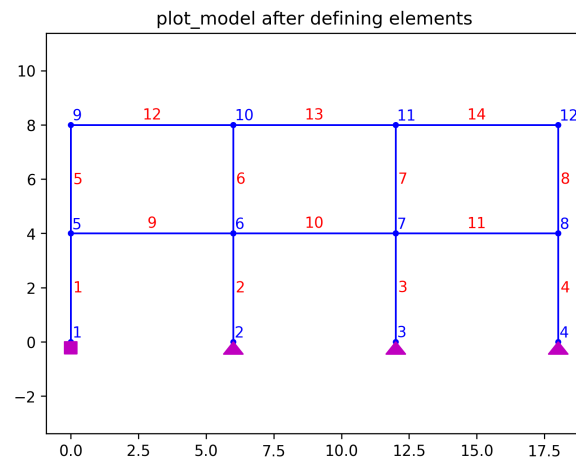


Figure 5: Geometry of the 2D frame.

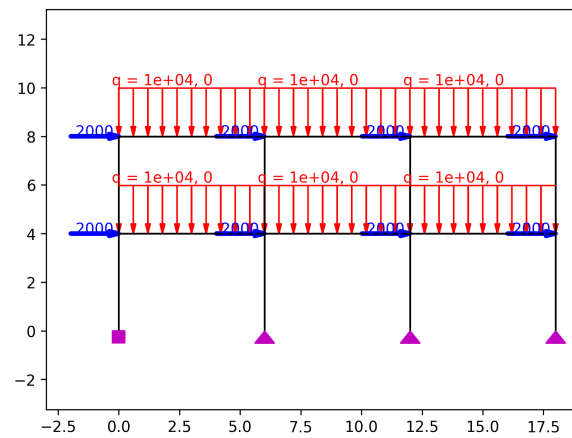


Figure 6: Load diagram of the 2D frame.

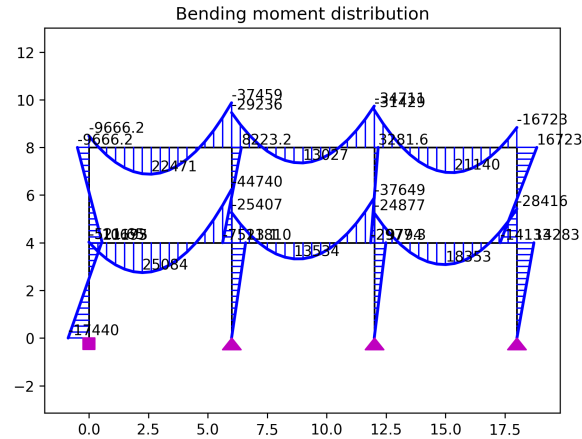


Figure 7: Bending moment diagram of the 2D frame.

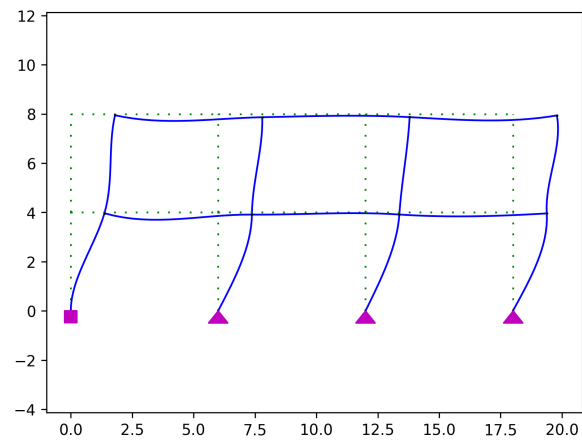


Figure 8: Deformation of the 2D frame.

16 Advanced Programming Concepts

This week introduces concepts that are foundational to understanding how modern software works. While we will not implement complex examples, knowing these ideas will help you read professional code and prepare you for any programming language (C++, Java, C#).

16.1 Memory and Pointers

In earlier weeks, we used variables as simple boxes that hold values. But how does a computer *actually* store data?

16.1.1 Memory as a Grid of Addresses

Computer memory (RAM) can be visualized as a long street of numbered post boxes. Each box has:

- A unique **Address** (e.g., 0x00A1, 0x00A2, ...)
- A **Value** (the actual data)

When you write `int x = 5;` in C# or C++, the computer:

1. Finds an empty box in memory (e.g., address 0x00A1).
2. Writes the value 5 into it.
3. Labels this box as `x` in its internal lookup table.

16.1.2 What is a Pointer?

A **Pointer** is a variable that stores an **Address**, not a value.

Analogy: Imagine you have a note (`ptr`) that says "Go to House #25". The note itself is not the house; it just tells you where the house is.

In C++:

```
int x = 10;           // A normal variable holding 10
int* ptr = &x;        // A pointer holding the ADDRESS of x
                     // & is the "address-of" operator
cout << *ptr;         // Output: 10. * is the "dereference" operator.
                     // It goes to the address and gets the value.
```

16.1.3 Why are Pointers Important?

- **Efficiency:** Instead of copying a gigabyte-sized array, you can just pass its address (a few bytes).
- **Low-Level Access:** Necessary for Operating Systems, Drivers, and Embedded Systems (e.g., a microcontroller in a sensor).
- **Data Structures:** Linked Lists, Trees, and Graphs require pointers to connect nodes.

16.1.4 Pointers in Python?

Python hides pointers from you. When you assign a list to another variable, you are copying a **reference** (an internal pointer), not the data itself.

```
list_a = [1, 2, 3]
list_b = list_a      # list_b now points to the SAME data
list_b[0] = 999
print(list_a)        # Output: [999, 2, 3]. Both changed!
```

This is why we sometimes need `copy.deepcopy()` to create a truly independent copy.

16.2 Object-Oriented Programming (OOP)

Until now, we wrote code as a sequence of instructions. This is **Procedural Programming**. For large projects (e.g., SAP2000, AutoCAD), this becomes unmaintainable. **OOP** organizes code around "Objects" that combine data and behavior.

16.2.1 Core Concept: Class and Object

- **Class:** A **Blueprint**. It defines what an object will look like.
- **Object:** An **Instance**. It is a real thing created from the blueprint.

Analogy: A "House Plan" (Class) vs. an "Actual House" (Object). You can build many houses from one plan.

```
// C# Example
public class Beam {
    public double Length; // Property (Data)
    public double Depth;

    // Method (Behavior)
    public double GetMomentOfInertia(double width) {
        return (width * Math.Pow(Depth, 3)) / 12;
    }
}

// Creating Objects (Instances)
Beam beam1 = new Beam();
beam1.Length = 5.0;
beam1.Depth = 0.5;
```

16.3 The Four Pillars of OOP

16.3.1 1. Encapsulation

Hiding the internal details of an object. You only expose what is necessary.

Analogy: A **Car**. You use the steering wheel and pedals (the public interface). You do not directly manipulate the fuel injectors (private internals).

In code, we use **private** and **public** keywords.

```
public class BankAccount {
    private double balance; // Cannot be accessed from outside

    public void Deposit(double amount) { // Public method
        if (amount > 0) balance += amount;
    }
}
```

16.3.2 2. Inheritance

Creating a **new class based on an existing one**. The new class inherits properties and methods from the parent.

Analogy: A **T-Beam** is a special type of **Beam**. It has everything a Beam has, plus a flange.

```
public class TBeam : Beam { // TBeam inherits from Beam
    public double FlangeWidth;

    // Override: We can change parent behavior
    public new double GetMomentOfInertia(double web) {
        // A more complex formula for T-sections
        return base.GetMomentOfInertia(web) + (FlangeWidth * 0.1);
    }
}
```

16.3.3 3. Polymorphism

"Many Forms". The **same command can behave differently** depending on the object type.

Analogy: The command "Draw yourself" given to different shapes. A **Circle** draws a circle. A **Rectangle** draws a rectangle. Same command, different results.

In code, this is achieved through method **overriding** (as shown above) or **interfaces**.

16.3.4 4. Abstraction

Focusing on **what an object does**, not **how it does it**.

Analogy: When you call `beam.CalculateDeflection()`, you don't care about the thousands of lines of Finite Element code running behind the scenes. You just want the answer.

In C#, this is often done with **abstract** classes or **interfaces**:

```
public abstract class StructuralElement {  
    public abstract double GetWeight(); // No implementation here  
}  
  
public class Column : StructuralElement {  
    public double Height;  
    public override double GetWeight() {  
        return Height * 2400 * 0.09; // Concrete density * area  
    }  
}
```

Note

Why OOP Matters for Engineers: Commercial engineering software (ETABS, ANSYS, Revit) are built with OOP. When you use APIs to automate these programs, you are interacting with their Objects (Nodes, Elements, LoadCases). Understanding OOP makes automation much easier.

16.4 Recursion

Recursion is when a function calls **itself**.

Analogy: Imagine a set of Russian Dolls (Matryoshka). To find the smallest doll, you open a doll, find another doll inside, and repeat until you reach the smallest one (the "base case").

```
def factorial(n):  
    if n == 1:  
        return 1 # Base Case: Stop here  
    else:  
        return n * factorial(n - 1) # Recursive Call  
  
print(factorial(5)) # 5 * 4 * 3 * 2 * 1 = 120
```

Engineering Use: Tree/Graph traversal algorithms, some optimization methods (Divide and Conquer).

Note

Warning: Every recursive function **MUST** have a **Base Case**. Otherwise, it will call itself forever and crash (Stack Overflow error).

16.5 Algorithm Complexity (Big O Notation)

Not all algorithms are equal. Some are fast, some are slow. **Big O Notation** describes how the runtime of an algorithm grows as the input size increases.

- **O(1) - Constant:** The operation takes the same time regardless of input size. (e.g., accessing an array element by index).
- **O(n) - Linear:** Time grows proportionally with input. (e.g., searching an unsorted list).
- **O(n²) - Quadratic:** Time grows with the square of the input. (e.g., Bubble Sort). **Avoid for large data!**
- **O(log n) - Logarithmic:** Extremely efficient. (e.g., Binary Search).

Analogy: Finding a name in a phone book.

- $O(n)$: Reading every page from the start ("linear search").
- $O(\log n)$: Opening the book in the middle, deciding if the name is before or after, then repeating ("binary search").

Why it matters: A $O(n^2)$ algorithm on 1,000 data points takes 1 million operations. On 1 million data points, it takes 1 **trillion** operations. Knowing this prevents you from writing code that never finishes.

16.6 Parallelism and Multithreading

Modern computers have multiple CPU cores. **Parallelism** means running multiple tasks at the same time on different cores.

Analogy: Building a house.

- **Sequential:** One worker does the foundation, then the walls, then the roof. Slow.
- **Parallel:** One team does foundation, another prepares wall materials simultaneously. Faster.

Engineering Use: Finite Element Analysis solvers (ANSYS, Abaqus) use parallelism to solve large stiffness matrices. Monte Carlo simulations run thousands of scenarios in parallel.

Note

Parallelism is powerful but tricky. If two "workers" try to modify the same data at the same time, you get a **Race Condition** bug, leading to unpredictable results.

16.7 Version Control (Git)

Version Control is a system that records changes to files over time. **Git** is the industry standard.

Analogy: An "Undo History" for your entire project, but you can also branch into alternate realities and merge them back.

Key Concepts:

- **Repository (Repo):** The project folder tracked by Git.
- **Commit:** A snapshot of the project at a point in time. Like saving a game.
- **Branch:** A parallel version of the code. Used for developing new features without breaking the main code.
- **Merge:** Combining changes from one branch into another.
- **GitHub/GitLab:** Online platforms to host repositories and collaborate with others.

Why it matters: "My code was working yesterday, now it's broken" is solved by Git. You can always go back to a working version. It is also essential for team projects.

16.8 APIs (Application Programming Interfaces)

An **API** is a set of rules that allows one piece of software to talk to another.

Analogy: A **Waiter** in a restaurant. You (the customer/your code) don't go into the kitchen (the software's internal code). You tell the waiter (the API) what you want, and they bring you the result.

Engineering Examples:

- **SAP2000 OAPI:** Allows Python or C# to create models, run analysis, and read results programmatically.
- **Revit API:** Automates BIM modeling tasks.
- **Web APIs:** Get earthquake data from USGS, weather data for site planning, etc.

When you write `import openseespy`, you are using an API that talks to the underlying OpenSees engine.

16.9 Debugging and Testing

Debugging is the process of finding and fixing errors in code.

16.9.1 Common Debugging Techniques

- **Print Statements:** The oldest trick. Print variable values to see what's happening.
- **Debugger Tools:** IDEs (VS Code, PyCharm) let you pause execution ("breakpoints"), step through code line by line, and inspect variables.
- **Rubber Duck Debugging:** Explain your code line by line to an inanimate object (or a colleague). Often, you find the bug while explaining.

16.9.2 Testing

Professional software includes **Unit Tests**: small functions that check if individual pieces of code work correctly.

```
# A simple unit test
def test_factorial():
    assert factorial(5) == 120
    assert factorial(1) == 1
    print("All tests passed!")
```

Why it matters: Code without tests is like a building without inspection. You *think* it's correct, but you don't *know*.

Appendix — Quick Reference

This appendix collects reference tables that you will use again and again. Keep this section bookmarked — it is meant to replace looking up syntax on external websites.

A. String Methods

Strings in Python are immutable: every method returns a **new** string. The original is untouched.

Method	Example	Result
<code>upper()</code>	<code>"steel".upper()</code>	<code>"STEEL"</code>
<code>lower()</code>	<code>"M30".lower()</code>	<code>"m30"</code>
<code>title()</code>	<code>"yield stress".title()</code>	<code>"Yield Stress"</code>
<code>strip()</code>	<code>" C25 \n".strip()</code>	<code>"C25"</code>
<code>lstrip()</code>	<code>" C25".lstrip()</code>	<code>"C25"</code>
<code>rstrip()</code>	<code>"C25 ".rstrip()</code>	<code>"C25"</code>
<code>split(sep)</code>	<code>"a,b,c".split(",")</code>	<code>['a', 'b', 'c']</code>
<code>join(iter)</code>	<code>", ".join(['a', 'b'])</code>	<code>"a,b"</code>
<code>replace(a,b)</code>	<code>"M30".replace("M", "C")</code>	<code>"C30"</code>
<code>startswith(s)</code>	<code>"Beam-1".startswith("B")</code>	<code>True</code>
<code>endswith(s)</code>	<code>"data.csv".endswith(".csv")</code>	<code>True</code>
<code>find(s)</code>	<code>"abc".find("b")</code>	<code>1 (or -1)</code>
<code>count(s)</code>	<code>"banana".count("a")</code>	<code>3</code>
<code>zfill(n)</code>	<code>"42".zfill(5)</code>	<code>"00042"</code>
<code>isdigit()</code>	<code>"42".isdigit()</code>	<code>True</code>
<code>isalpha()</code>	<code>"abc".isalpha()</code>	<code>True</code>
<code>format()</code>	<code>"{} kN".format(50)</code>	<code>"50 kN"</code>

B. List, Tuple, Set, Dict Methods

B.1 List methods (mutable)

Method	Example	Effect
<code>append(x)</code>	<code>loads.append(50)</code>	Add to end
<code>extend(it)</code>	<code>loads.extend([1,2])</code>	Add many to end
<code>insert(i,x)</code>	<code>loads.insert(0, 99)</code>	Insert at index
<code>remove(x)</code>	<code>loads.remove(50)</code>	Remove first match
<code>pop(i)</code>	<code>loads.pop()</code>	Remove and return last
<code>index(x)</code>	<code>loads.index(50)</code>	First position of x
<code>count(x)</code>	<code>loads.count(0)</code>	How many times x appears
<code>sort()</code>	<code>loads.sort()</code>	In-place ascending sort
<code>sort(reverse=True)</code>	<code>loads.sort(reverse=True)</code>	In-place descending
<code>reverse()</code>	<code>loads.reverse()</code>	In-place reverse
<code>clear()</code>	<code>loads.clear()</code>	Remove all items
<code>copy()</code>	<code>loads.copy()</code>	Shallow copy

B.2 Tuple (immutable) — only two methods

`count(x)` and `index(x)`. Everything else would mutate, which tuples forbid.

B.3 Set methods

Method / Operator	Example	Result
<code>add(x)</code>	<code>s.add(5)</code>	Insert if absent
<code>remove(x)</code>	<code>s.remove(5)</code>	Error if missing
<code>discard(x)</code>	<code>s.discard(5)</code>	Silent if missing
<code> </code> (union)	<code>{1,2} {2,3}</code>	<code>{1,2,3}</code>
<code>&</code> (intersection)	<code>{1,2} & {2,3}</code>	<code>{2}</code>
<code>-</code> (difference)	<code>{1,2,3} - {2}</code>	<code>{1,3}</code>
<code>^</code> (sym. diff.)	<code>{1,2} ^ {2,3}</code>	<code>{1,3}</code>
<code>in</code>	<code>5 in s</code>	True/False (O(1)!)

B.4 Dict methods

Method	Example	Result
<code>keys()</code>	<code>d.keys()</code>	View of keys
<code>values()</code>	<code>d.values()</code>	View of values
<code>items()</code>	<code>d.items()</code>	View of (key,value) pairs
<code>get(k,default)</code>	<code>d.get("E", 0)</code>	Value or default (no error)
<code>pop(k)</code>	<code>d.pop("I")</code>	Remove and return
<code>update(d2)</code>	<code>d.update({"v":0.3})</code>	Merge another dict
<code>setdefault(k,v)</code>	<code>d.setdefault("rho", 2400)</code>	Add if missing

C. f-string Format Mini-Language

Inside an f-string, after the variable name and a colon you can specify formatting: `f"{value:[fill][align][sign][#][0]`

Spec	Example	Result
<code>:.2f</code>	<code>f"{3.14159:.2f}"</code>	"3.14"
<code>:.3e</code>	<code>f"{210000:.3e}"</code>	"2.100e+05"
<code>:, .2f</code>	<code>f"{1234567.89: ,.2f}"</code>	"1,234,567.89"
<code>:.1%</code>	<code>f"{0.075:.1%}"</code>	"7.5%"
<code>>10</code>	<code>f"{'A':>10}"</code>	"_____A" (right-aligned, width 10)
<code><10</code>	<code>f"{'A':<10}"</code>	"A " (left-aligned)
<code>^10</code>	<code>f"{'A':^10}"</code>	Centered, width 10
<code>:05d</code>	<code>f"{42:05d}"</code>	"00042"
<code>:08.3f</code>	<code>f"{3.14:08.3f}"</code>	"0003.140"
<code>:b</code>	<code>f"{10:b}"</code>	"1010" (binary)
<code>:x</code>	<code>f"{255:x}"</code>	"ff" (hex)
<code>:o</code>	<code>f"{8:o}"</code>	"10" (octal)
<code>!r</code>	<code>f"{name!r}"</code>	Calls <code>repr()</code> (adds quotes)

D. NumPy / Pandas Quick Reference

D.1 NumPy (import numpy as np)

Function	Purpose
<code>np.array([...])</code>	Build an array from a list
<code>np.zeros((n,m))</code>	$n \times m$ zero matrix
<code>np.ones((n,m))</code>	Matrix of ones
<code>np.eye(n)</code>	Identity matrix $n \times n$
<code>np.linspace(a,b,N)</code>	N evenly spaced points in $[a, b]$
<code>np.arange(a,b,step)</code>	Like <code>range()</code> but for floats
<code>a.shape</code>	Tuple of dimensions
<code>a.reshape((r,c))</code>	Change shape without copying
<code>a.flatten()</code>	Convert to 1D
<code>a.T</code>	Transpose
<code>a @ b</code> or <code>np.dot(a,b)</code>	Matrix product
<code>a[a > 0]</code>	Boolean filtering
<code>a.sum(axis=0)</code>	Column sums (axis=1: row sums)
<code>a.mean()</code> , <code>a.std()</code>	Mean / standard deviation
<code>np.linalg.solve(K, F)</code>	Solve $Kx = F$
<code>np.linalg.inv(A)</code>	Matrix inverse

D.2 Pandas (import pandas as pd)

Call	Purpose
<code>pd.read_csv("f.csv")</code>	Load CSV into DataFrame
<code>pd.read_excel("f.xlsx")</code>	Load Excel
<code>df.head(n)</code>	First n rows
<code>df.describe()</code>	Statistics summary
<code>df.info()</code>	Types and non-null counts
<code>df["col"]</code> or <code>df.col</code>	One column (Series)
<code>df[["a","b"]]</code>	Multiple columns
<code>df.iloc[i,j]</code>	Positional access (row i, col j)
<code>df.loc[label,"col"]</code>	Label-based access
<code>df[df.X > 100]</code>	Boolean filter rows
<code>df.groupby("cat").mean()</code>	Group then aggregate
<code>df.sort_values("col")</code>	Sort rows by column
<code>df.dropna()</code>	Drop rows containing NaN
<code>df.fillna(0)</code>	Replace NaN
<code>df.apply(func)</code>	Apply function column-wise
<code>df.to_csv("out.csv")</code>	Save to CSV

E. Common Python Errors — Diagnostic Dictionary

When Python prints a red traceback, the *last line* is the error type. Match it here:

Error	Typical cause and fix
<code>SyntaxError</code>	Misspelled keyword, missing colon, unclosed bracket. Fix: re-read the line marked by <code>^</code> .
<code>IndentationError</code>	Inconsistent spaces/tabs after <code>:</code> . Fix: use 4 spaces throughout.
<code>NameError</code>	Variable used before being assigned, or typo. Fix: check spelling and scope.
<code>TypeError</code>	Operation between incompatible types (e.g. <code>"5"+5</code>). Fix: cast with <code>int()/str()</code> .
<code>ValueError</code>	Right type, wrong value (e.g. <code>int("2.5")</code>). Fix: pre-validate or use <code>float()</code> first.
<code>IndexError</code>	Index out of list bounds. Fix: check <code>len(list)</code> before indexing.
<code>KeyError</code>	Dict key missing. Fix: use <code>d.get(k, default)</code> or <code>if k in d:</code> .
<code>AttributeError</code>	Method/attribute does not exist on this object (e.g. <code>"abc".append("x")</code>). Fix: check <code>type(obj)</code> .
<code>ZeroDivisionError</code>	Division by zero. Fix: guard with <code>if denom != 0:</code> .
<code>FileNotFoundError</code>	Path does not exist. Fix: print <code>os.getcwd()</code> and verify path.
<code>ModuleNotFoundError</code>	Library not installed in this environment. Fix: <code>pip install <name></code> .
<code>UnboundLocalError</code>	Assigning to a name inside a function shadows the global. Fix: use <code>global x</code> or rename.
<code>RecursionError</code>	A recursive function with no base case. Fix: add <code>if n == 0: return</code>
<code>json.JSONDecodeError</code>	Malformed JSON. Fix: validate the file in an online JSON viewer.
<code>UnicodeDecodeError</code>	Wrong text encoding. Fix: <code>open(f, encoding="utf-8")</code> .

F. Programming Glossary (TR–EN)

Türkçe	English	Türkçe	English
Değişken	Variable	Tür / Tip	Type
Atama	Assignment	Tür dönüşümü	Type casting
Yorum satırı	Comment	Sabit	Constant
İfade	Expression	Deyim / Komut	Statement
Operatör	Operator	Operand	Operand
Koşul	Condition	Döngü	Loop
Akış denetimi	Control flow	Yineleme	Iteration
Fonksiyon	Function	Argüman / Parametre	Argument / Parameter
Geri dönüş değeri	Return value	Kapsam (yerel/küresel)	Scope (local/global)
Liste	List	Demet	Tuple
Sözlük	Dictionary	Küme	Set
İndeks	Index	Dilimleme	Slicing
Anahtar	Key	Değer	Value
Modül	Module	Kütüphane	Library / Package
İçe aktarma	Import	Paket yöneticisi	Package manager
Hata	Error / Exception	İstisna yakalama	Exception handling
Hata ayıklama	Debugging	Kesme noktası	Breakpoint
Derleyici	Compiler	Yorumlayıcı	Interpreter
Söz dizimi	Syntax	Anlambilim	Semantics
Yığın	Stack	Kuyruk	Queue
Dosya açma kipi	File open mode	Dosya tanıtıcı	File handle

Notes